

Project Vitriol – NEA Testing

1 Testing Strategy Overview

Testing was conducted to verify that the implemented system satisfies the functional and non-functional objectives defined in the Analysis section. The purpose of this stage is not simply to demonstrate that the program runs, but to show that it behaves correctly under normal, boundary, and abnormal conditions.

Testing was structured into:

- **Internal Logic Verification** – confirming that core calculations and control logic (indexing, command execution, viewport calculations, camera behaviour, serialisation) operate deterministically.
- **Functional Subsystem Testing** – validating that observable behaviours (map creation, editing, persistence, runtime rendering) align with specified requirements.
- **Boundary and Abnormal Data Testing** – evaluating validation rules and system resilience when exposed to extreme or invalid input.
- **Performance Observation** – assessing rendering behaviour and responsiveness under larger map conditions.

Where issues were identified, corrective modifications were applied and the affected subsystem was retested to confirm stability. Representative samples are presented below to demonstrate that the system fulfils its intended purpose.

2 Subsystem Testing

2.1 Map Creation and Validation

This subsystem verifies that map initialisation follows the defined structural rules and that invalid configurations are rejected prior to persistence. It directly supports objectives relating to data integrity, autosave behaviour, and safe project management.

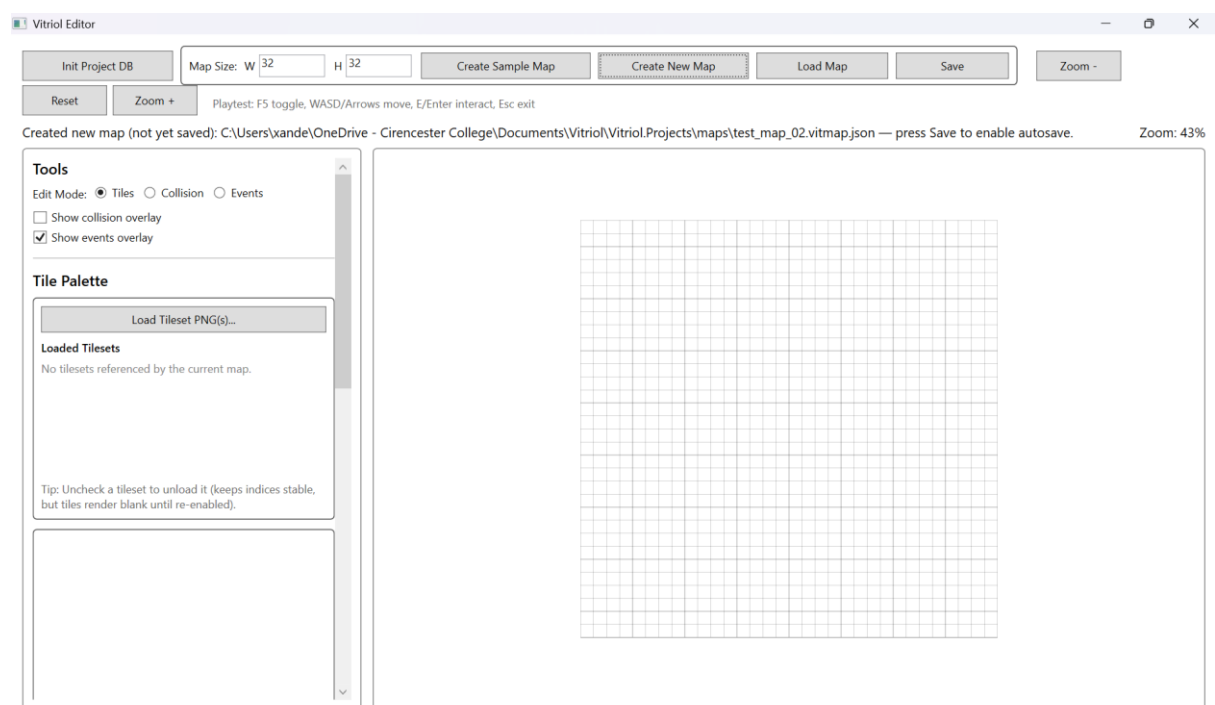
Test Table 1 – Map Creation & Validation

Test ID	Purpose of Test	Test Data	Expected Outcome	Actual Outcome	Screenshot ID
MC1	Verify new map creates valid empty grid	Width = 32 Height = 32	1024-tile grid initialised with default values	Pass	Figures T1
MC2	Validate manual save required before autosave	Create new map -> modify tile without saving	Autosave disabled until first manual save	Pass	Figures T2.1, T2.2

MC3	Boundary test – minimum size	Width = 1 Height = 1	Valid 1x1 grid created without error	Pass	Figures T3
MC4	Boundary test – large size	Width = 300 Height = 300	Map created successfully. Viewport rendering prevents performance degradation	Pass	Figures T4
MC5	Abnormal input – negative size	Width = -10	Map creation rejected with validation error	Pass	Figures T5
MC6	Abnormal input – zero dimension	Width = 0	Validation prevents creation	Pass	Figures T6

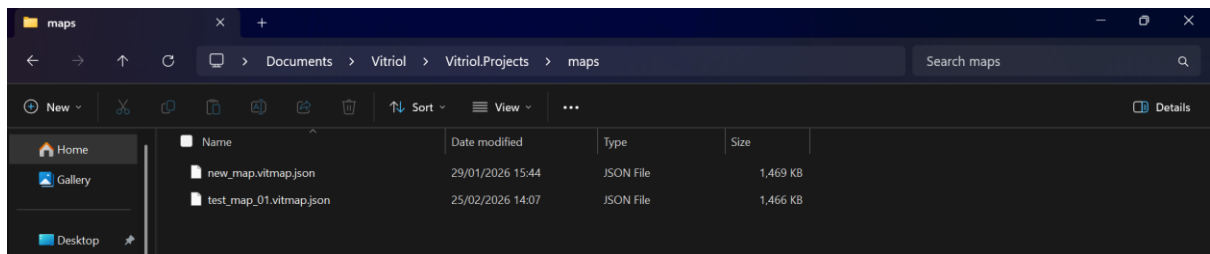
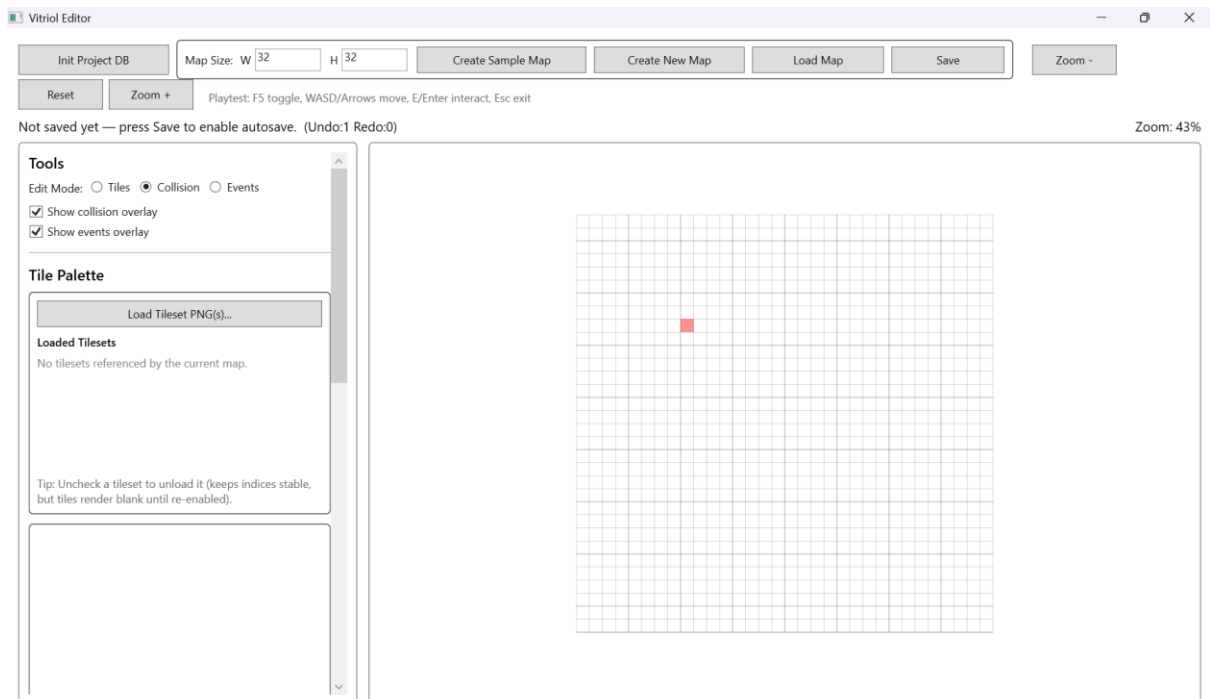
Test Table 1 – Screenshot Evidence

Figure T1 - Standard Map Creation (32×32 Grid)



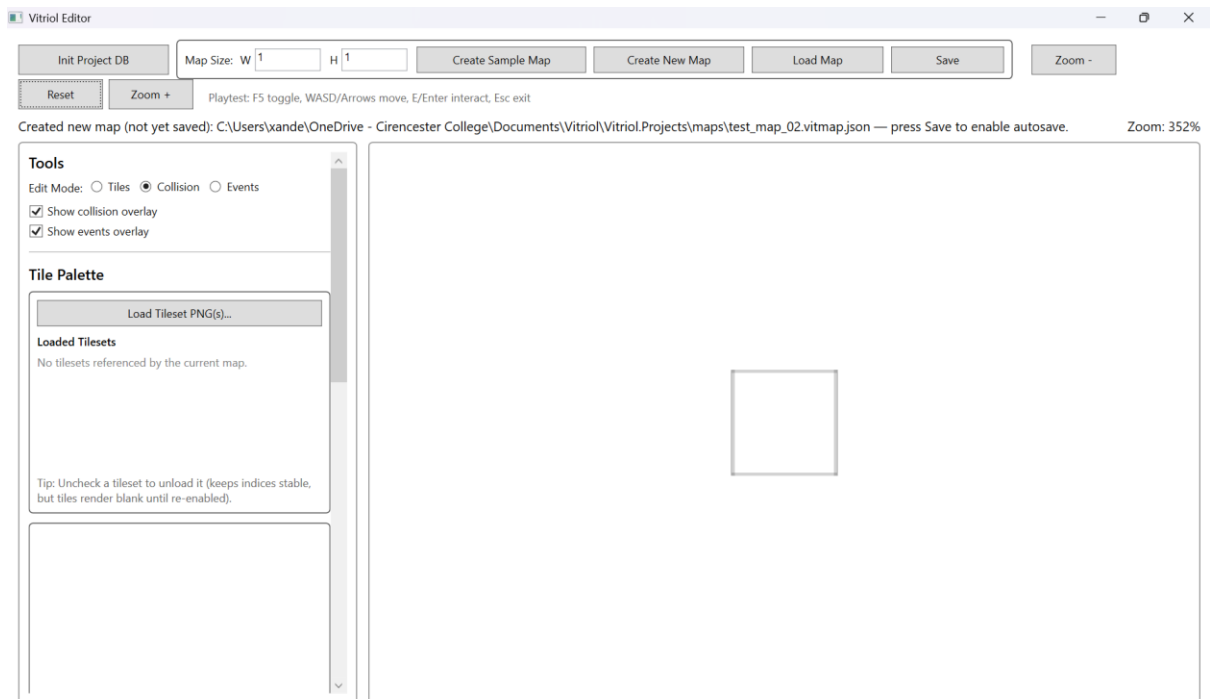
Demonstrates successful creation of a default-sized map with correctly initialised tile grid.

Figure T2.1, T2.2 – Autosave Disabled Prior to First Manual Save



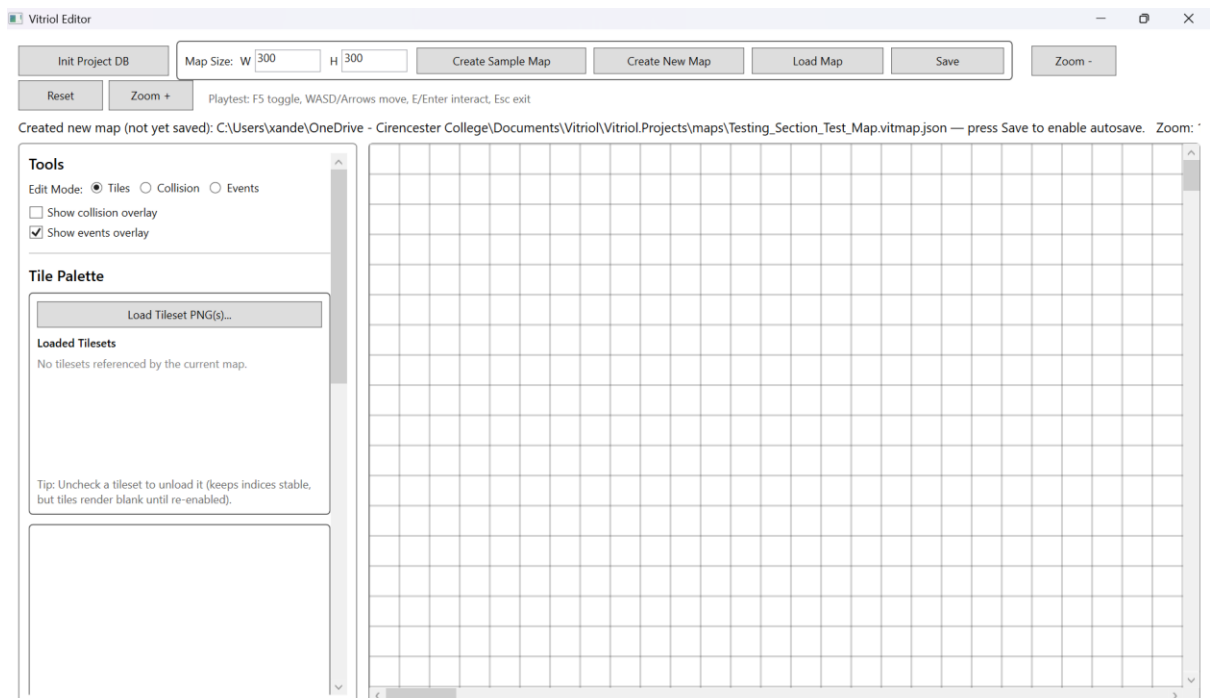
Demonstrates that modifying a newly created map does not trigger autosave before an explicit manual save is performed.

Figure T3 – Minimum Map Dimension Boundary Test (1×1 Grid)



Demonstrates correct handling of the smallest valid map configuration without runtime instability.

Figure T4 – Large Map Creation (300×300) with Viewport Rendering Active



Demonstrates stable creation and navigation of a large map using region-based viewport rendering.

Figure T5 – Negative Dimension Validation Rejection

Demonstrates system rejection of invalid negative dimension input prior to grid allocation.

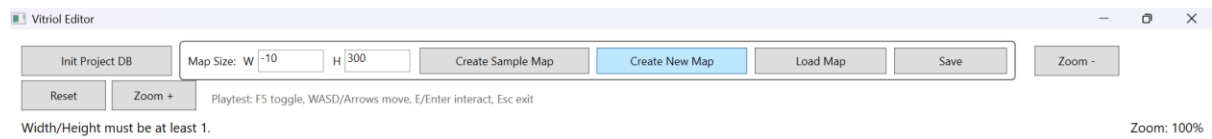
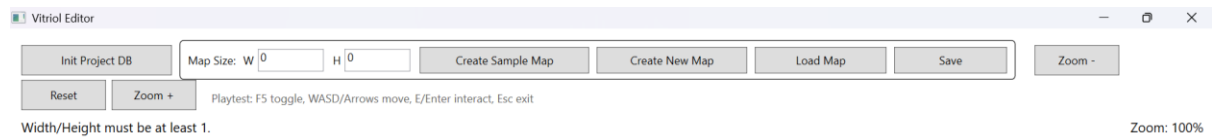


Figure T6 – Zero Dimension Validation Handling



Demonstrates prevention of zero-sized map creation through input validation logic.

2.2 Tile Editing and Undo/Redo

This subsystem verifies that editing operations correctly modify the underlying data model and that the Undo/Redo mechanism reliably restores previous states without introducing inconsistencies.

The command-based editing model was tested to ensure:

- Each paint operation produces exactly one reversible command
- Undo restores the previous tile state
- Redo reapplies the change deterministically
- New edits correctly clear the redo stack
- Autosave behaviour does not interfere with command execution

Test Table 2 – Tile Editing & Undo/Redo

Test ID	Purpose of Test	Test Data	Expected Outcome	Actual Outcome	Screenshot ID
UR1	Verify tile paint modifies correct grid index	Paint tile at (5,5)	Tile at (5,5) updated only	Pass	Figures T7.1, T7.2
UR2	Verify Undo restores previous state	Paint tile -> Undo	Tile returns to original value	Pass	Figures T8.1, T8.2
UR3	Verify Redo reapplies change	Undo -> Redo	Tile updated again	Pass	Figures T9.1, T9.2
UR4	Verify redo stack clears after new action	Paint -> Undo -> Paint new tile	Redo unavailable	Pass	Figures T10.1, T10.2
UR5	Multi-step undo stability	Paint 3 tiles -> Undo 3 times	Grid returns to original state	Pass	Figures T11.1, T11.2, T11.3, T11.4

Test Table 2 – Screenshot Evidence

Figure T7.1 – Tile Grid Before Paint Operation at Coordinate (5,5)

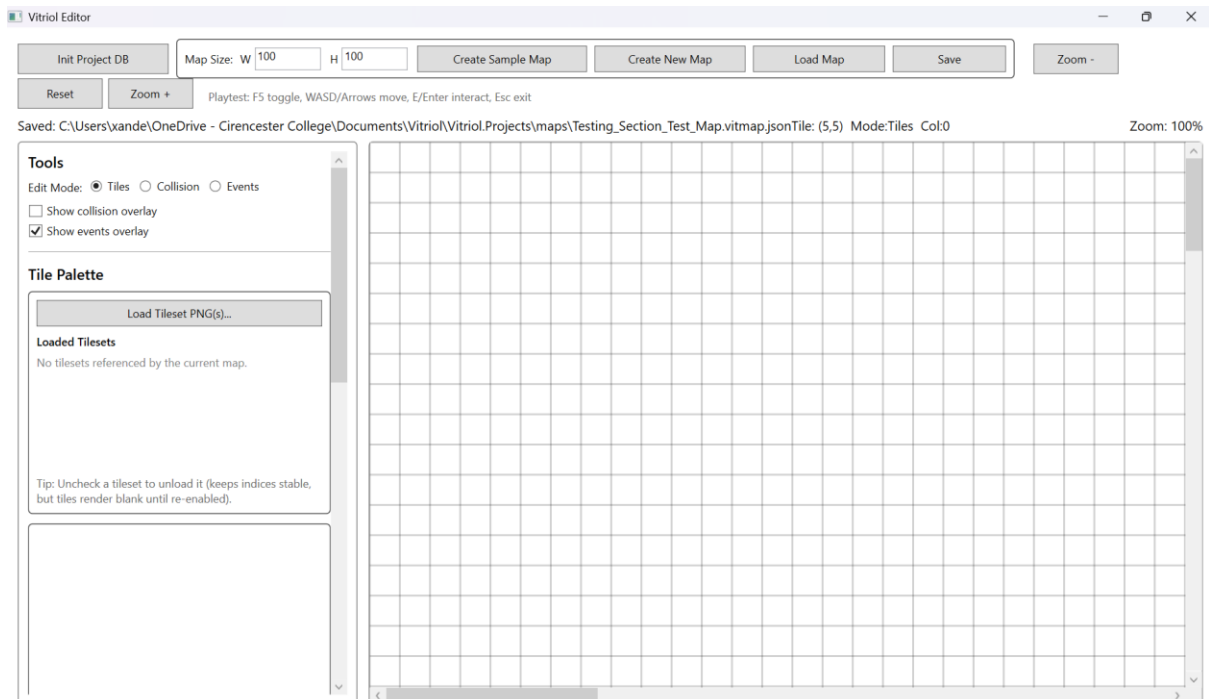
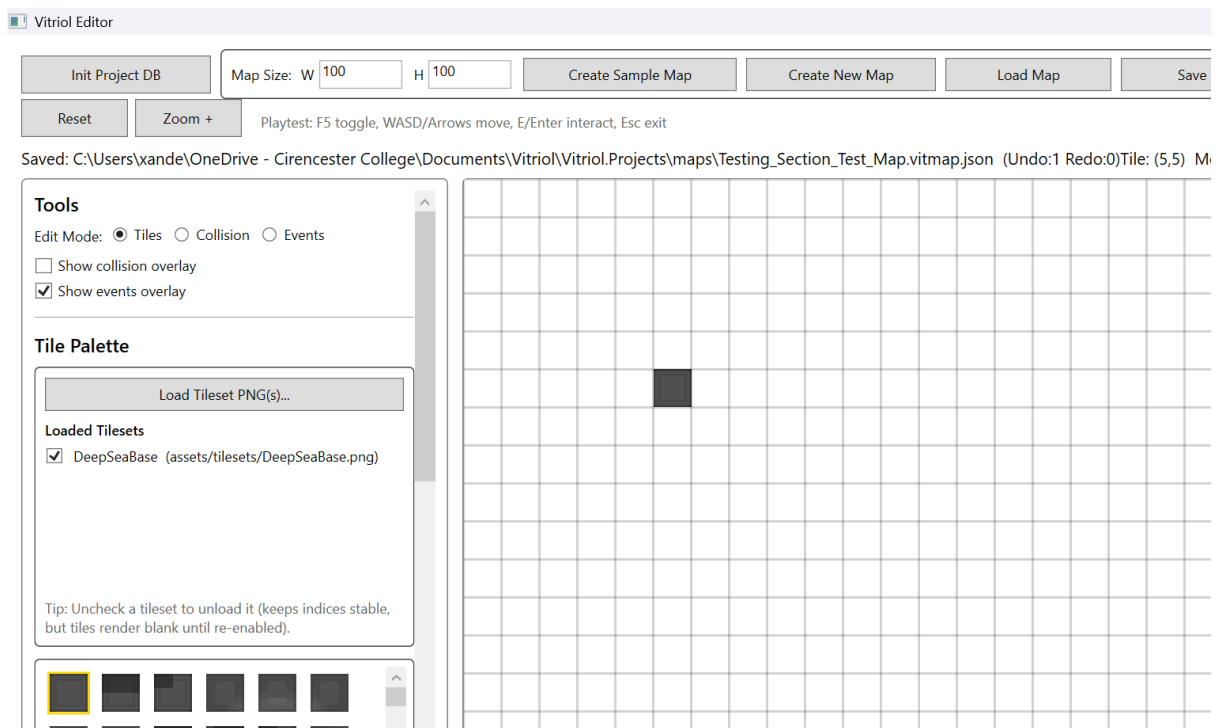


Figure T7.2 – Tile Grid After Paint Operation at Coordinate (5,5)



Demonstrates that the paint command updates only the intended grid coordinate, confirming correct 2D-to-1D index conversion logic and isolated state mutations.

Figure T8.1 – Tile State Immediately After Paint operation

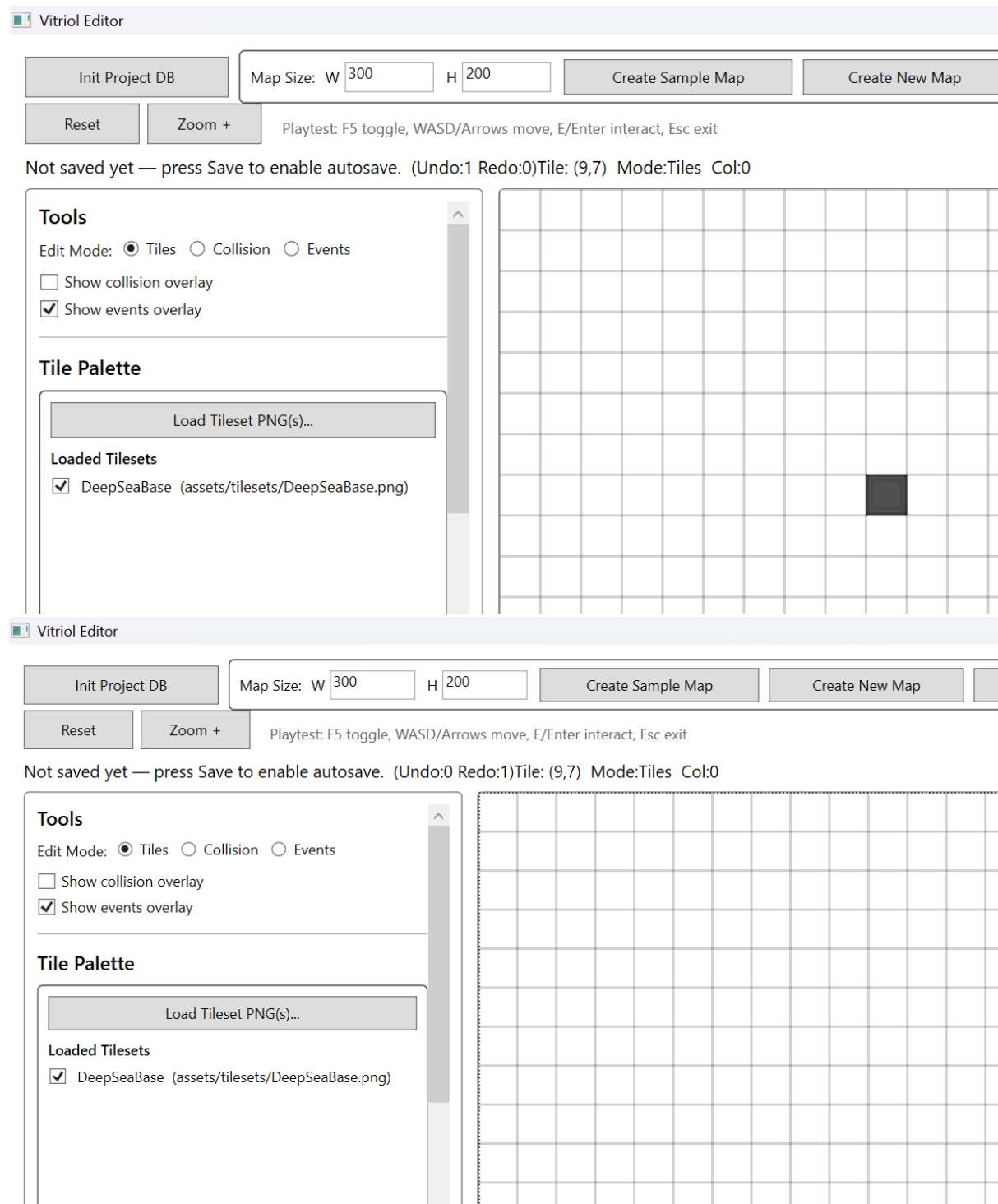


Figure T8.2 Tile State After Undo Invocation (Ctrl+Z)

Demonstrates that Undo restores the previous tile value without affecting unrelated grid cells, validating reversible command execution

Figure T9.1 – Tile State after Undo Operation

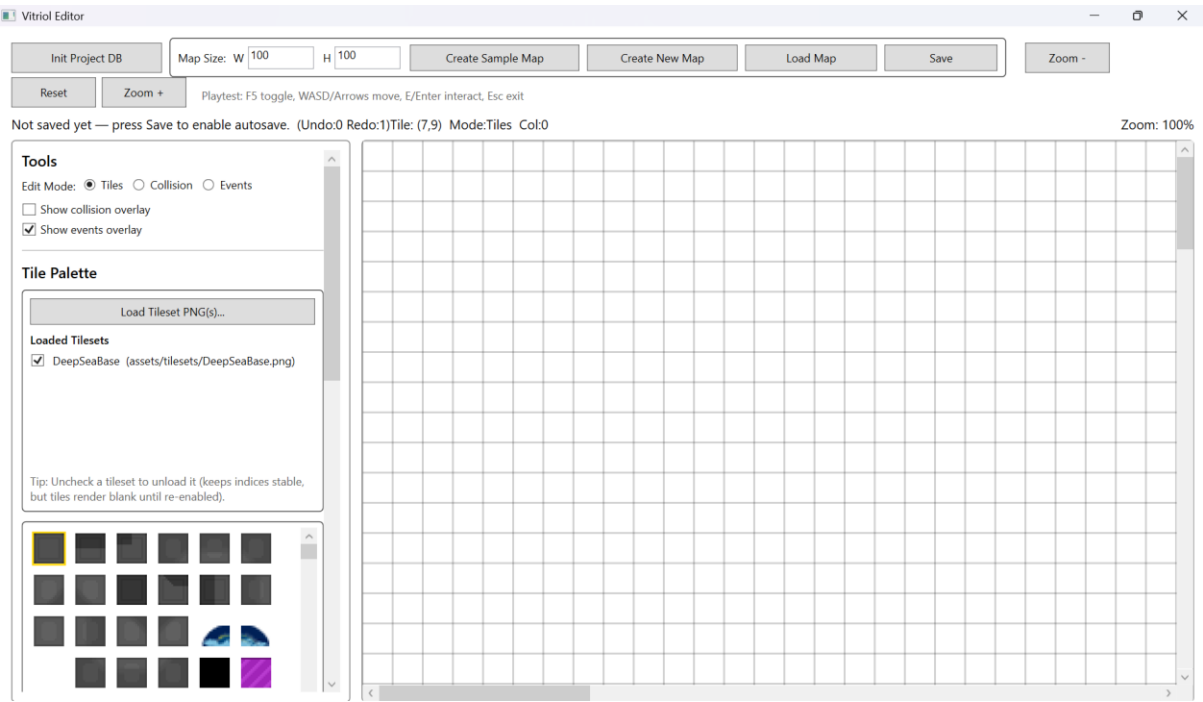
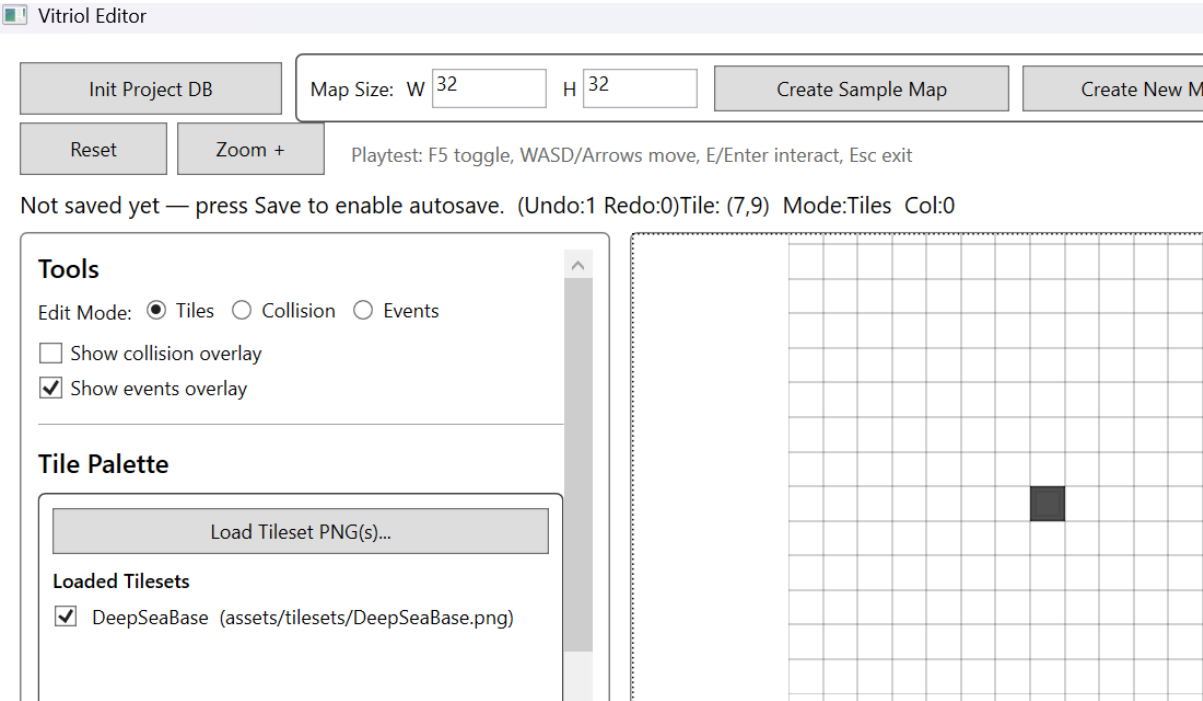


Figure T9.2 – Tile State after Redo Invocation (Ctrl+Y)



Demonstrates deterministic reapplication of a previously undone command, confirming symmetrical execution behaviour

Figure T10.1 – Tile State After Undo Operation

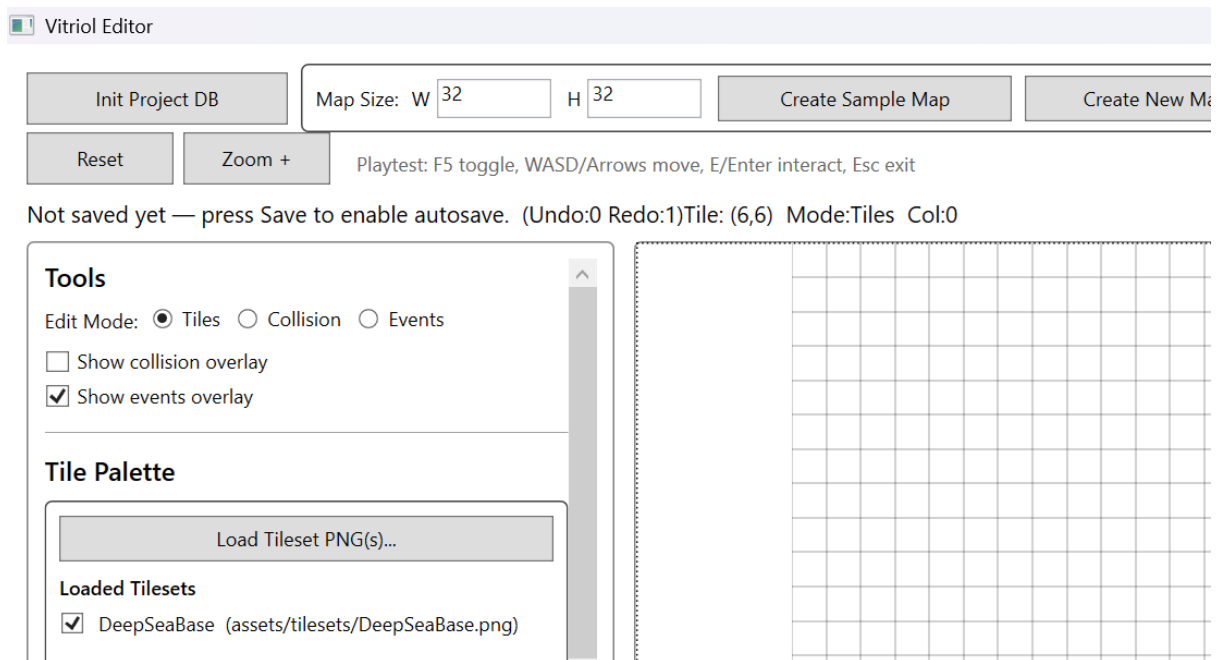
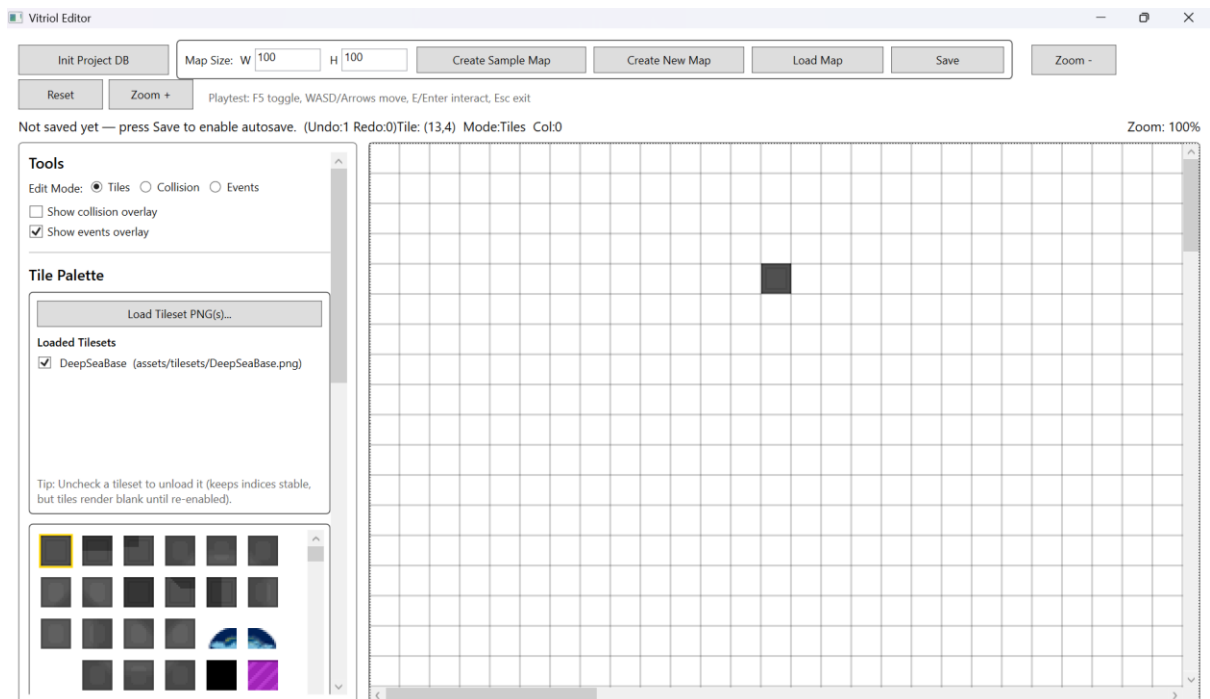


Figure 10.2 – Tile State After New Paint Action (Redo Attempt Blocked)



Demonstrates that performing a new edit after Undo correctly clear the redo stack, preventing invalid state replay

Figure T11.1 Grid After 3 Paint Operations

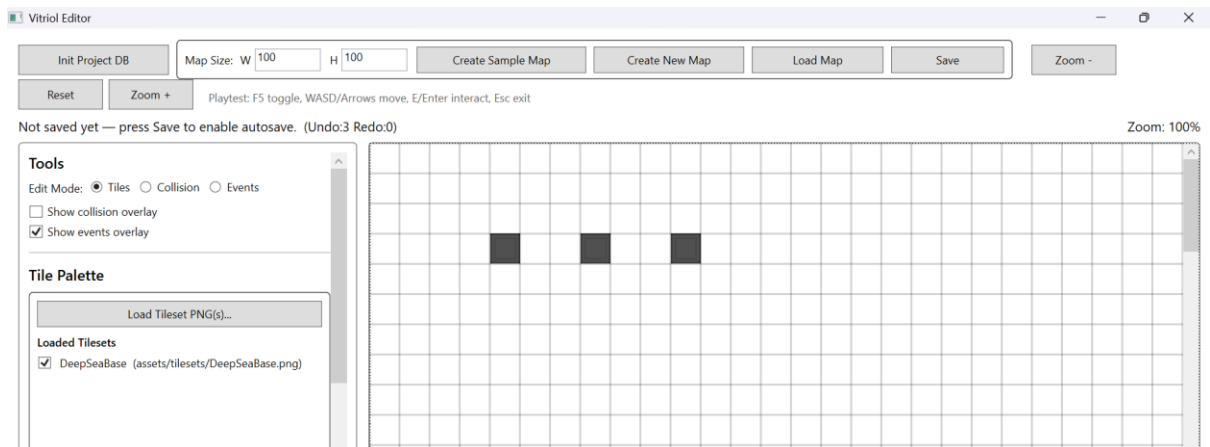


Figure T11.2 – Grid After 1st Undo Operation

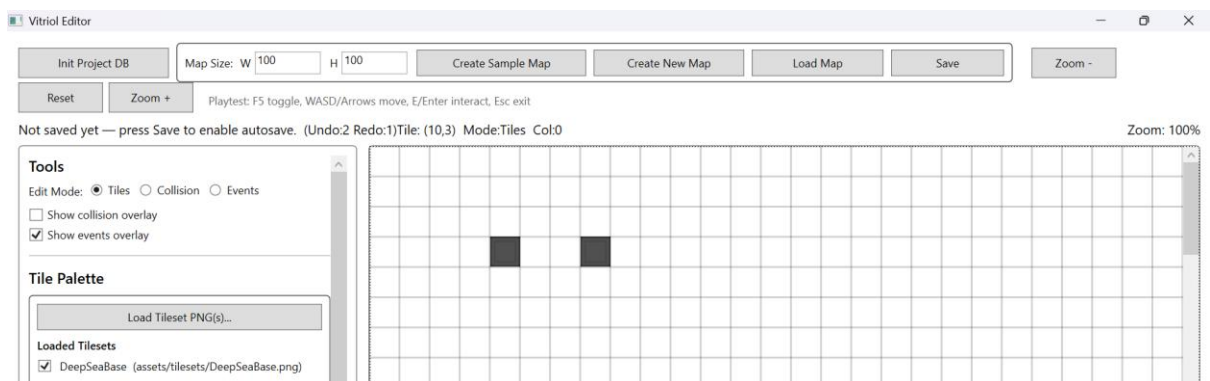


Figure T11.3 – Grid After 2nd Undo Operation

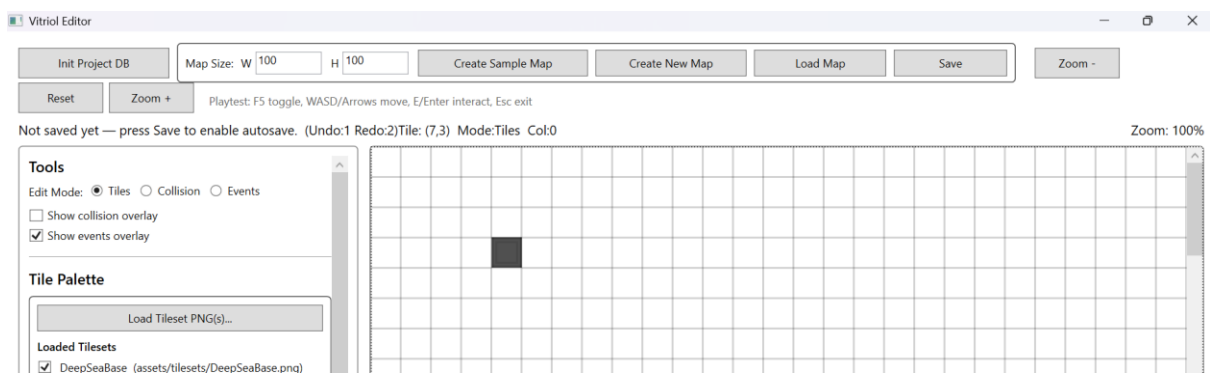
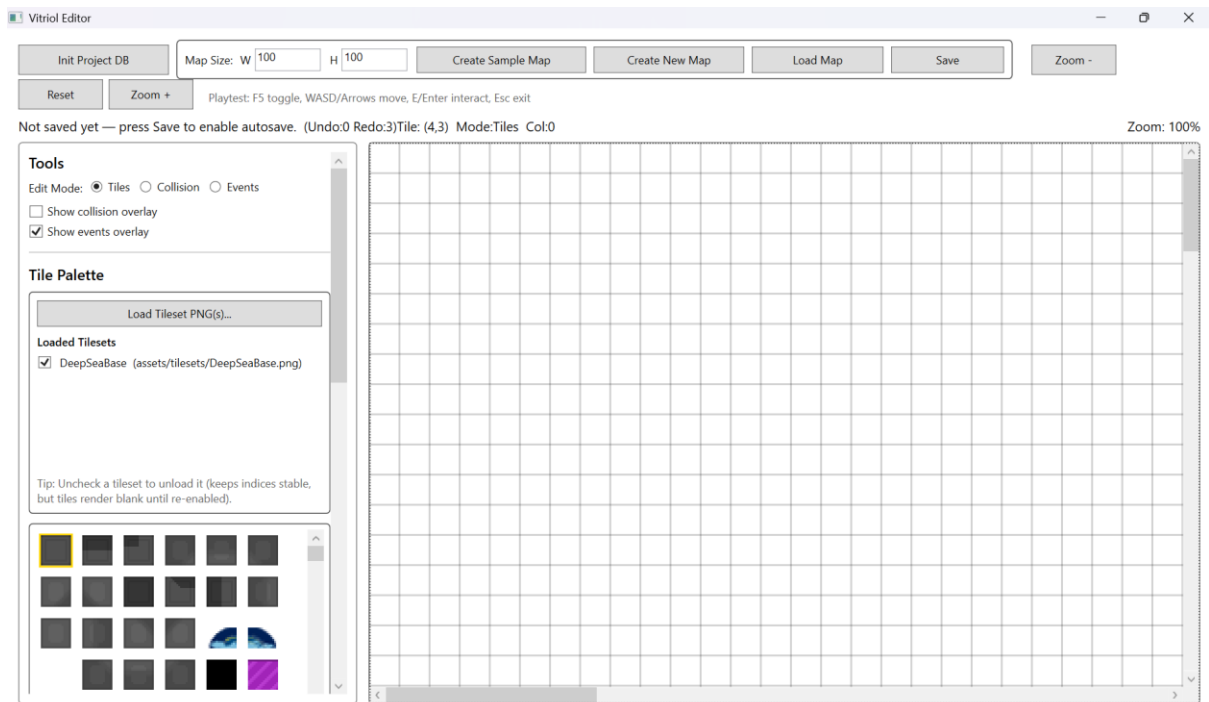


Figure T11.4 – Grid after 3th Undo Operation



Demonstrates correct last-in-first-out stack behaviour, restoring the grid to its original state after sequential command reversals

2.3 Tileset Loading and Referencing

This subsystem verifies that tilesets are referenced and loaded in a controlled and stable manner, preserving tile indices while allowing memory-efficient toggling of loaded assets.

Testing focused on:

- Correct storage of relative paths
- Successful loading and rendering of tileset images
- Stable tile indexing when toggling tilesets
- Correct behaviour when referenced tileset files are missing

Test Table 3 – Tileset Loading & Referencing

Test ID	Purpose of Test	Test Data	Expected Outcome	Actual Outcome	Screenshot ID
TS1	Verify tileset reference stored as relative path	Select tileset from project folder	JSON stores relative path from project root	Pass	Figures T12.1, T12.2
TS2	Verify tileset loads and renders correctly	Load tileset and paint tile	Tiles render correctly in viewport	Pass	Figures T13.1, T13.2

TS3	Verify toggling tileset does not corrupt map	Load -> Unload -> Reload	Tile indices preserved	Pass	Figures T14.1, T14.2, T14.3
TS4	Abnormal test – missing tileset file	Rename/remove PNG file	Validation error shown or load prevented	Pass	Figures T15.1, T15.2

Test Table 3 – Screenshot Evidence

Figure T12.1 – Tileset Selected in Editor

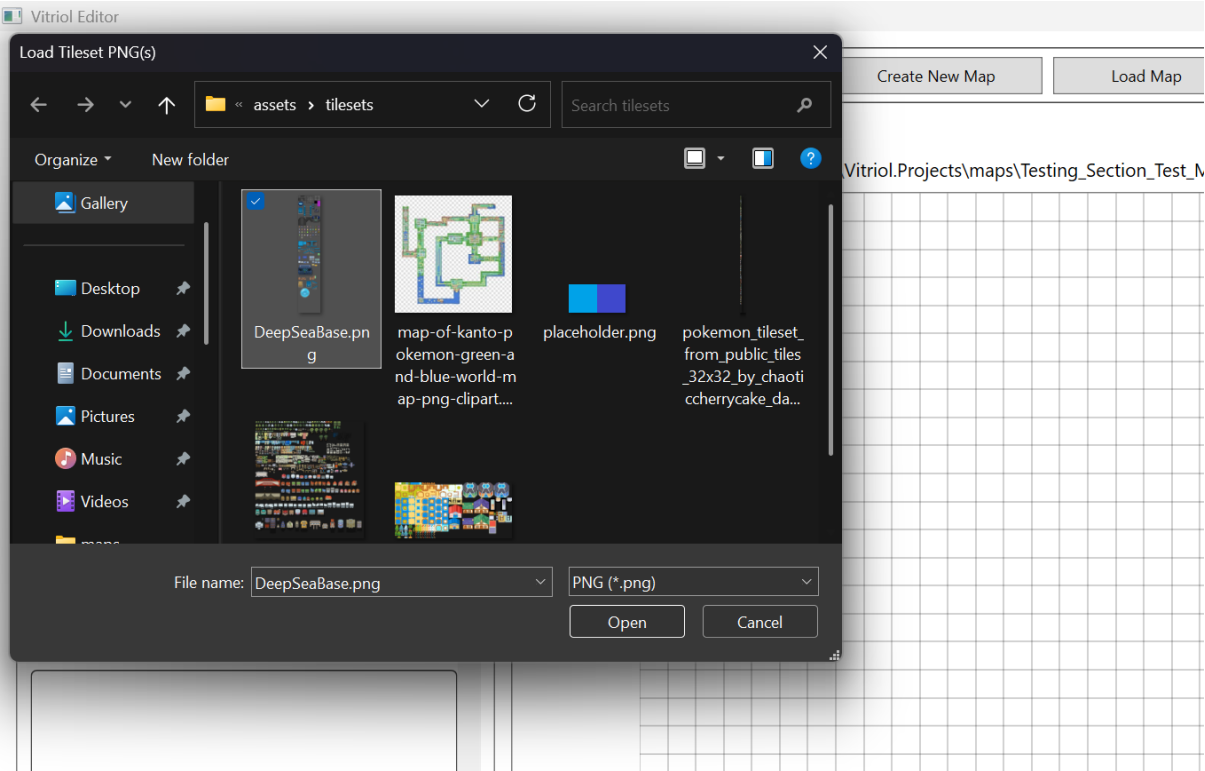
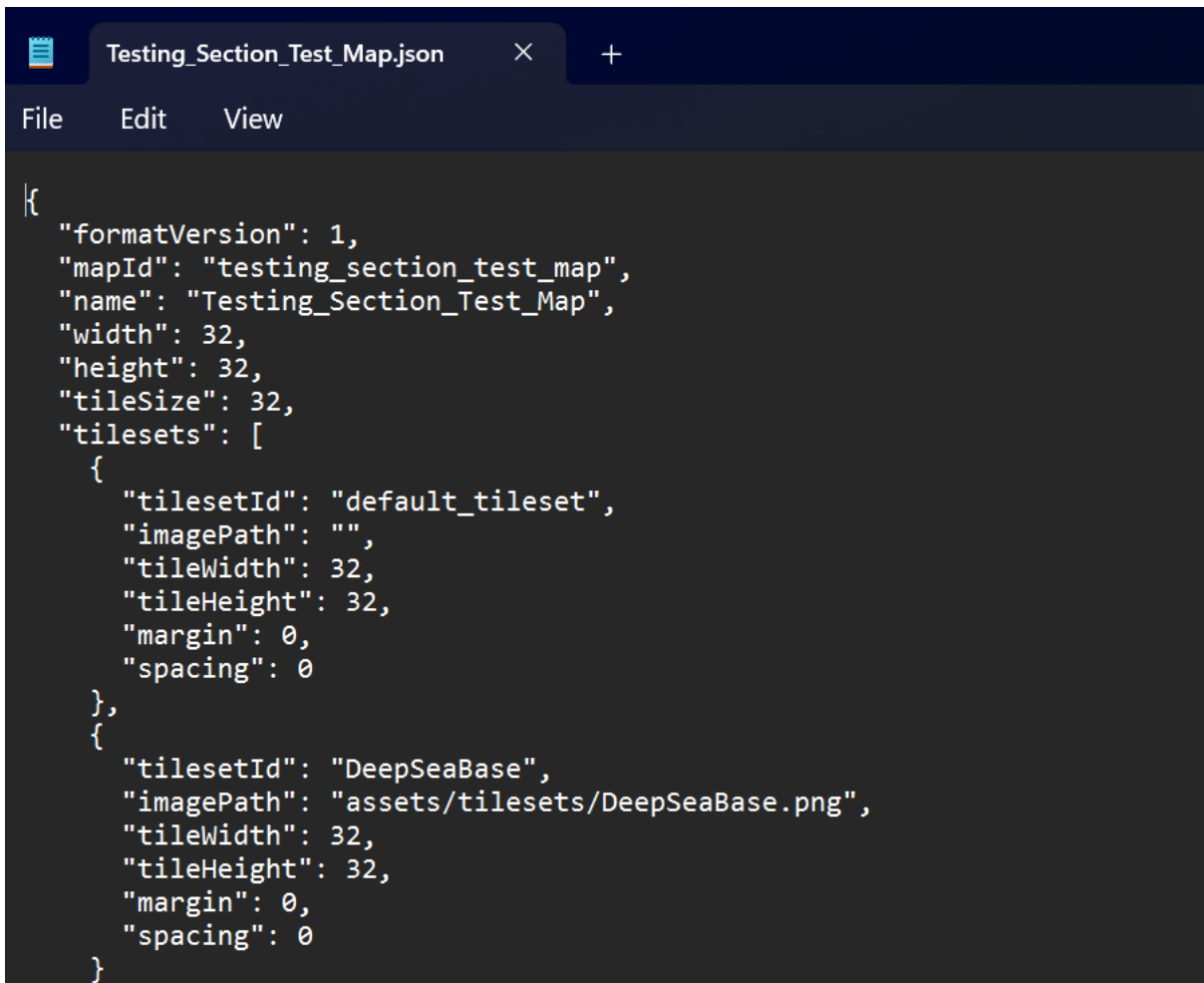


Figure T12.2 – JSON File Showing Relative Tileset Path



```
{
  "formatVersion": 1,
  "mapId": "testing_section_test_map",
  "name": "Testing_Section_Test_Map",
  "width": 32,
  "height": 32,
  "tileSize": 32,
  "tilesets": [
    {
      "tilesetId": "default_tileset",
      "imagePath": "",
      "tileWidth": 32,
      "tileHeight": 32,
      "margin": 0,
      "spacing": 0
    },
    {
      "tilesetId": "DeepSeaBase",
      "imagePath": "assets/tilesets/DeepSeaBase.png",
      "tileWidth": 32,
      "tileHeight": 32,
      "margin": 0,
      "spacing": 0
    }
  ]
}
```

Demonstrates that tileset references are stored using project-relative paths, ensuring portability and version-control compatibility

Figure T13.1 – Tileset Loaded in Editor Palette

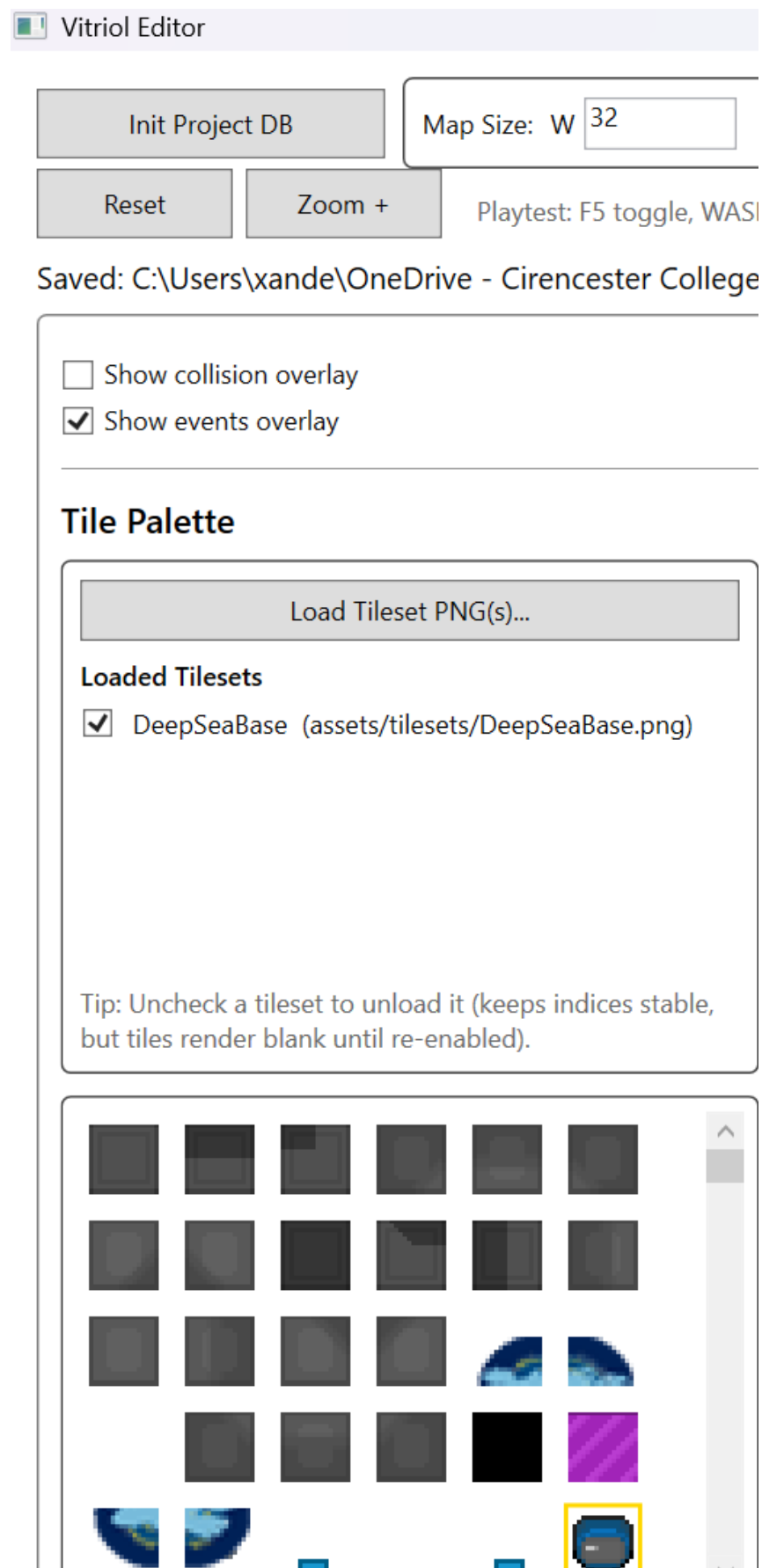
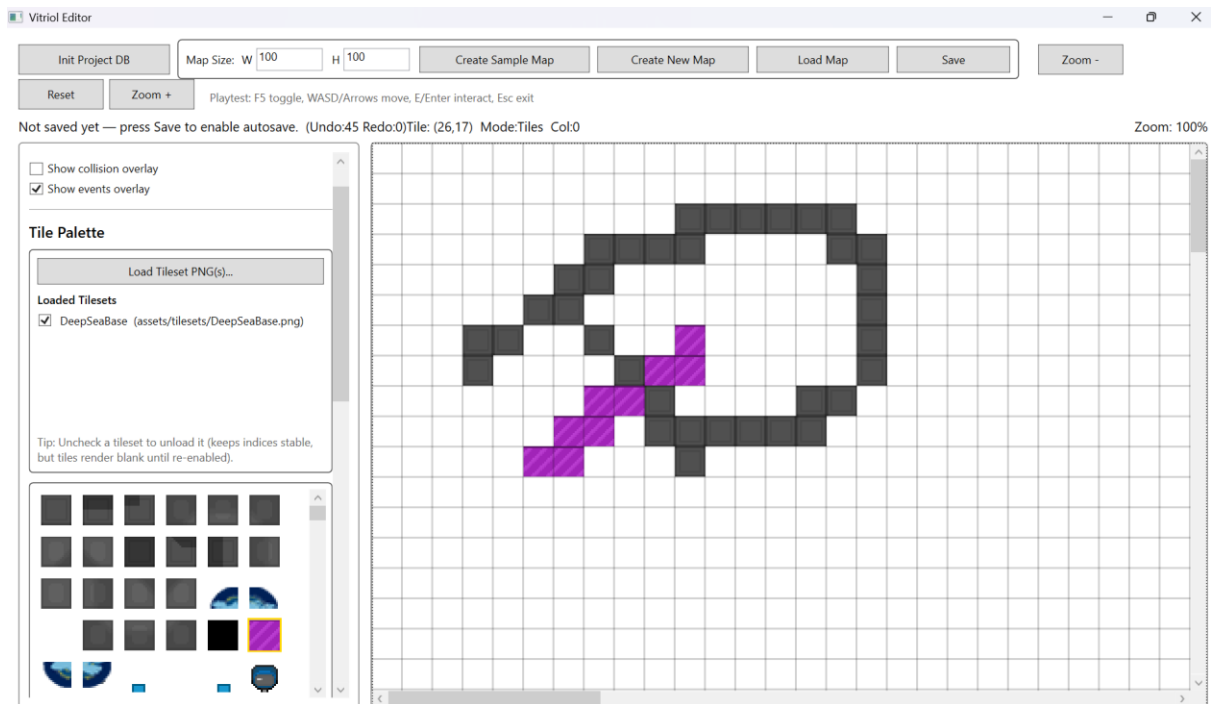


Figure T13.2 – Painted Tiles Rendering Correctly in Viewport



Demonstrates correct tileset loading and rendering within the editor viewport

Figure T14.1 – Painted Map Before Tileset Toggle

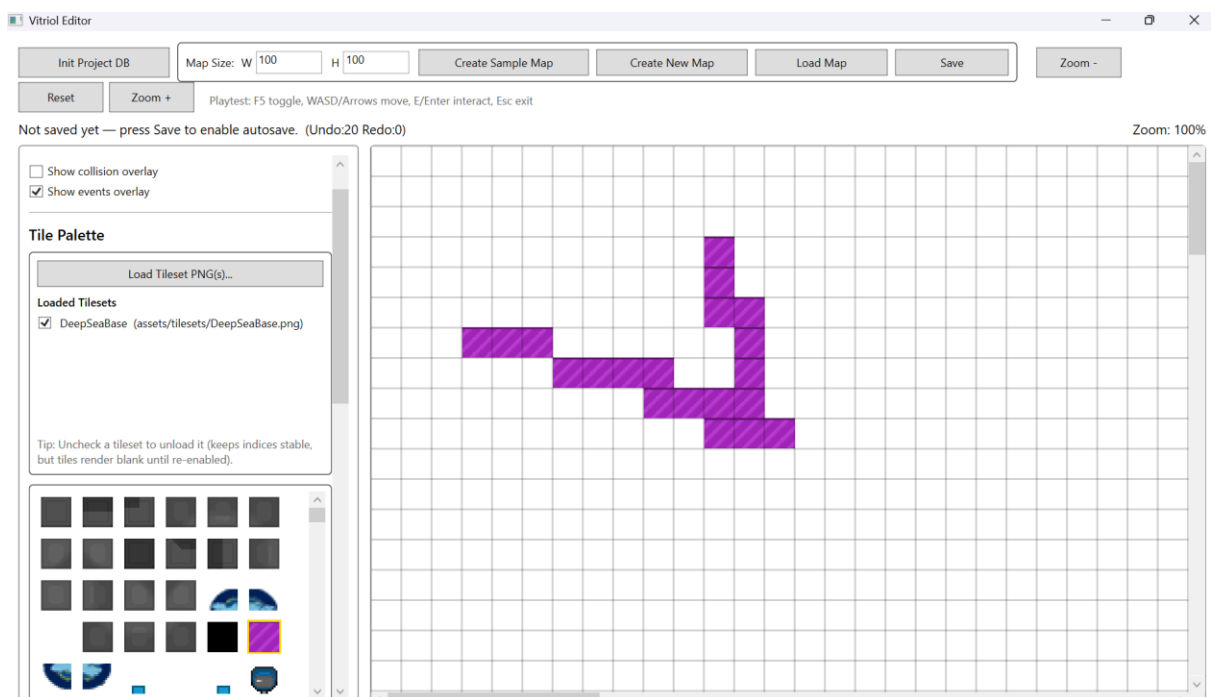


Figure T14.2 – Map After Tileset Toggle

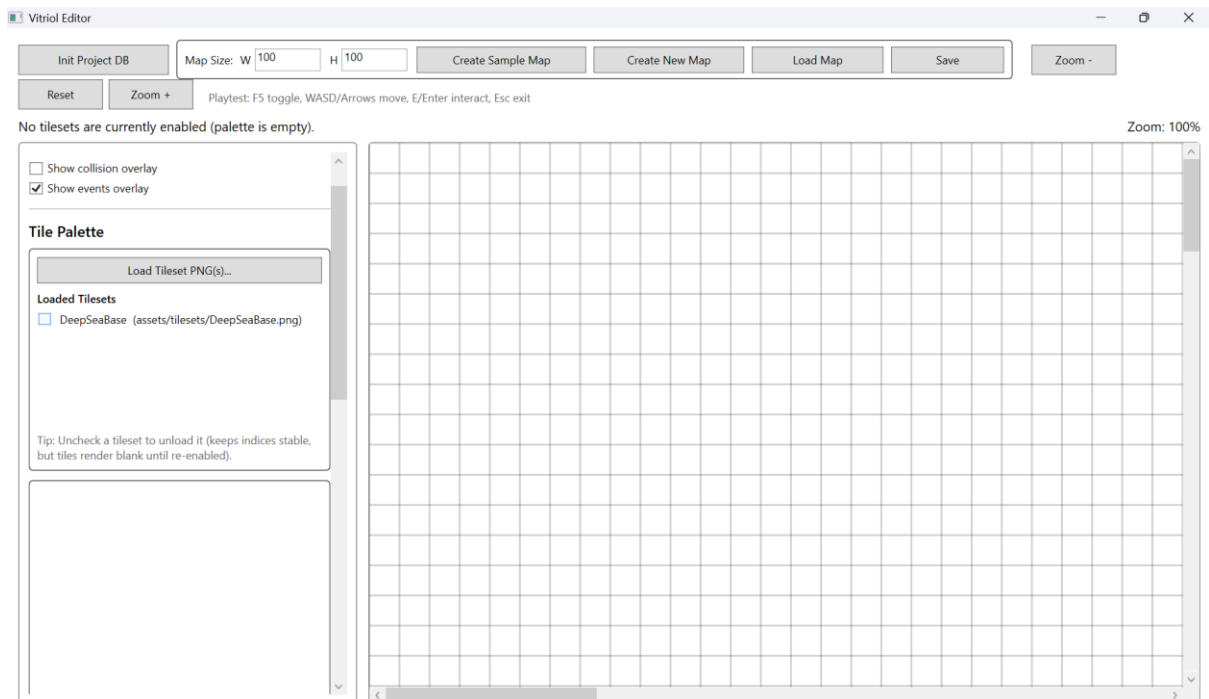
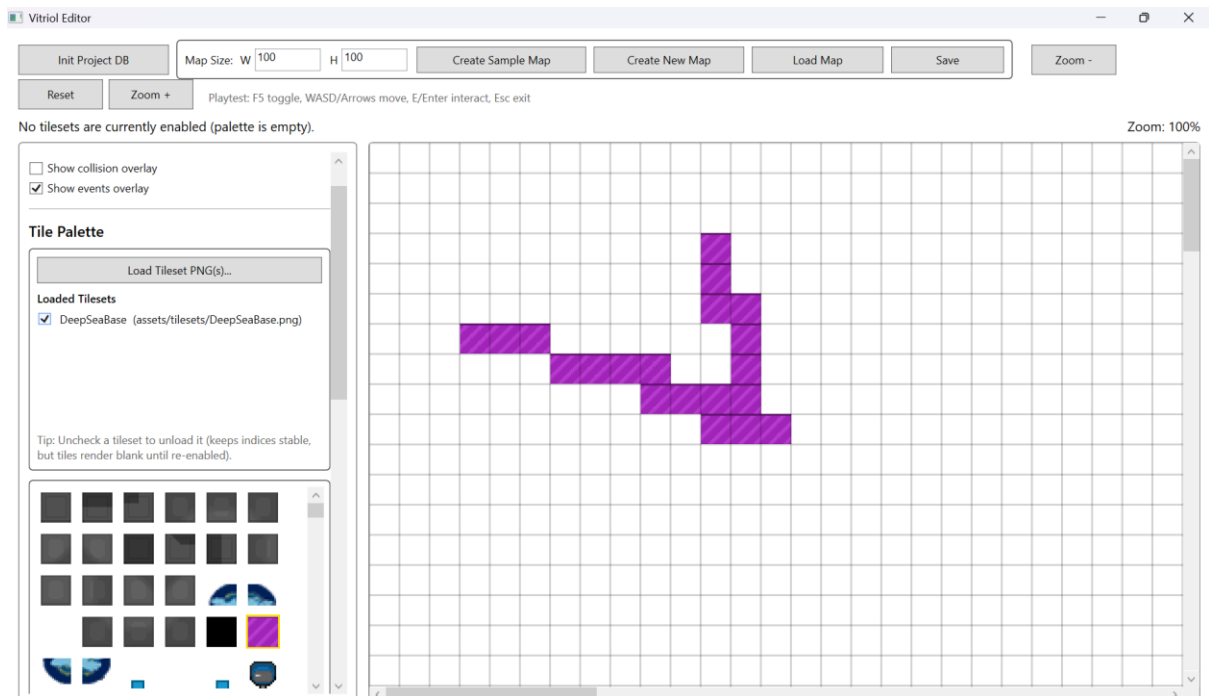


Figure T14.3 – Map After Tileset Reload (Indices Preserved)



Demonstrates that tile indices remain stable when tilesets are unloaded and reloaded, preventing structural corruption

Figure T15.1 – Loaded PNG File Renamed

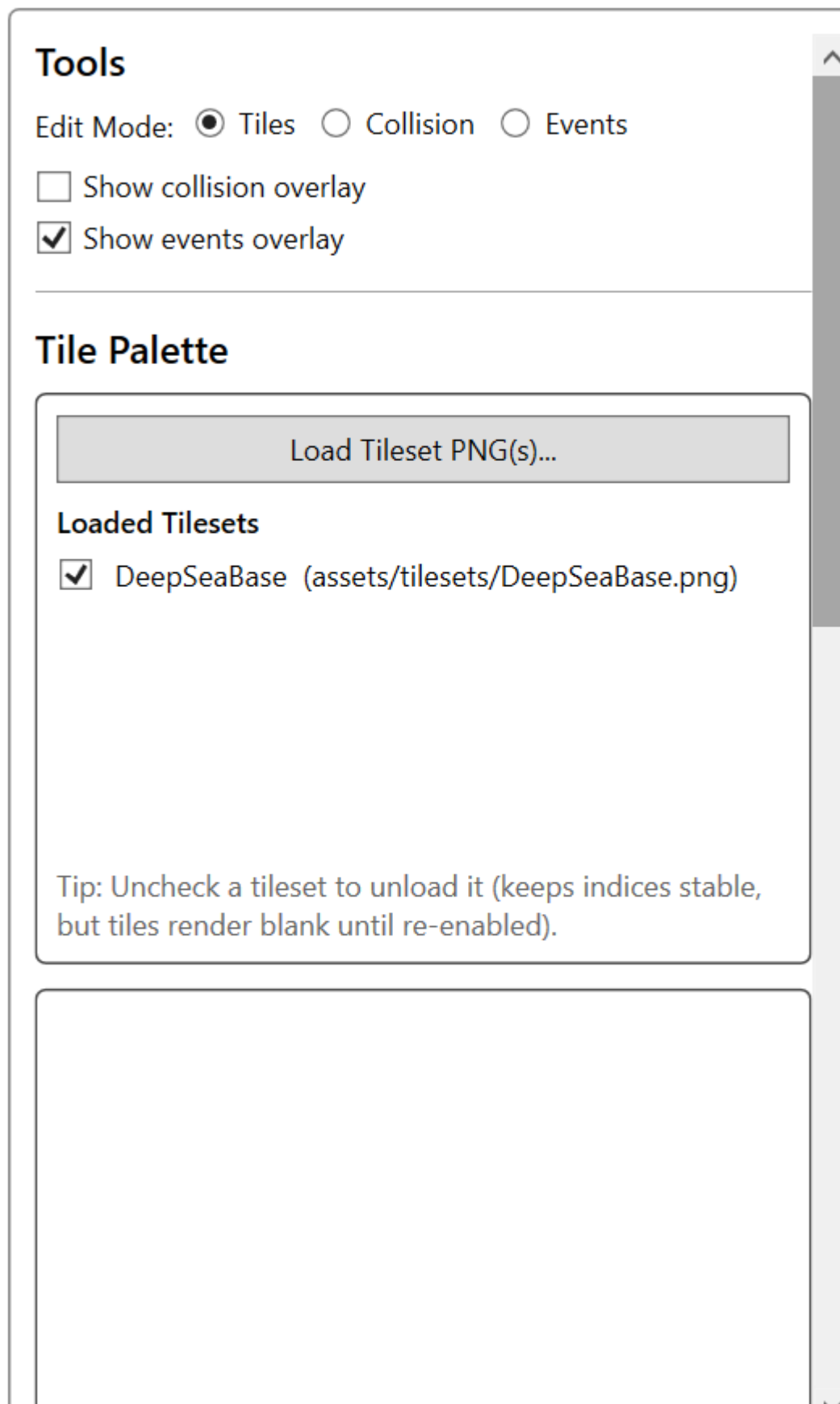
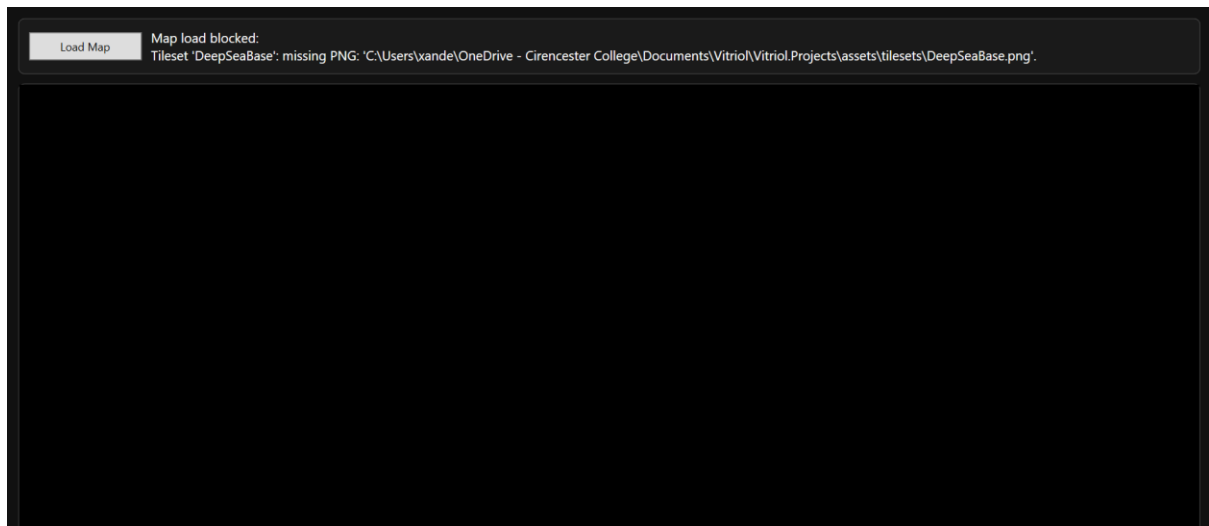


Figure T15.2 – Validation/Error Output for “Missing” Asset



Demonstrates controlled handling of missing asset references without runtime crash

2.4 Event System Testing

Test Table 4 – Event System Testing

Test ID	Purpose of Test	Test Data	Expected Outcome	Actual Outcome	Screenshot ID
EV1	Verify event placement on map grid	Place NPC event at (10,10)	Event appears at correct coordinate	Pass	Figures T16.1, T16.2
EV2	Verify event serialisation integrity	Save map after event creation	Event stored correctly in JSON	Pass	Figures T17.1, T17.2
EV3	Verify event reload stability	Close editor -> reopen map	Event reappears at correct location	Pass	Figures T18.1, T18.2
EV4	Verify runtime event trigger execution	Trigger event in runtime	Event executes associated behaviour	Pass	Figures T19.1, T19.2
EV5	Abnormal test – invalid event parameters	Remove required argument manually in JSON	Validation prevents crash/handles safely	Pass	Figures T20.1, T20.2

Test Table 4 – Screenshot Evidence

Figure T16.1 – Map Grid Before Event Placement

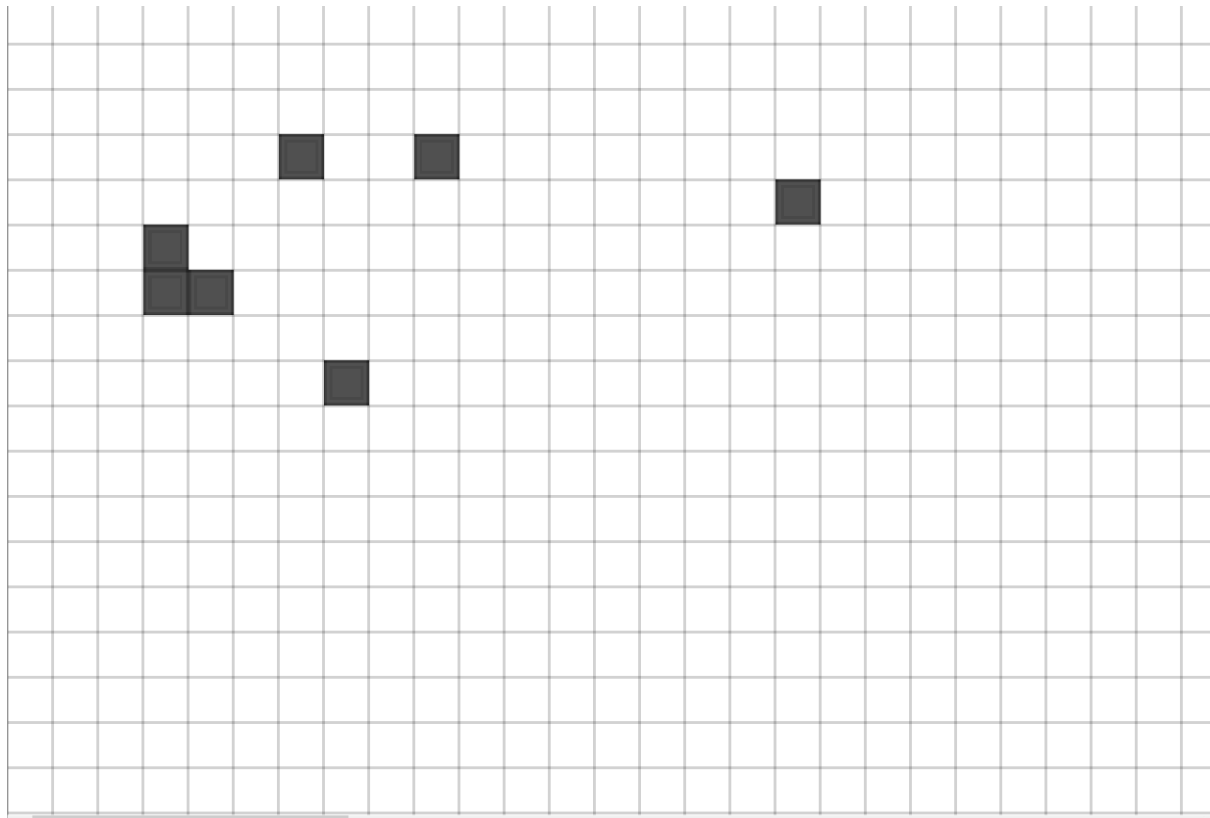
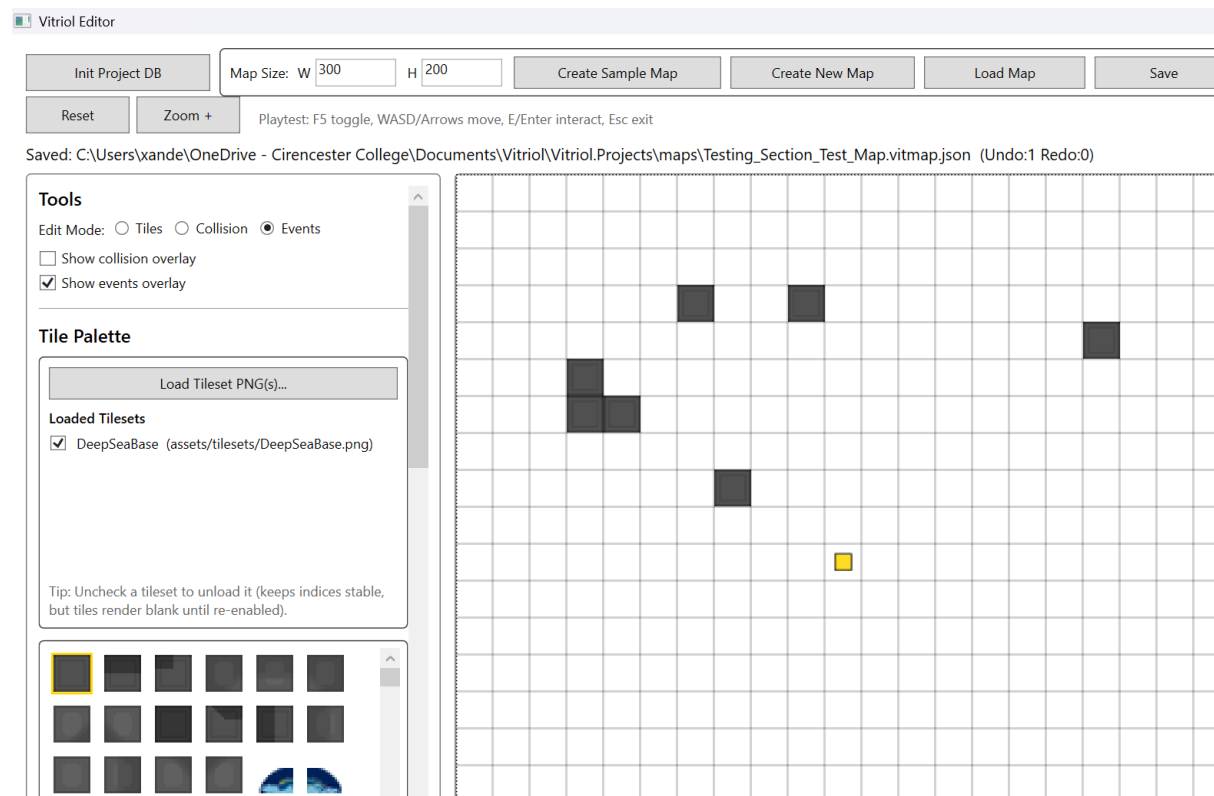


Figure T16.2 – Event Placed At Coordinate (10,10)



Demonstrates correct mapping between grid coordinate and event object placement

Figure T17.1 – Event Visible in Editor Prior to Save

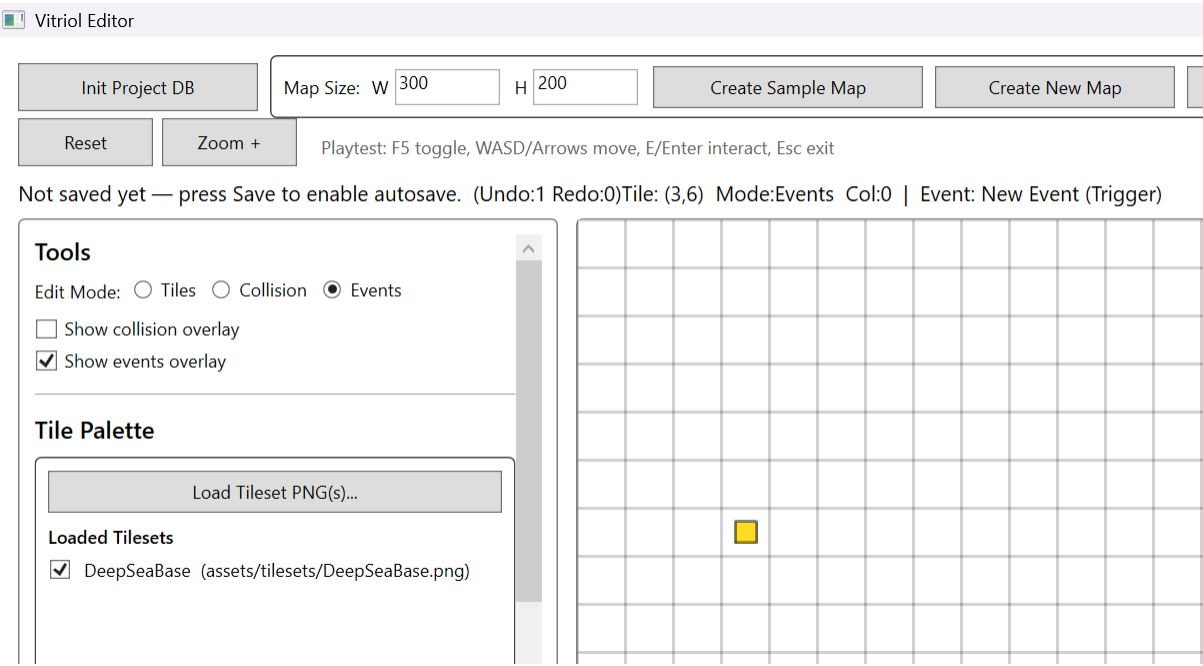
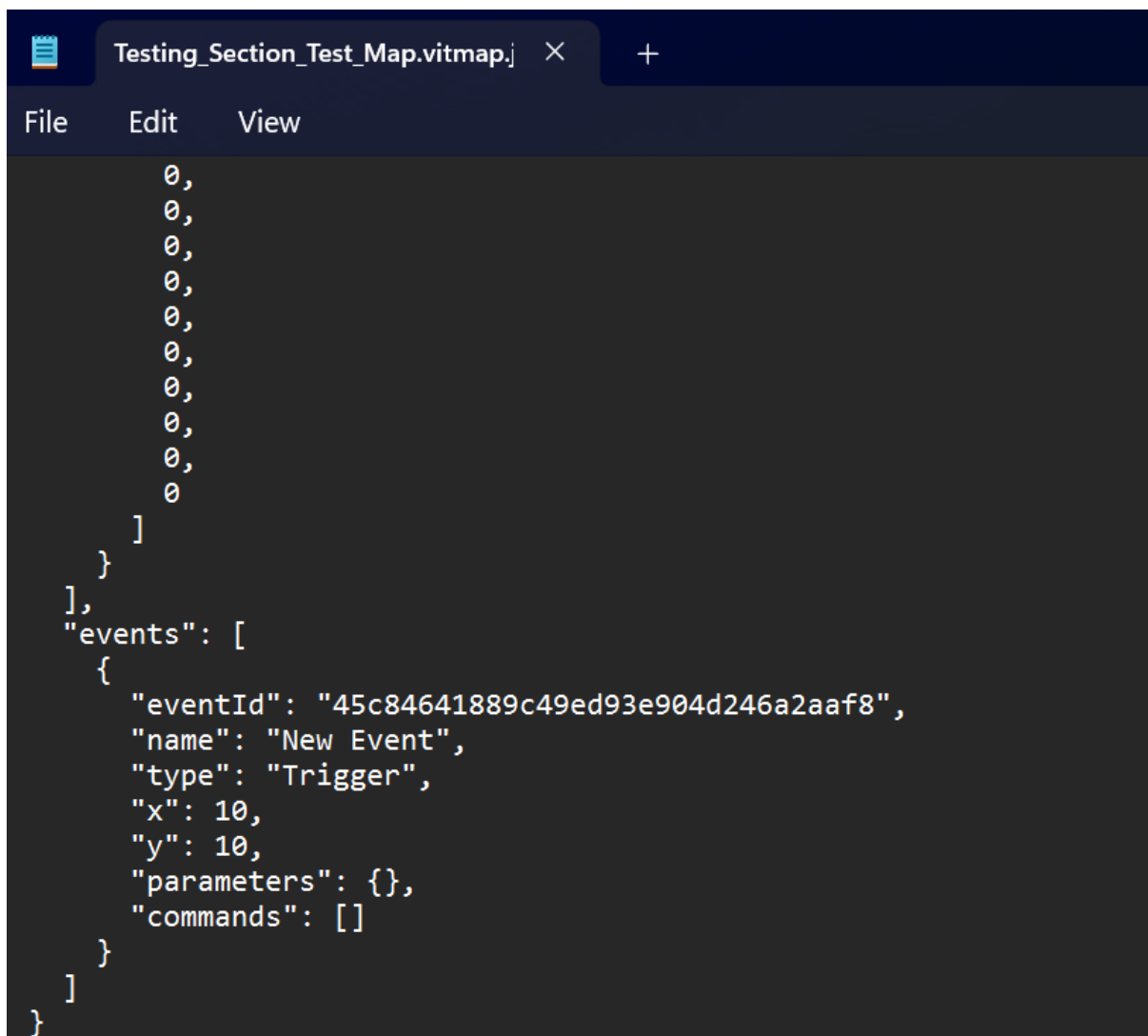


Figure T17.2 – JSON Representation of Saved Event Data

A screenshot of a code editor window titled 'Testing_Section_Test_Map.vitmap.j'. The editor has a dark theme and shows a JSON file. The JSON structure is as follows: an array of 10 zeros, followed by a closing bracket, then a closing brace, then a comma, then an object with a key 'events' containing an array of one object. This object has keys 'eventId', 'name', 'type', 'x', 'y', 'parameters', and 'commands'. The 'eventId' is a long hexadecimal string, 'name' is 'New Event', 'type' is 'Trigger', 'x' and 'y' are 10, 'parameters' is an empty object, and 'commands' is an empty array. The editor has a menu bar with 'File', 'Edit', and 'View' options.

```
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0  
]  
}  
],  
"events": [  
  {  
    "eventId": "45c84641889c49ed93e904d246a2aaf8",  
    "name": "New Event",  
    "type": "Trigger",  
    "x": 10,  
    "y": 10,  
    "parameters": {},  
    "commands": []  
  }  
]  
}
```

Demonstrates accurate serialisation of event metadata including ID, type, position, and command structure

Figure T18.1 – Event Visible Before Editor Restart

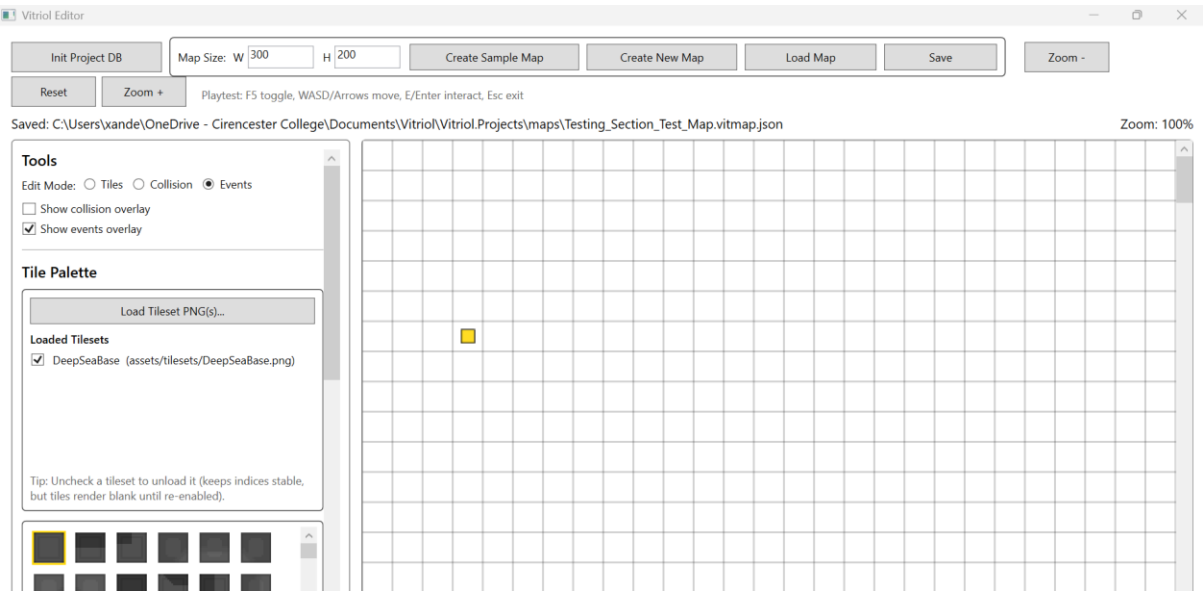
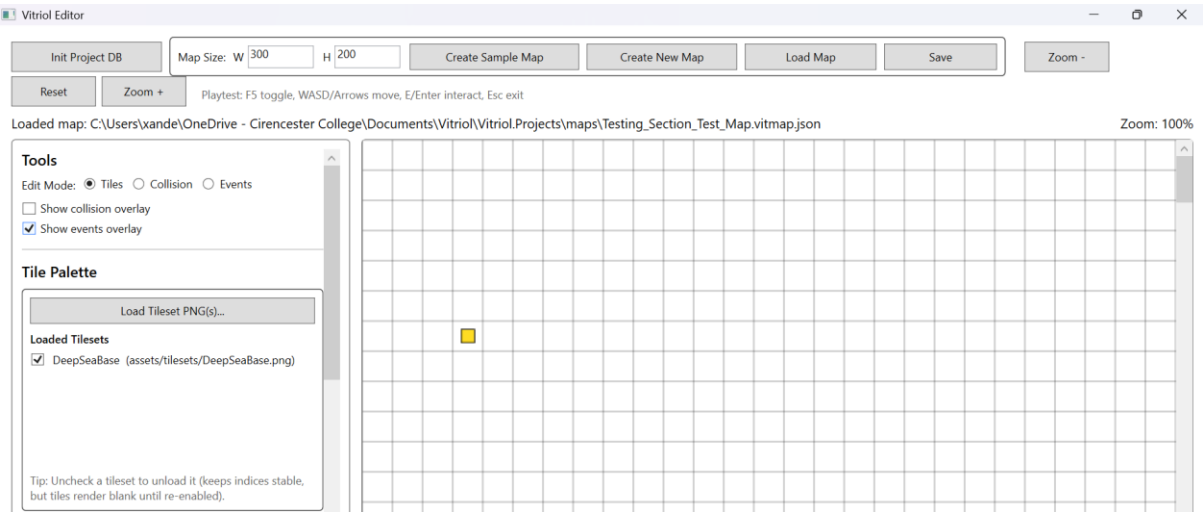


Figure T18.2 – Event Correctly Restored After Reload



Demonstrates persistence integrity across sessions without data loss

Figure T19.1 – Player Approaching Event Trigger Tile

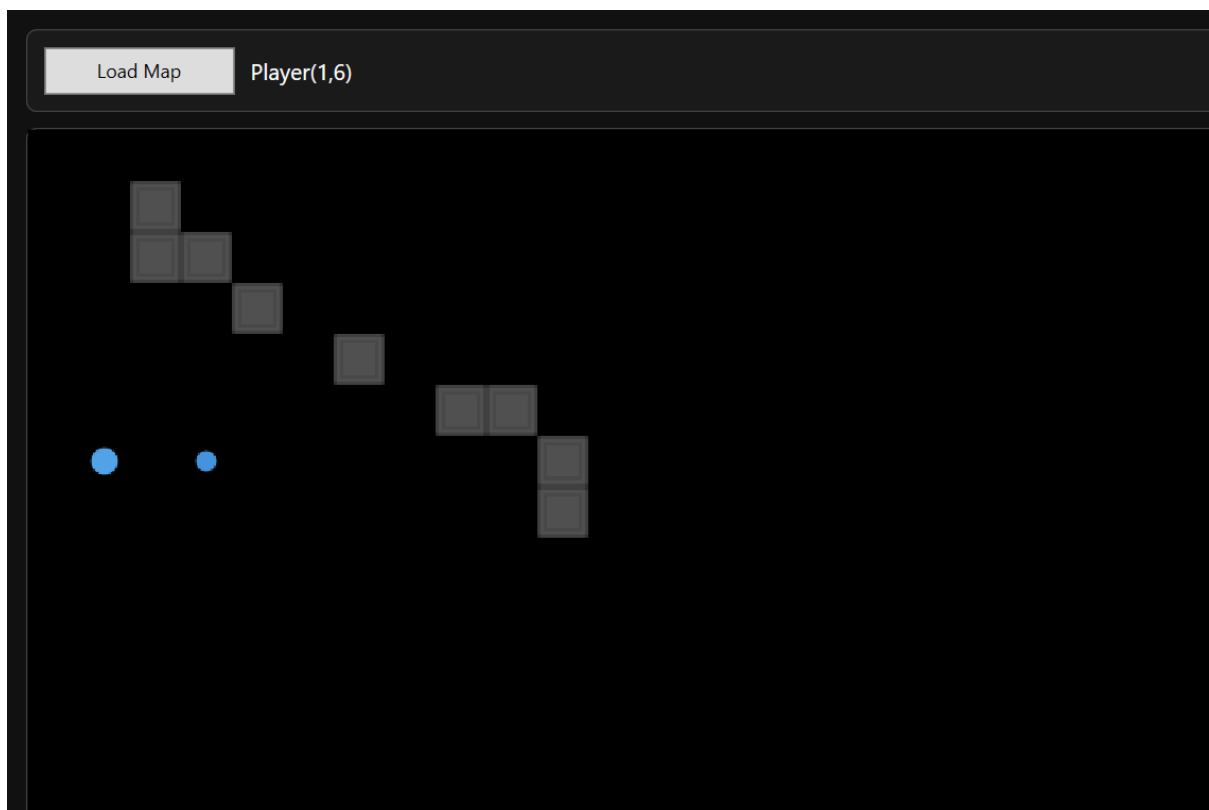
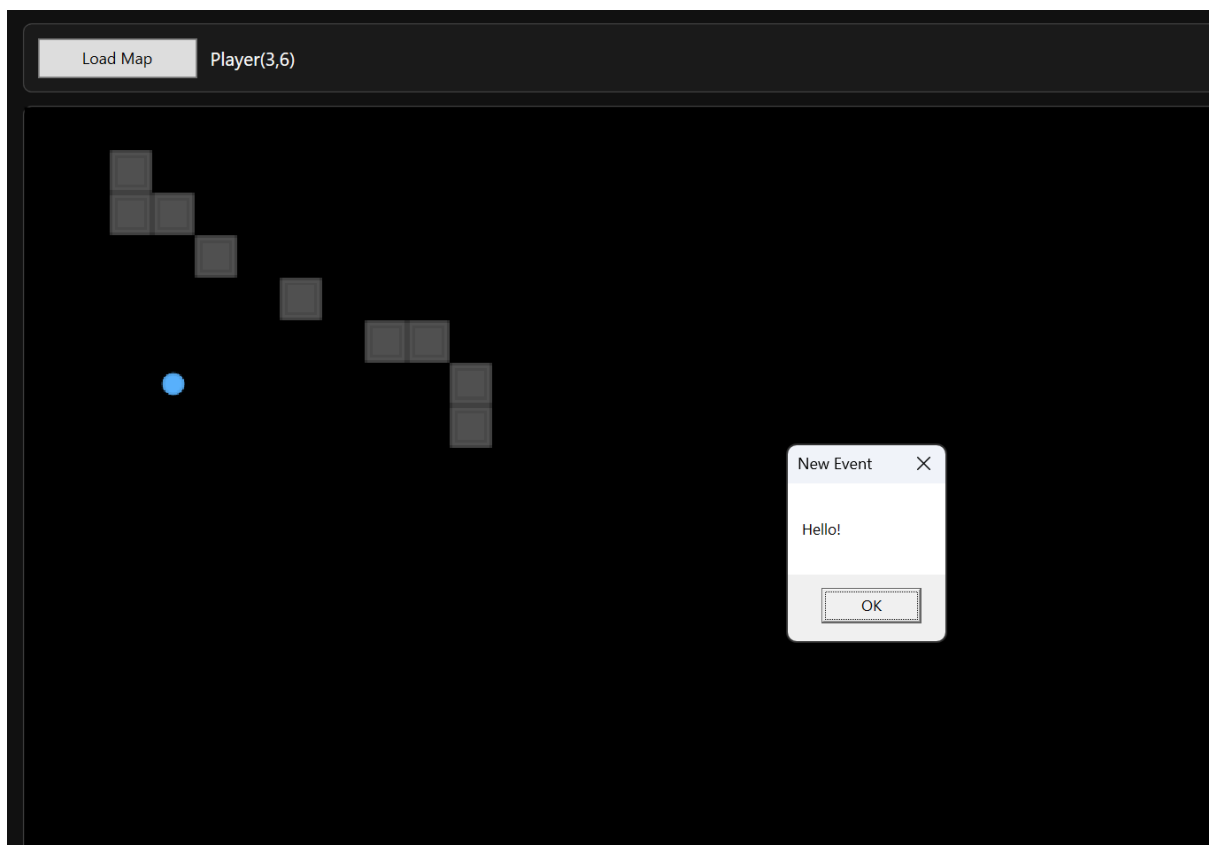
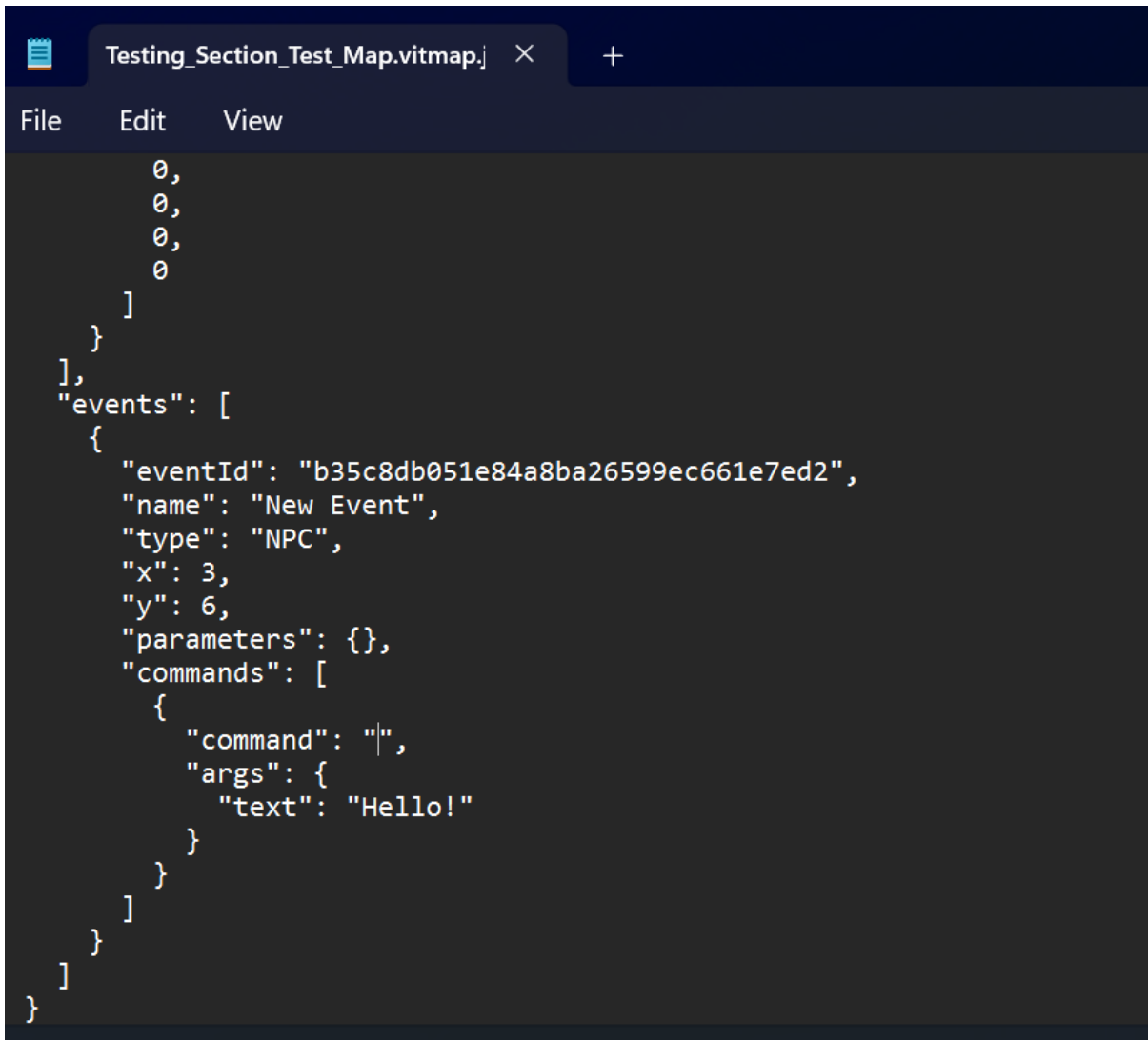


Figure T19.2 – Event Behaviour Executed in Runtime (Dialogue/Effect Displayed)



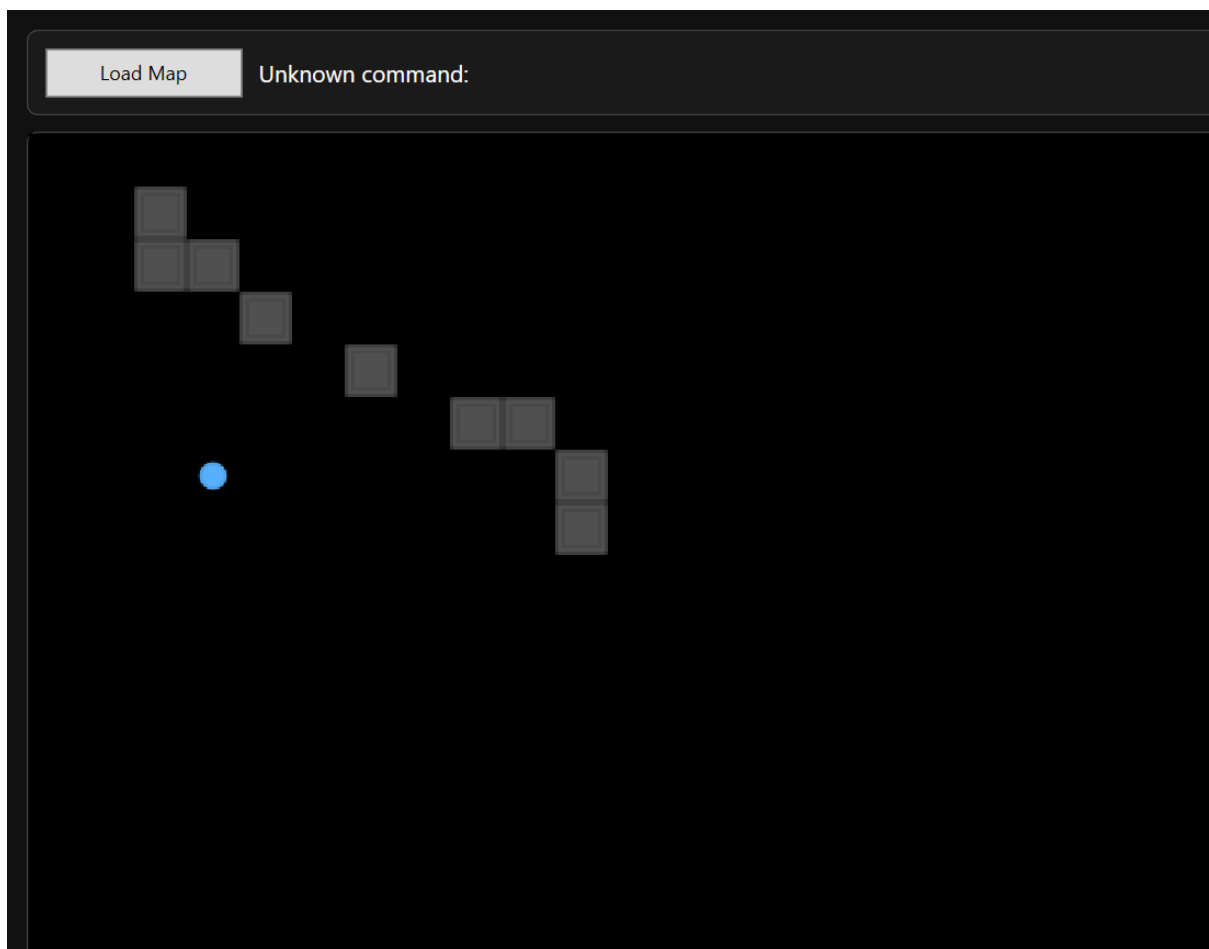
Demonstrates correct linkage between editor-defined event data and runtime execution logic

Figure T20.1 Modified JSON with Missing Event Parameter



```
0,  
0,  
0,  
0  
]  
}  
],  
"events": [  
  {  
    "eventId": "b35c8db051e84a8ba26599ec661e7ed2",  
    "name": "New Event",  
    "type": "NPC",  
    "x": 3,  
    "y": 6,  
    "parameters": {},  
    "commands": [  
      {  
        "command": "|",  
        "args": {  
          "text": "Hello!"  
        }  
      }  
    ]  
  }  
]  
}
```


Figure T20.2 – Validation Message or Safe Handling During Load



Demonstrates defensive validation preventing malformed event data from causing runtime instability

2.5 Persistence and JSON Integrity

The persistence subsystem is responsible for ensuring that map data is reliably stored as structured JSON files and can be reloaded without corruption or structural drift. Because the project deliberately uses JSON files instead of a relational database, testing focused on validating:

- Correct file creation and naming
- Autosave behaviour rules
- JSON schema stability
- Round-trip integrity (save -> close -> reload)
- Cross-application compatibility (Editor -> Runtime)

The following tests validate that:

- Manual save produces a valid .vitmap.json file
- Autosave follows intended behavioural rules
- JSON structure remains consistent with the defined schema

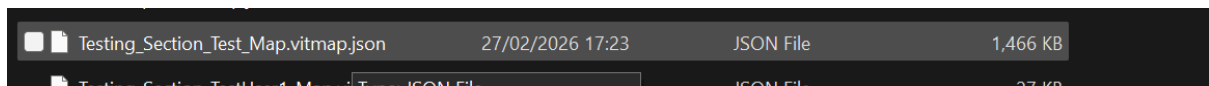
- Map state survives editor restart
- Runtime can load editor-produced files without format mismatch

Test Table 5 – Persistence & JSON Integrity

Test ID	Purpose of Test	Test Data	Expected Outcome	Actual Outcome	Screenshot ID
PJ1	Verify manual save creates valid JSON file	Create new user map -> paint tiles -> Save As	JSON file created in maps folder, correct structure present	Pass	Figures T21.1, T21.2
PJ2	Verify autosave rules (sample vs user map)	Edit sample map and new user map	Sample autosaves automatically, new map requires first manual save	Pass	Figures T22.1, T22.2, T22.3
PJ3	Verify JSON schema structure	Open saved JSON in Notepad	File contains required root fields (FormatVersion, Width, Height, Layers, Tilesets, Events)	Pass	Figures T23.1, T23.2, T23.3
PJ4	Verify round-trip persistence	Save -> close editor -> reopen -> reload map	Tiles, events, and layout restored correctly	Pass	Figures T24.1, T24.2
PJ5	Verify runtime compatibility	Load same map in runtime	Runtime loads successfully and renders identical layout	Pass	Figures T25.1, T25.2

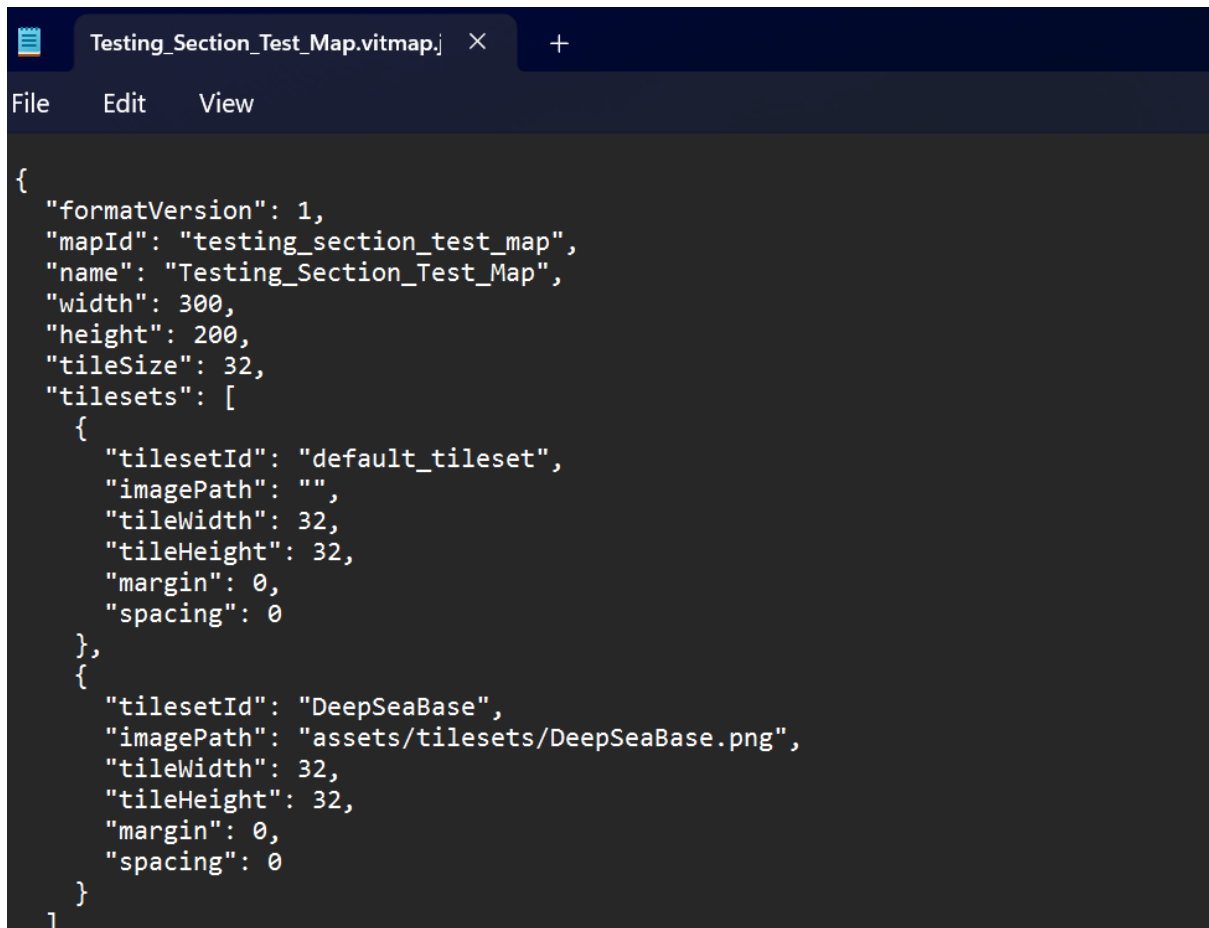
Test Table 5 – Screenshot Evidence

Figure T21.1 – Maps Folder Showing Newly Created .vitmap.json File



Testing_Section_Test_Map.vitmap.json	27/02/2026 17:23	JSON File	1,466 KB
Testing_Section_Test_Map1_Map.vitmap.json		JSON File	27 KB

Figure T21.2 – JSON File Open in Notepad Showing Root Structure



```
{
  "formatVersion": 1,
  "mapId": "testing_section_test_map",
  "name": "Testing_Section_Test_Map",
  "width": 300,
  "height": 200,
  "tileSize": 32,
  "tilesets": [
    {
      "tilesetId": "default_tileset",
      "imagePath": "",
      "tileWidth": 32,
      "tileHeight": 32,
      "margin": 0,
      "spacing": 0
    },
    {
      "tilesetId": "DeepSeaBase",
      "imagePath": "assets/tilesets/DeepSeaBase.png",
      "tileWidth": 32,
      "tileHeight": 32,
      "margin": 0,
      "spacing": 0
    }
  ]
}
```

Demonstrates correct file creation and schema structure after manual save

Figure T22.1 – Sample Map Modified Timestamp Updating Automatically

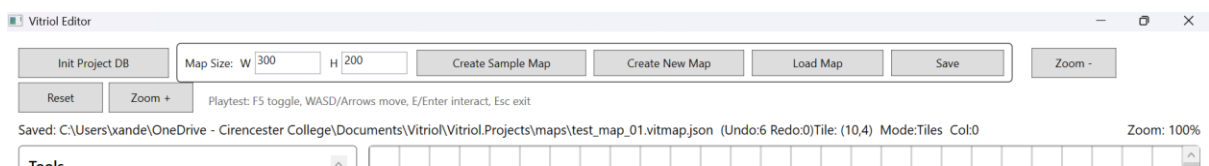


Figure T22.2 – User Map Not Saved Before First Manual Save

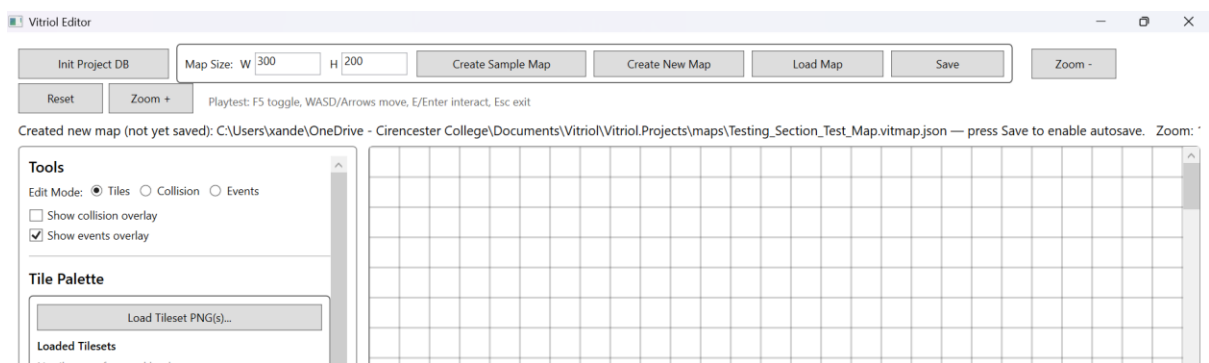
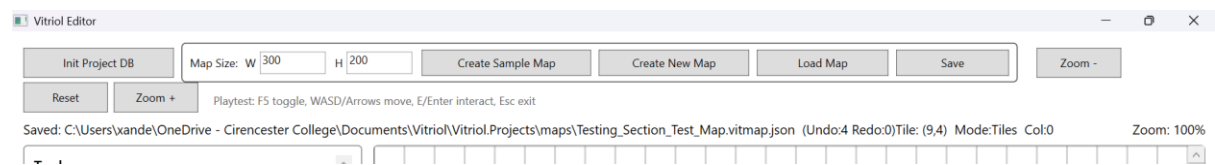


Figure T22.3 – User Map Autosave Active After Initial Manual Save



Demonstrates correct enforcement of autosave safety logic

Figure T23.1 – JSON Root Fields Visible

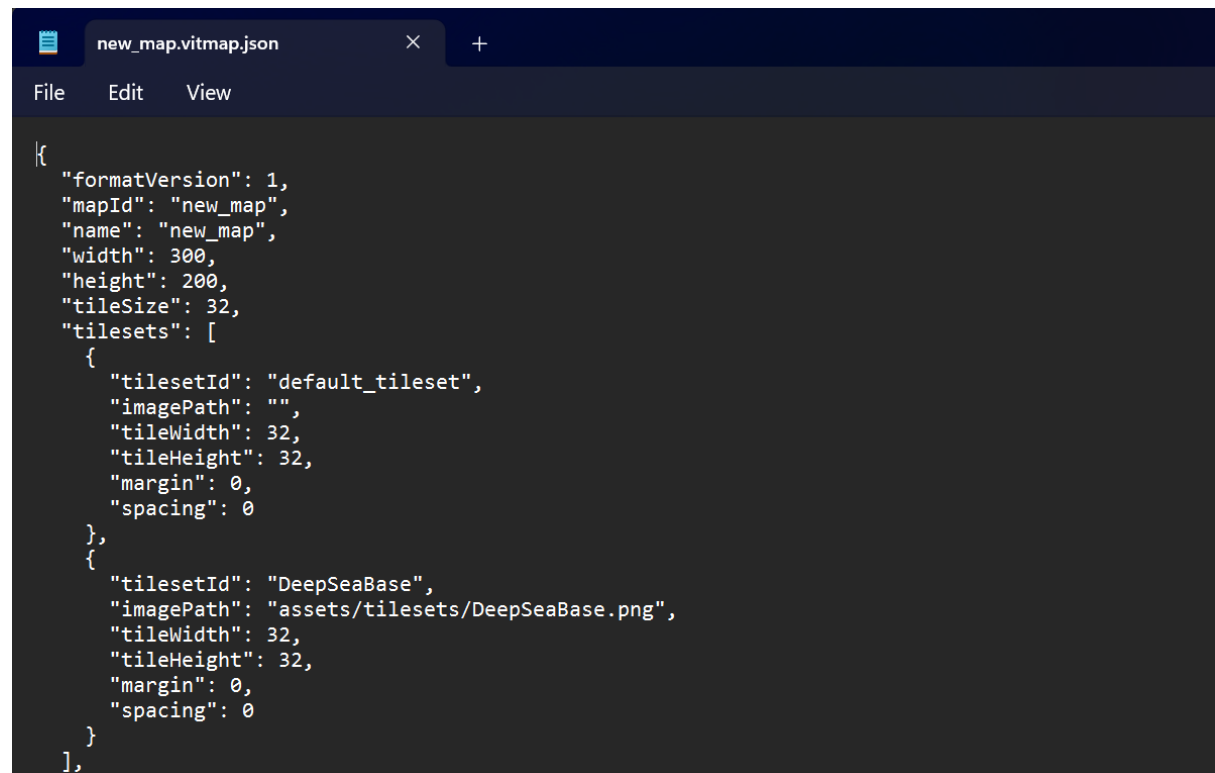


Figure T23.2, T23.3 – Layers and Events Structure Visible

[illegible]

Figure T24.1 – Map State Before Closing Editor

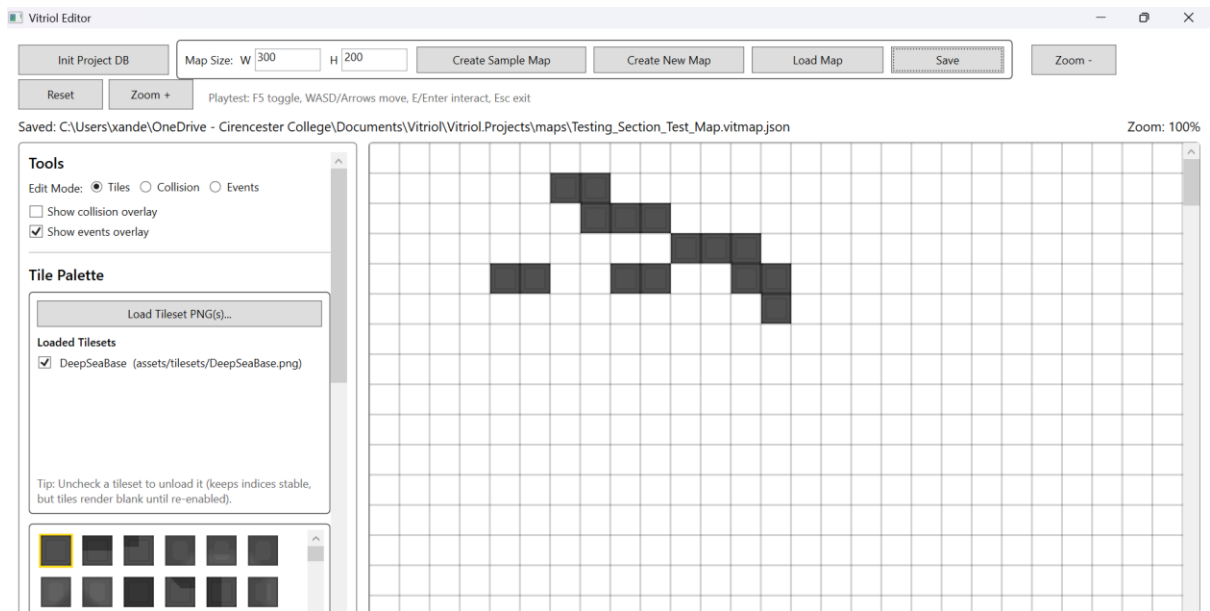
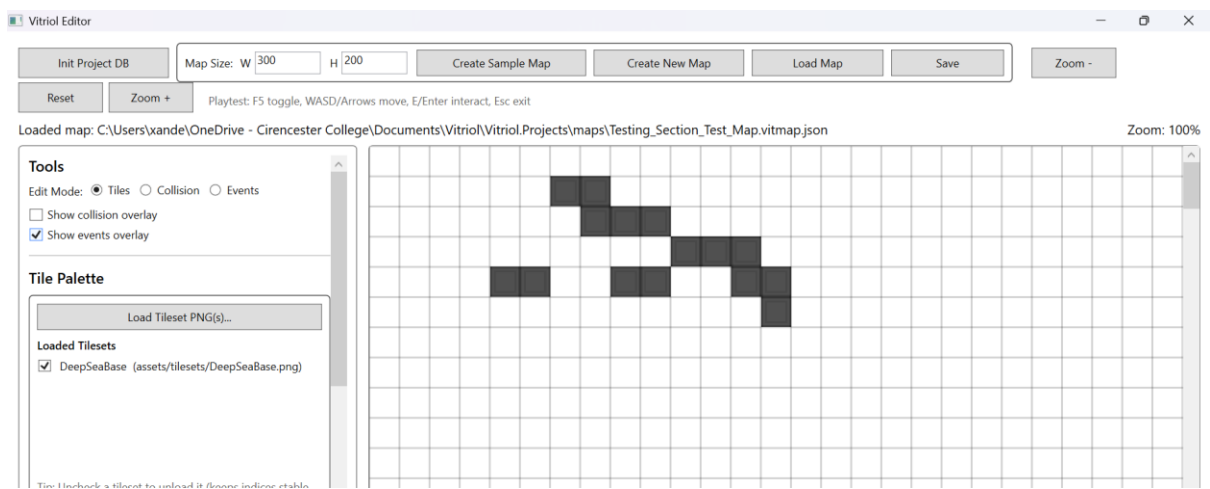


Figure T24.2 – Map State Restored After Reload



Demonstrates no data loss across sessions

Figure T25.1 – Runtime Loading Map File



Figure T25.2 – Runtime Rendering Saved Map Layout



Demonstrates editor/runtime format alignment and absence of schema drift

2.6 Runtime Rendering and Viewport Behaviour

The runtime rendering subsystem is responsible for displaying the map efficiently while maintaining responsive gameplay. Because Vitriol uses viewport-only rendering, the

runtime should never attempt to draw the entire map every frame. Instead, it renders only the visible region (plus padding), producing a single composited output image that updates as the camera moves.

Testing focused on validating:

- Viewport-only rendering correctness (only visible region drawn)
- Stable rendering while scrolling (no blank gaps / tearing)
- Correct tile alignment as player moves across tile boundaries
- Render update logic triggered by movement and camera shifts
- Behaviour on larger maps where full rendering would be expensive

The following tests validate that:

- The runtime renders a cropped region rather than the entire map
- The viewport updates correctly when the player moves
- Large maps remain usable without frame collapse
- Visual output stays aligned to the map grid during camera movement

Test Table 6 – Runtime Rendering & Viewport

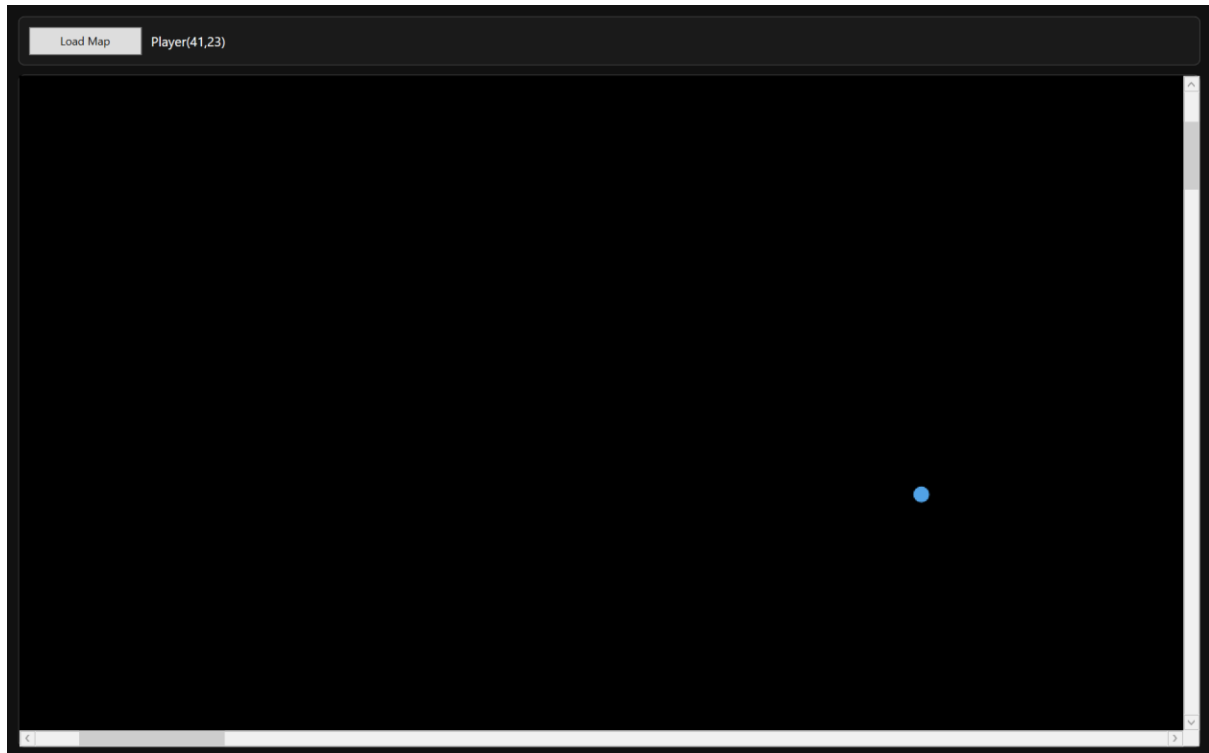
Test ID	Purpose of Test	Test Data	Expected Outcome	Actual Outcome	Screenshot ID
RV1	Verify viewport-only rendering (cropped region)	Load a large map and observe only current region visible	Only visible area is drawn, map can scroll without rendering full map	Pass	Figures T26.1, T26.2
RV2	Verify render updates on player movement	Move player across several tiles	Viewport redraws correctly, no lag/tearing	Pass	Figures T27.1, T27.2
RV3	Verify render stability during scroll/camera change	Move until camera shifts multiple times	Camera shifts smoothly, no blank regions or misalignment	Pass	Figures T28.1, T28.2
RV4	Boundary test – map edge rendering	Move player to far edge of map	Viewport clamps correctly, no drawing outside bounds	Pass	Figures T29.1, T29.2

Test Table 6 – Screenshot Evidence

Figure T26.1 – Runtime Loaded Large Map (Initial Viewport Region)



Figure T26.2 – Same Map After Moving Player (Viewport Region Changed)



Demonstrates viewport-only rendering where the visible region updates rather than full-map rendering

Figure T27.1 – Player Position Before Movement

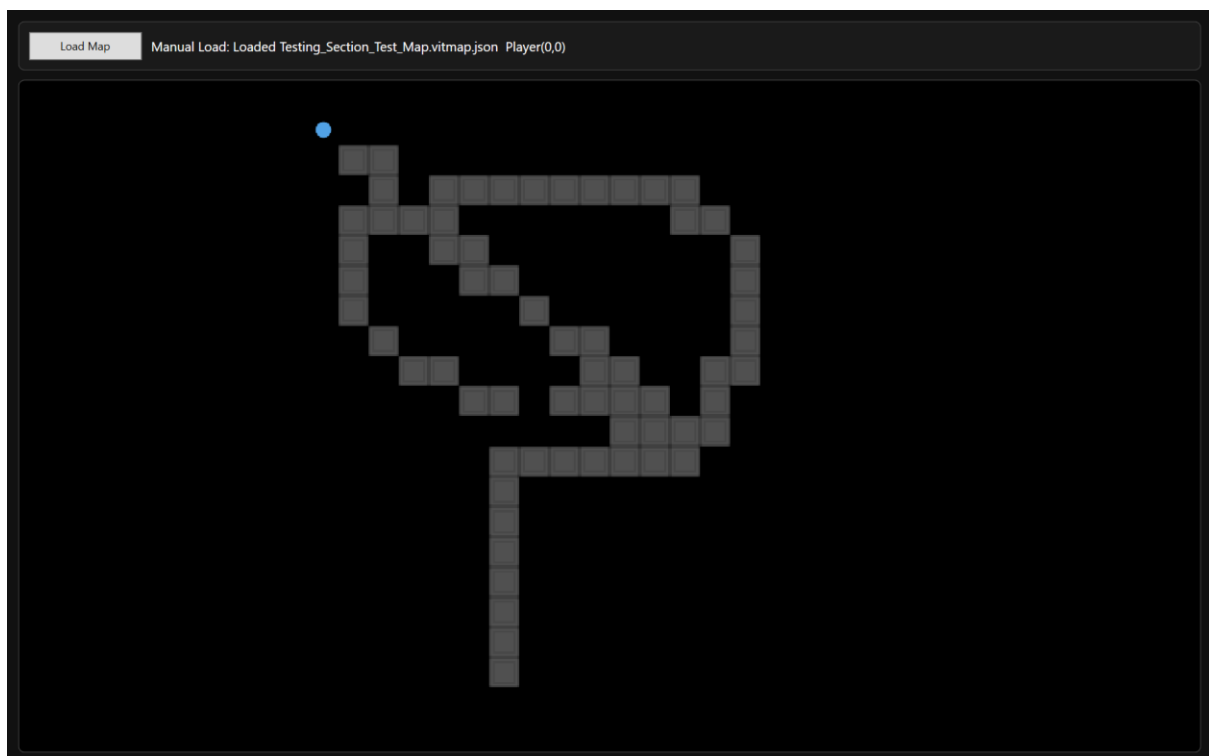
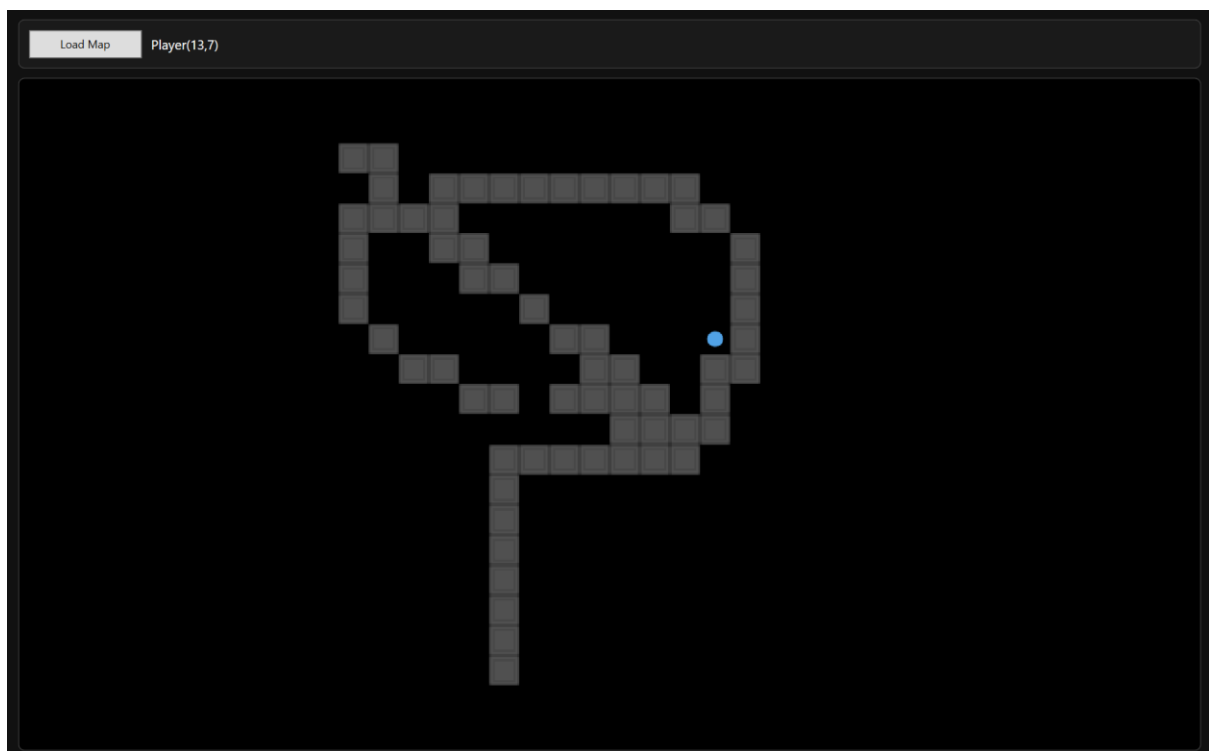


Figure T27.2 – Player Position After Movement (Viewport Updated Correctly)



Demonstrates render refresh correctness per movement step

Figure T28.1 – Camera Before Deadzone Exit (Player Near Edge of Deadzone)

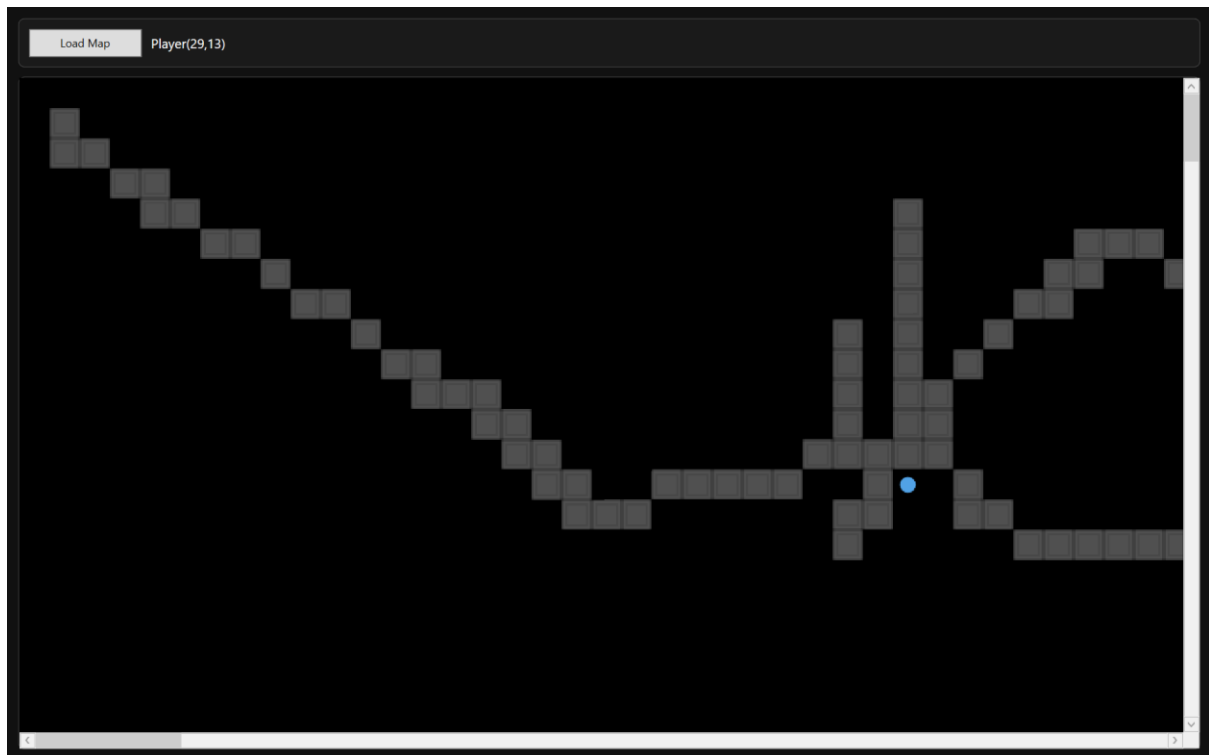
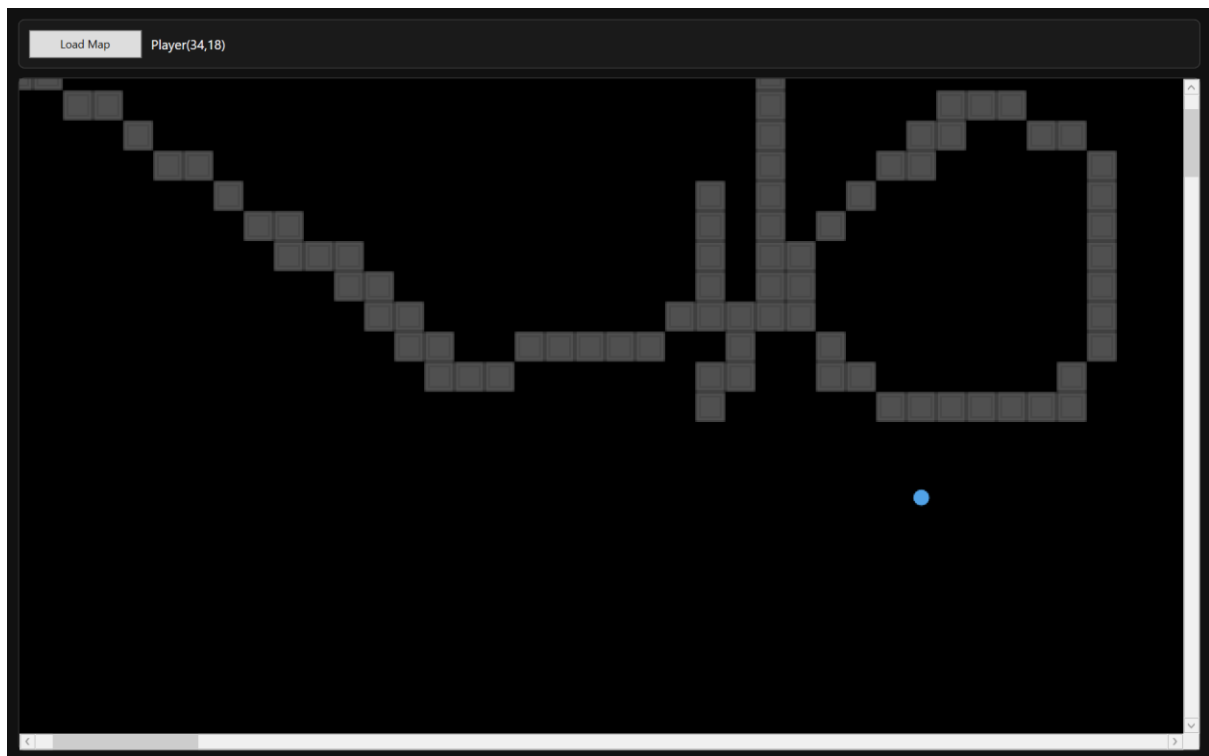


Figure T28.2 – Camera After Deadzone Exit (Viewport Shifted, Still Correct)



Demonstrates stable redraw during camera-driven viewport relocation

Figure T29.1 – Player at Map Edge (Viewport Clamped)



Figure T29.2 – Opposite Edge (Viewport Still Clamped Correctly)



Demonstrates correct clamping and safe region calculation at boundaries

2.7 Camera Deadzone

The camera subsystem uses a deadzone-based follow system rather than centering on the player every frame. This design prevents constant micro-adjustments and produces smoother visual motion. The camera only shifts when the player exits a defined deadzone region within the viewport.

Testing focused on validating:

- Deadzone boundaries behave as defined
- Camera does not move unnecessarily
- Camera shifts only when player crosses threshold
- Camera clamping prevents scrolling outside map bounds
- Player remains visible at all times

The following tests validate that:

- The player can move within the deadzone without camera movement
- Exiting the deadzone triggers controlled camera shift
- Camera does not overshoot or jitter
- Camera remains clamped at map edges

Test Table 7 – Camera Deadzone

Test ID	Purpose of Test	Test Data	Expected Outcome	Actual Outcome	Screenshot ID
CD1	Verify no camera movement inside deadzone	Move player small distance within viewport centre	Camera remains static	Pass	Figures T30.1, T30.2
CD2	Verify camera shift on deadzone exit	Move player until crossing deadzone boundary	Camera shifts smoothly to re-centre region	Pass	Figures T31.1, T31.2
CD3	Verify no jitter during continuous movement	Hold movement key to repeatedly cross deadzone	Camera updates smoothly without flicker	Pass	Figures T32
CD4	Verify camera clamping at map boundaries	Move player to extreme edges of map	Camera does not scroll beyond map bounds	Pass	Figures T33.1, T33.2

Test Table 7 – Screenshot Evidence

Figure T30.1 – Player Inside Deadzone (Initial Position)

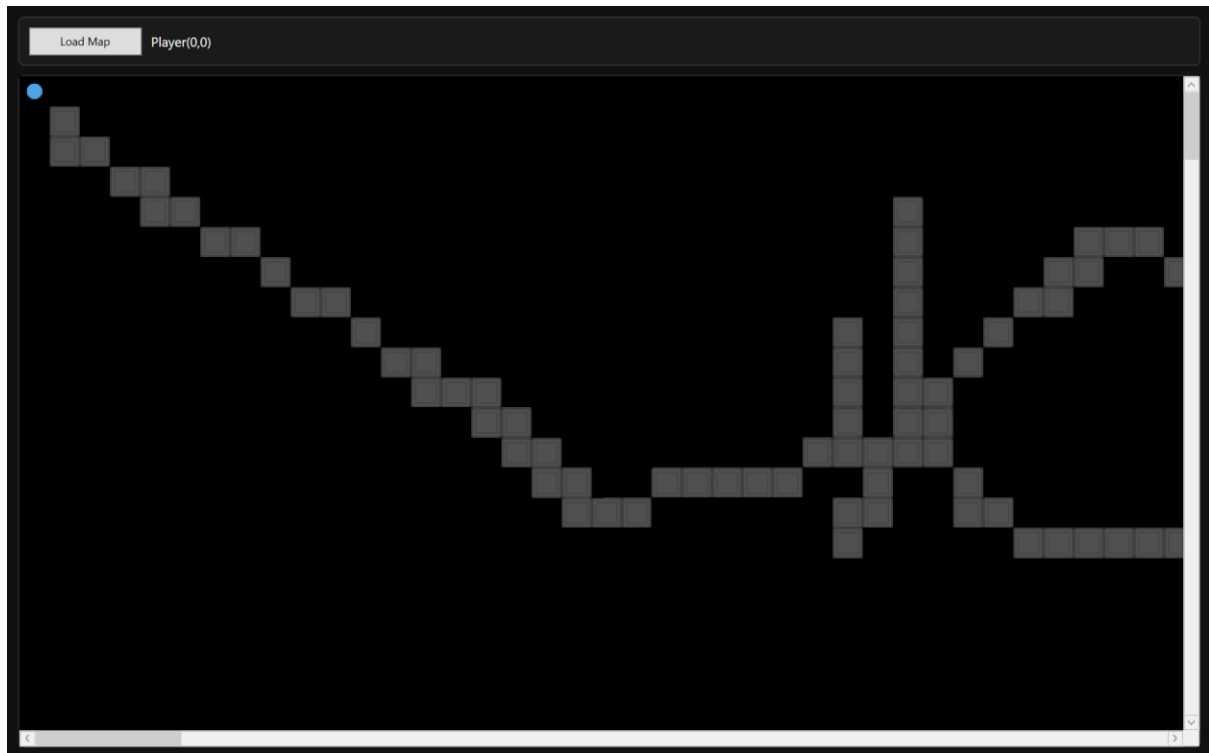
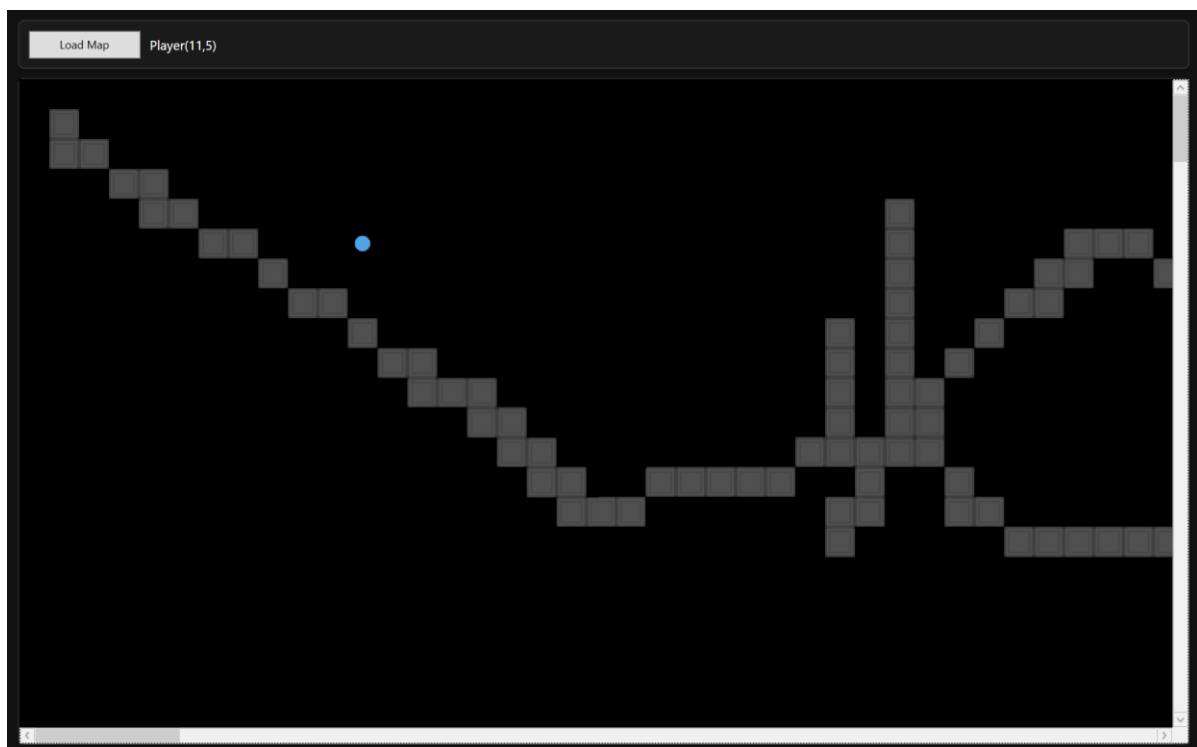


Figure T30.2 – Player Moved Slightly (Camera Unchanged)



Demonstrates that camera does not respond to minor player movement within deadzone

Figure T31.1 – Player at Deadzone Boundary (Before Camera Shift)

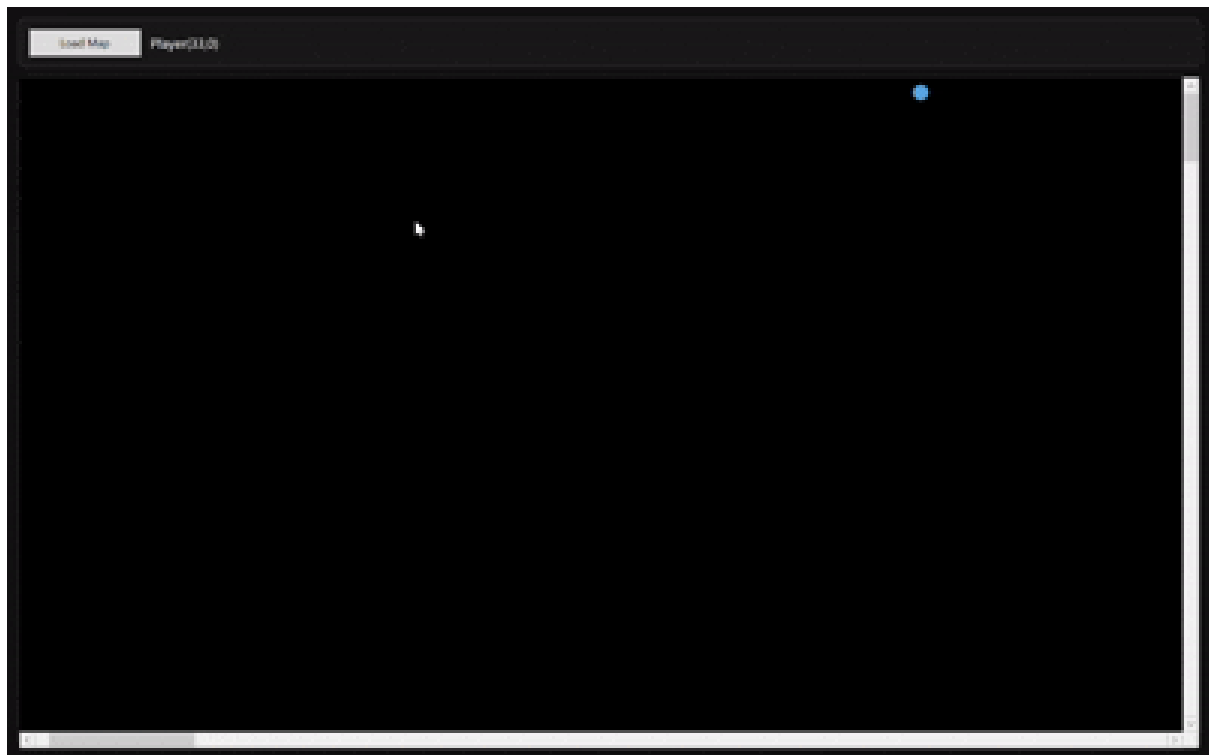


Figure T31.2 – Camera Shift After Crossing Boundary



Demonstrates threshold-based camera activation logic

Figure T32 – GIF Showing Continuous Movement



Demonstrates stable camera update logic across repeated shifts

Figure T33.1 – Camera Clamped at Top/Left Edge

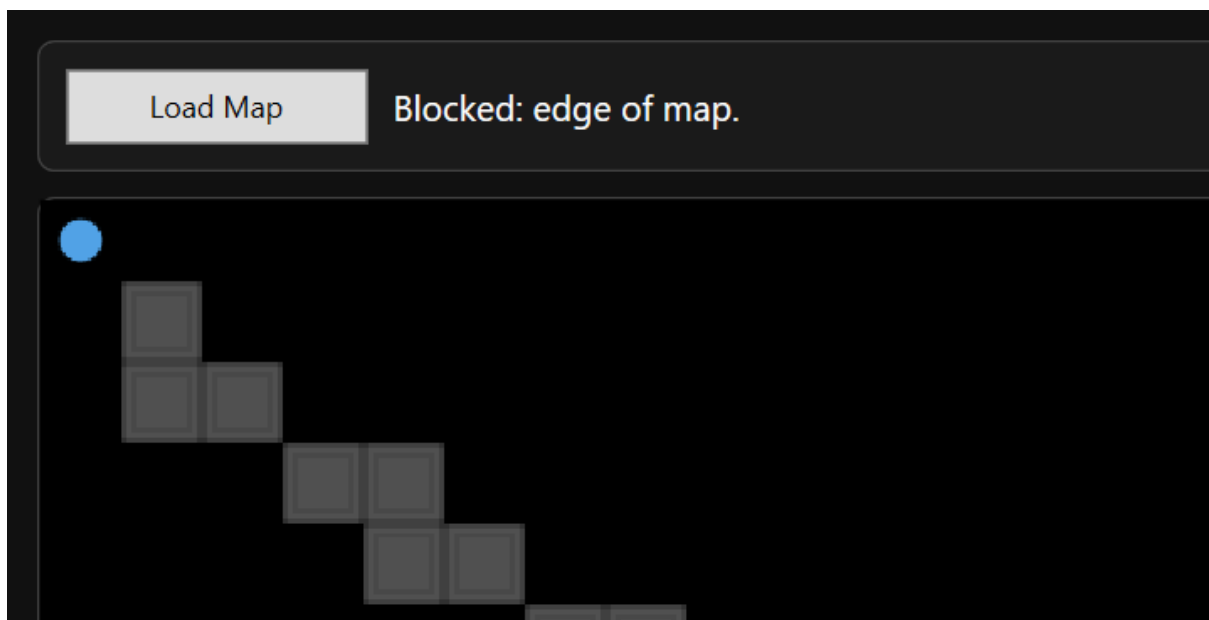
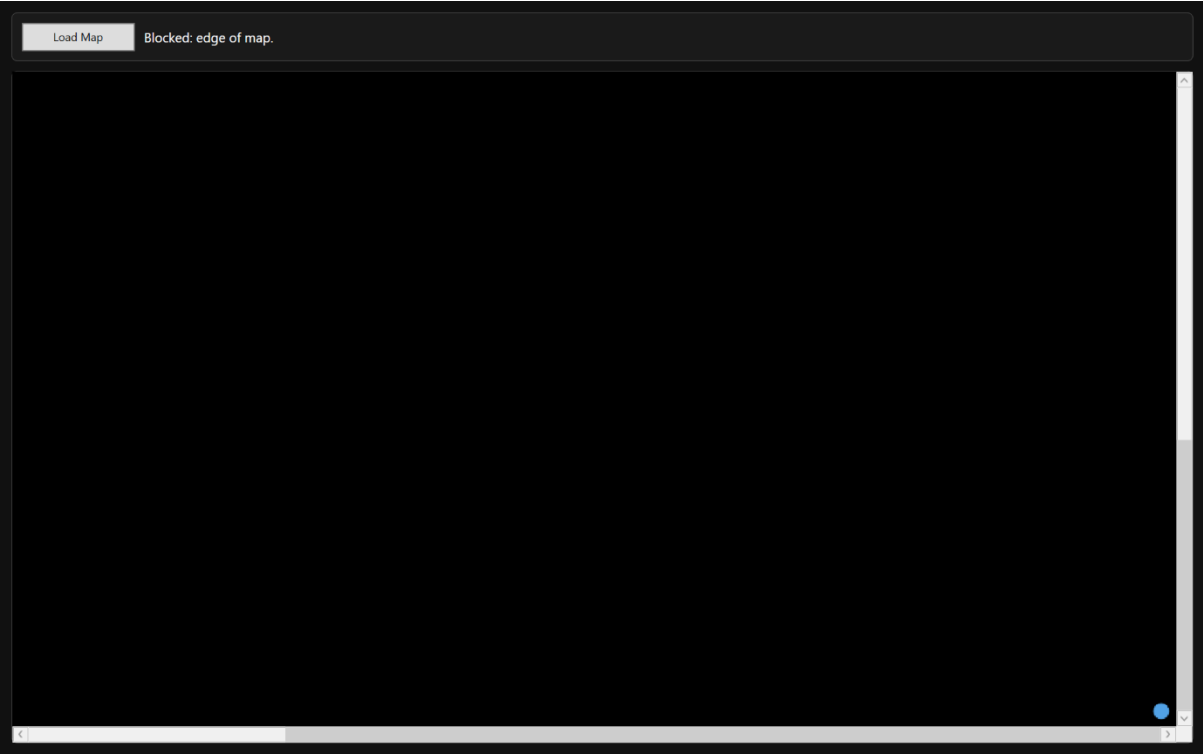


Figure T33.2 – Camera Clamped at Bottom/Right Edge



Demonstrates correct clamping constraints

3 Boundary and Abnormal Data Testing

3.1 Map Dimension Boundary Testing

This test is used to verify that extreme or invalid map dimensions do not cause runtime crashes, invalid array allocation, or graphical instability. It evaluates:

- Minimum acceptable dimensions
- Zero or negative values
- Large grid stress handling

Test Table 1 – Map Dimension Boundary Testing

Test ID	Purpose of Test	Test Data	Expected Outcome	Actual Outcome	Screenshot ID
BDM1	Attempt to create map with zero width	Width = 0	Creation blocked	Pass	Figures T34
BDM2	Attempt to create map with negative height	Height = -5	Creation blocked	Pass	Figures T35
BDM3	Create large map (e.g. 200x200)	200x200 grid	Map rendered correctly	Pass	Figures T36.1, T36.2
BDM4	Load large map into runtime	200x200	Smooth viewport render	Pass	Figures T37

Test Table 1 – Screenshot Evidence

Figure T34 – Zero Width Validation

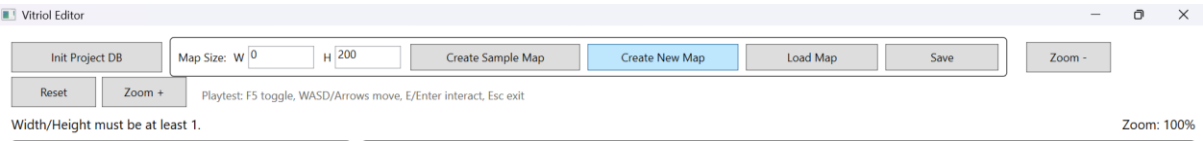


Figure T35 – Negative Height Validation

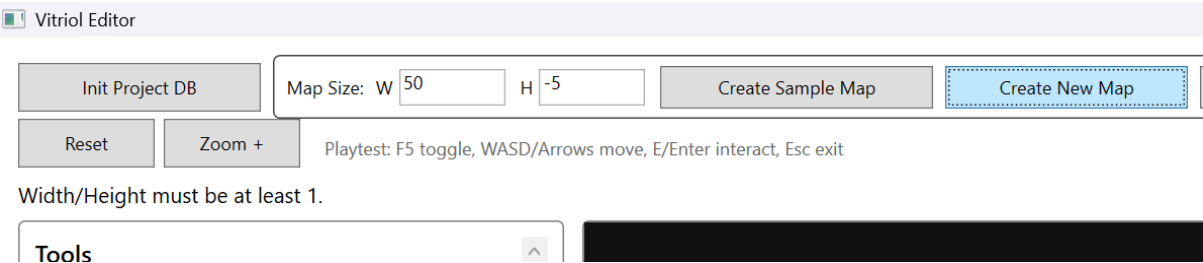


Figure T36.1 – Large Map Creation (200x200)

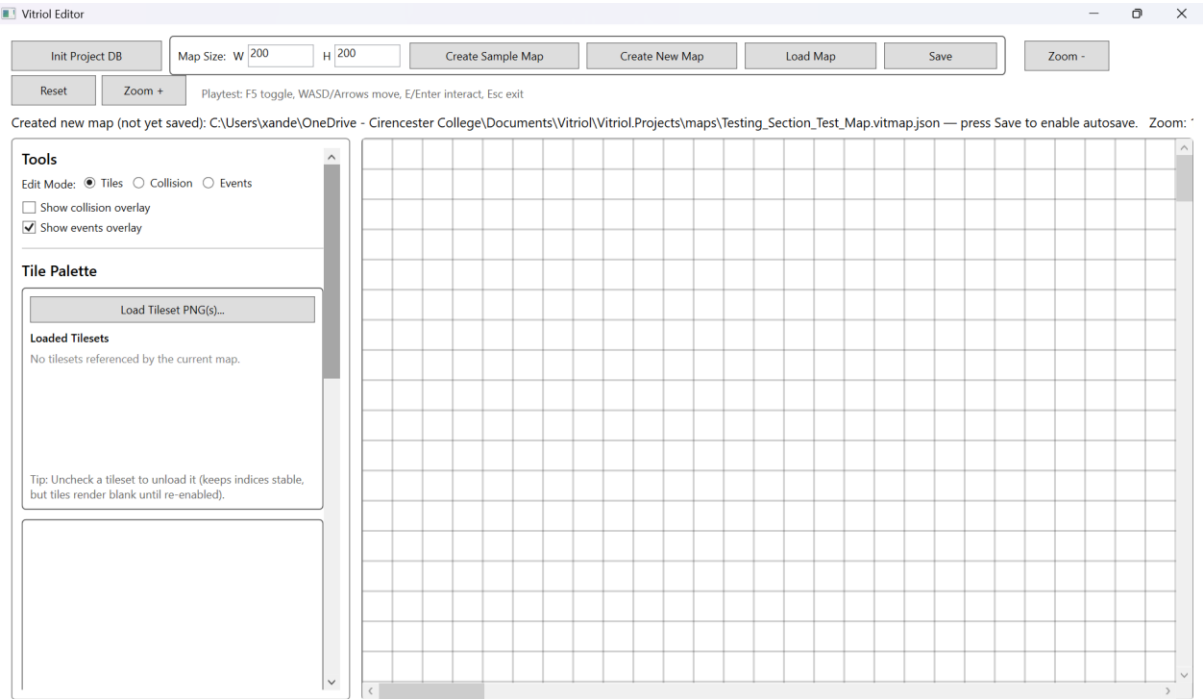


Figure T36.2 – Large Map Viewport Navigation

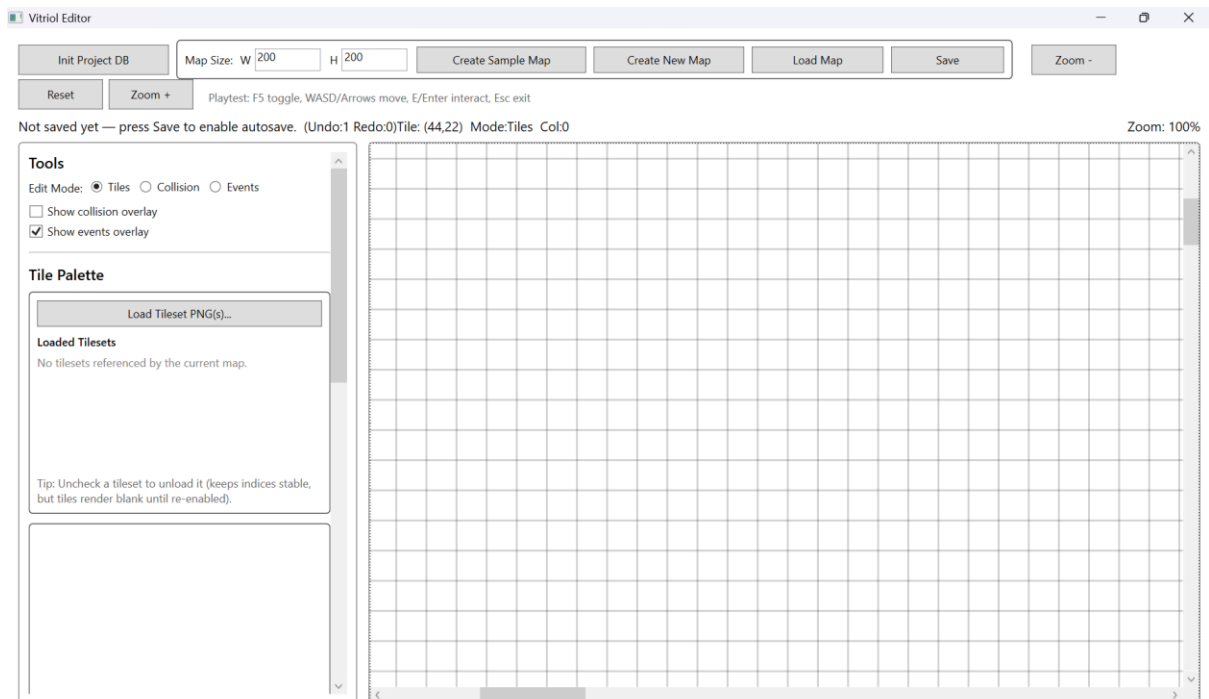
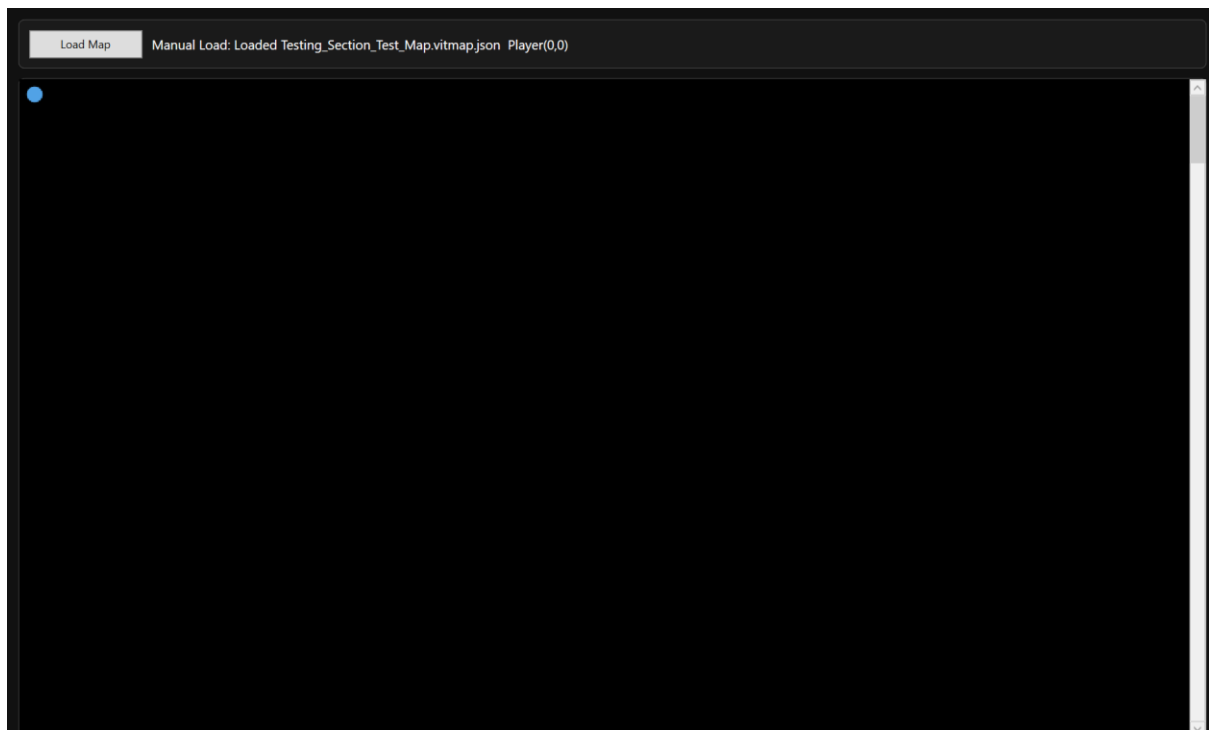


Figure T37 – Runtime Rendering of Large Map



What These Tests Demonstrate:

- Defensive validation against invalid dimension input
- Correct allocation of 1D tile array for large datasets
- Viewport-only rendering prevents performance degradation
- No array index overflow occurs

3.2 Tileset Boundary and Missing Asset Handling

Tilesets represent a critical external dependency within the system. Since maps reference PNG assets stored within the project architecture, it is essential that:

- Missing tileset files are detected before runtime execution
- Invalid relative paths do not cause silent corruption
- Oversized or malformed tilesets do not crash rendering
- Tile indexing remains stable when tilesets are unavailable

This subsection validates defensive asset handling and ensures that map integrity is not compromised by missing or invalid graphical resources

Test Table 2 – Tileset Boundary & Missing Asset Handling

Test ID	Purpose of Test	Test Data	Expected Outcome	Actual Outcome	Screenshot ID
BTS1	Load map referencing missing PNG	Tileset file manually removed from assets folder	Map load blocked with clear error message	Pass	Figures T38
BTS2	Modify JSON to use invalid relative path	ImagePath altered to non-existent path	Runtime validation detects missing file	Pass	Figures T39
BTS3	Load map with valid but large tileset image	High resolution PNG tileset	Tileset loads and crops directly	Pass	Figures T40.1, T40.2
BTS4	Toggle tileset load state	Unload/reload reference	Tile indices remain consistent	Pass	Figures T41.1

Test Table 2 – Screenshot Evidence

Figure T38 – Missing PNG Validation

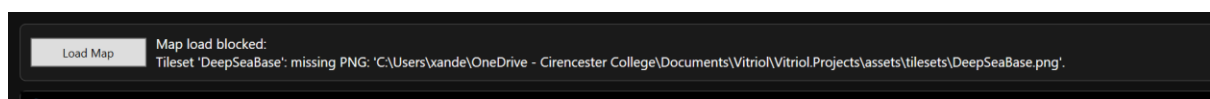


Figure T39 – Invalid Relative Path Detection

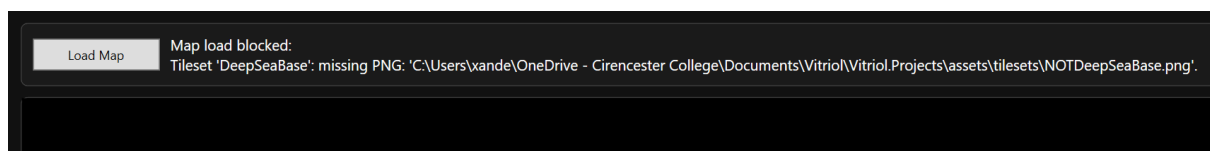


Figure T40.1 – Large Tileset Loaded Successfully

Tile Palette

Load Tileset PNG(s)...

Loaded Tilesets

☒ tileset (assets/tilesets/tileset.png)

Tip: Uncheck a tileset to unload it (keeps indices stable, but tiles render blank until re-enabled).

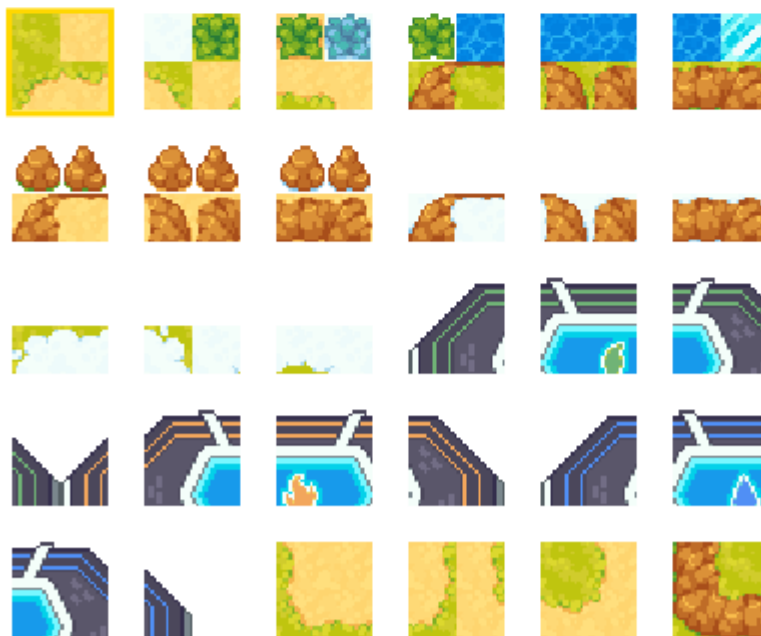


Figure T40.2 – Cropped Rendering From Large Tilesheet

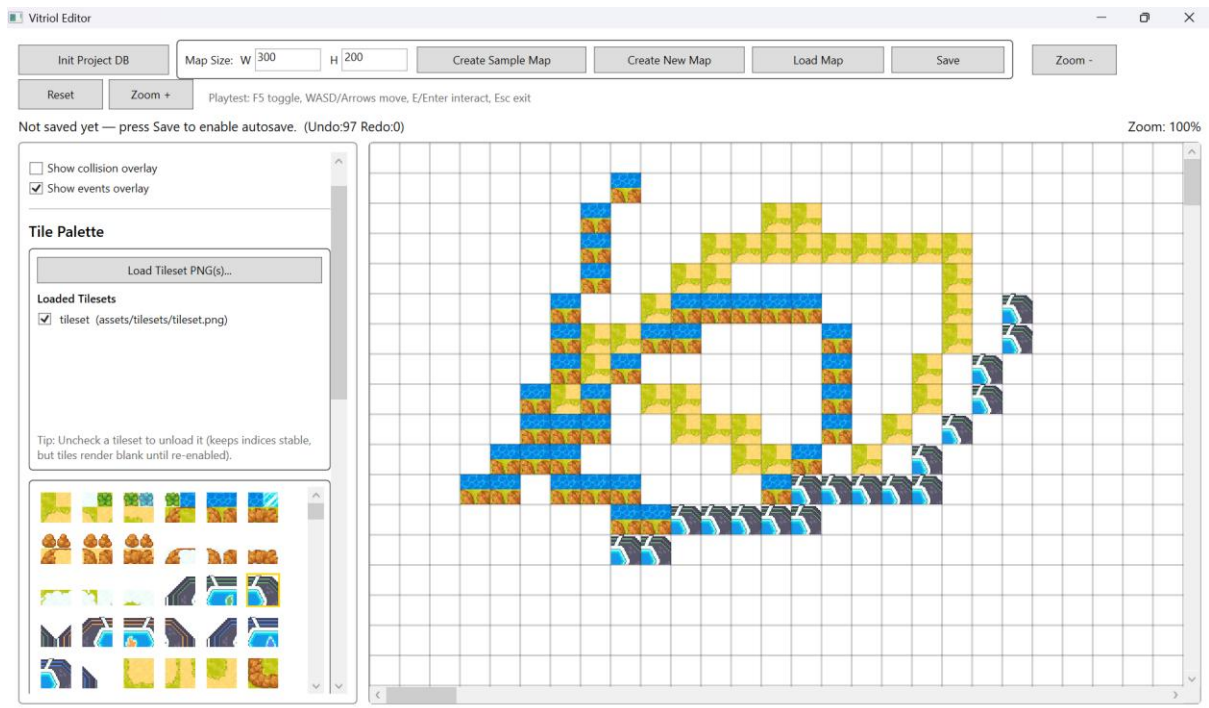
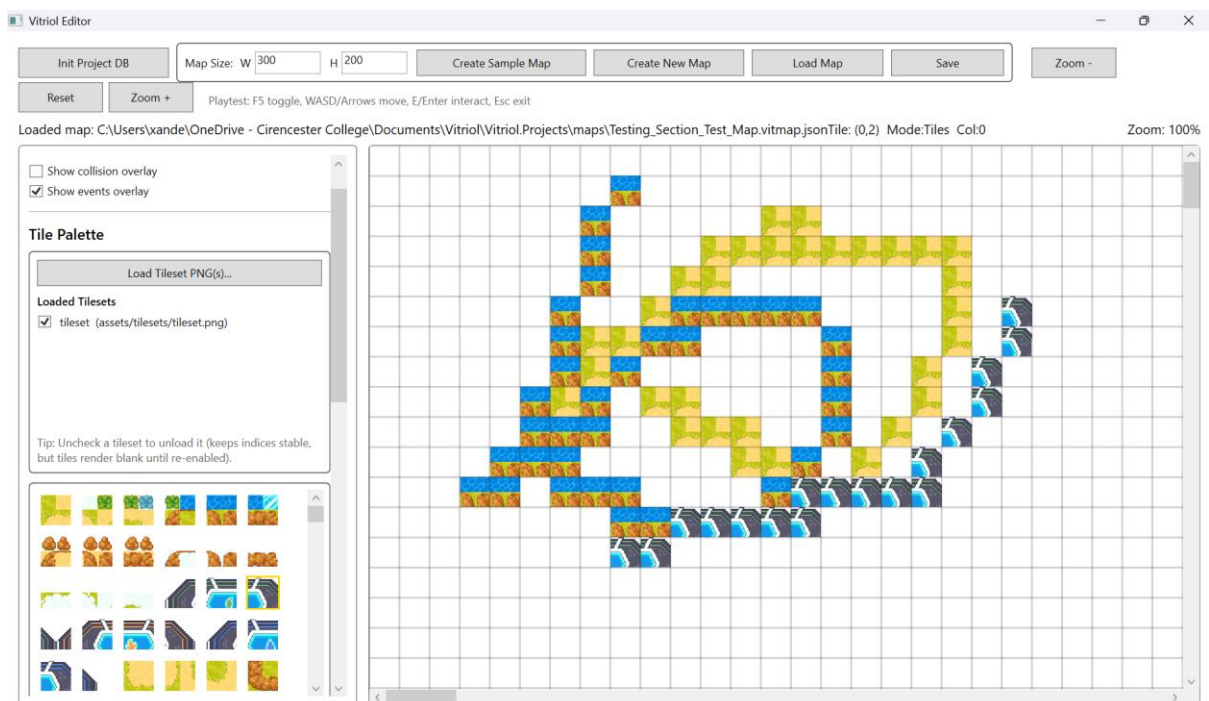


Figure T41 – Tileset Reference Stability



What These Tests Demonstrate:

- Strict enforcement of asset existence before runtime execution
- Protection against broken relative paths
- Safe cropping logic under high-resolution assets
- Stability of global tile indexing system
- Prevention of silent tile corruption

3.3 Event Parameter and Command Validation

Events in Vitriol are data-driven through MapEventDto and EventCommandDto, which means behaviour is controlled entirely by JSON-defined command structures. While this allows extensibility, it also introduces risk if:

- Commands are malformed
- Required arguments are missing
- Unknown command identifiers are used
- Parameter dictionaries contain invalid keys

This subsection verifies that:

- Invalid command data does not crash the runtime
- Missing arguments are handled defensively
- Unknown commands are reported clearly
- Event execution remains stable even with malformed input

Test Table 3 – Event Parameter & Command Validation

Test ID	Purpose of Test	Test Data	Expected Outcome	Actual Outcome	Screenshot ID
BDE1	Execute event with empty command list	Event with no commands	Runtime displays safe message	Pass	Figures T42
BDE2	ShowText command missing “text” argument	Remove text parameter in JSON	Runtime handles gracefully without crash	Pass	Figures T43.1, T43.2
BDE3	Unknown command type	Change command to invalid string	Unknown Command Does Not Execute	Pass	Figures T44

Test Table 3 – Screenshot Evidence

Figure T42 – Empty Command List Execution

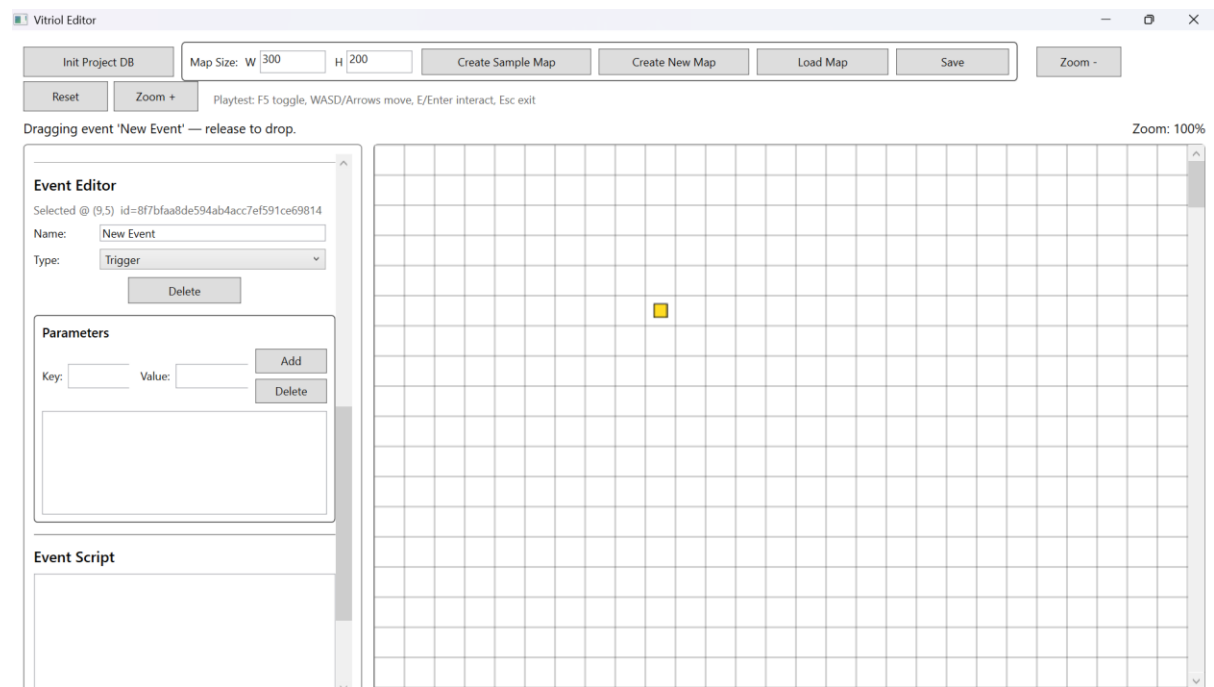


Figure T43.1 – Modified JSON Missing Text Parameter

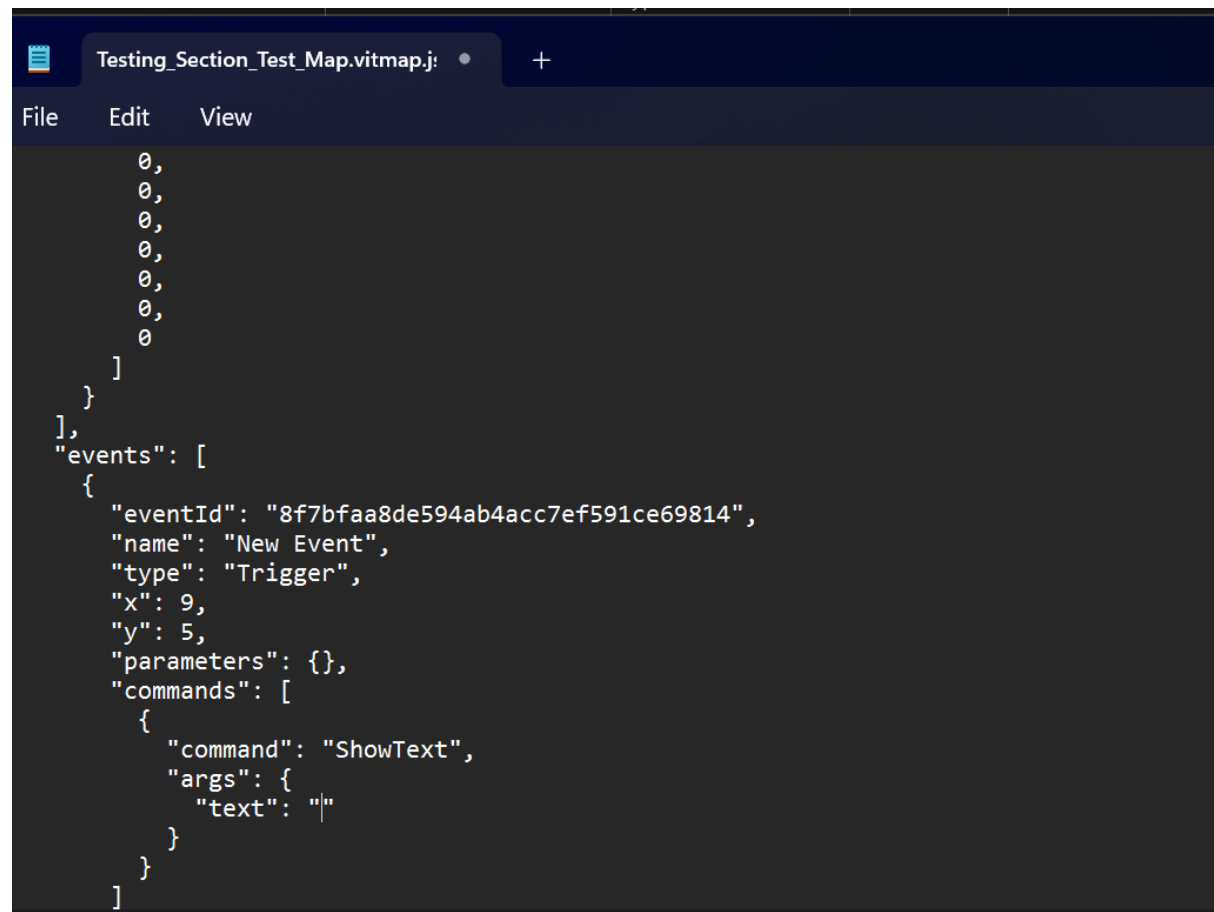


Figure T43.2 – Runtime Handling Missing Text

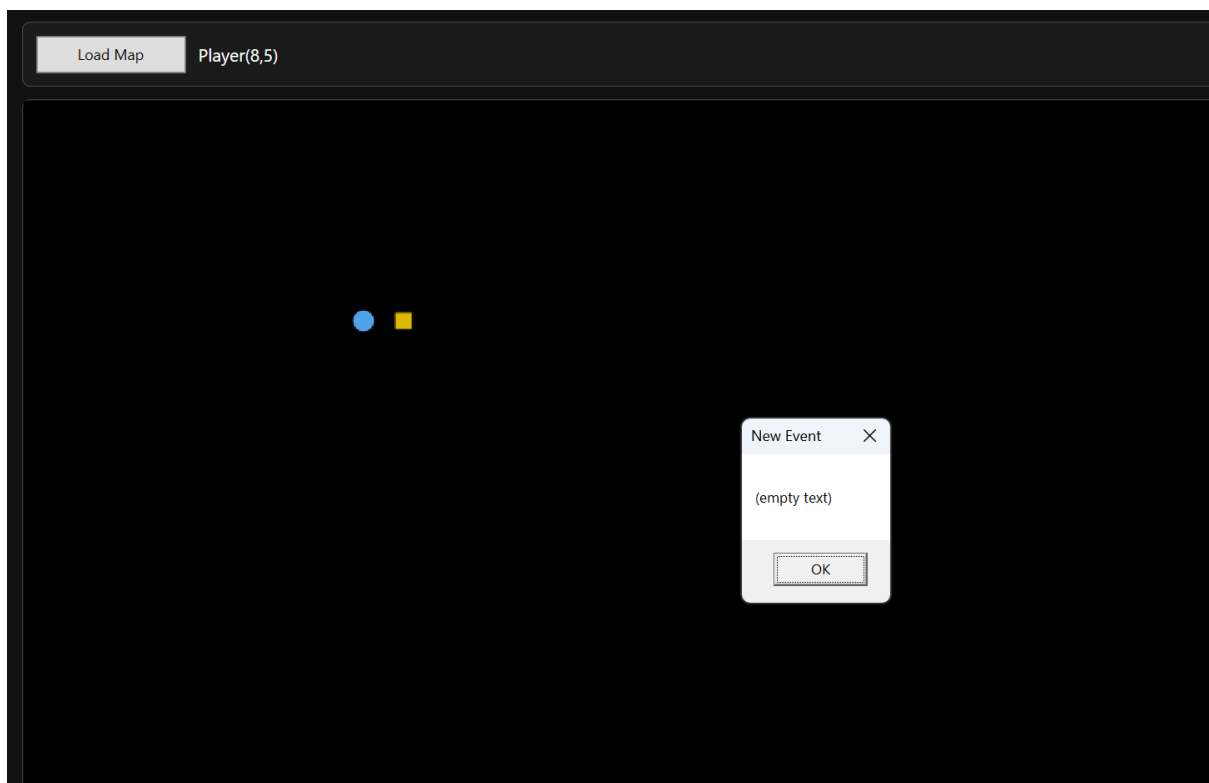
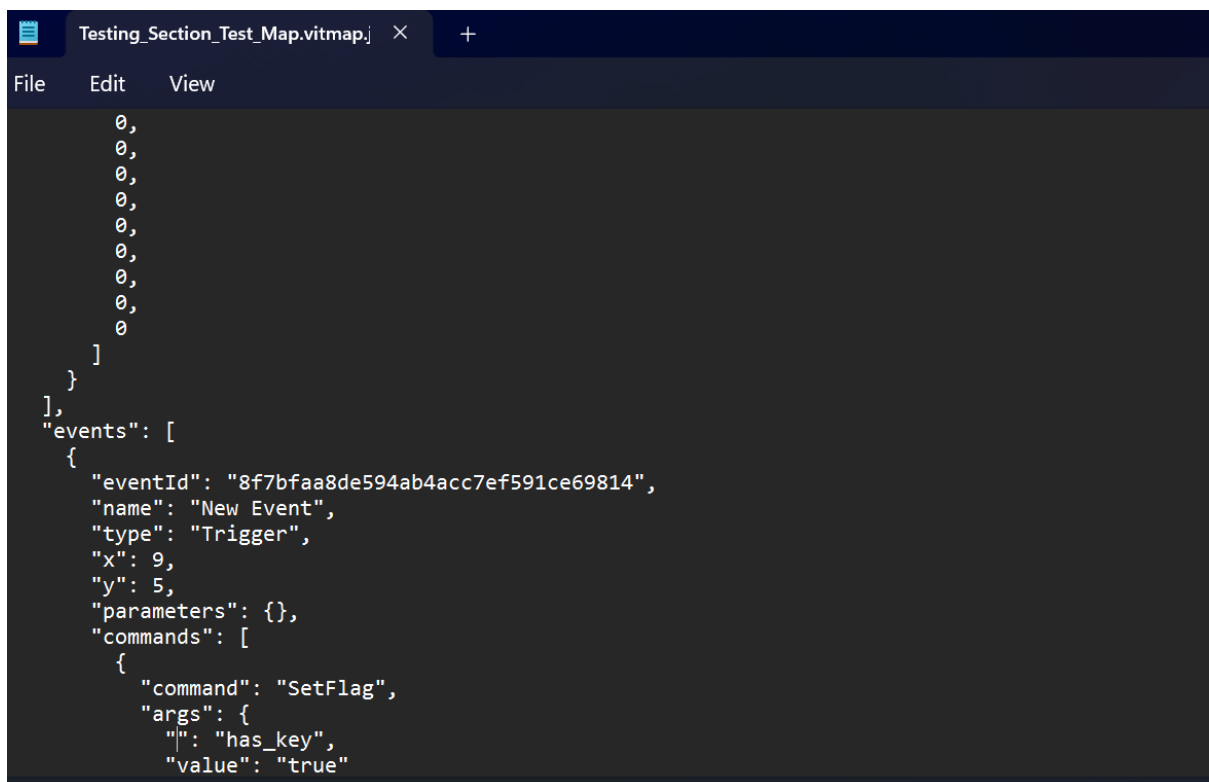


Figure T44 – Unknown Command Handling (Event Does Not Execute)



What These Tests Demonstrate:

- Defensive parsing of command dictionaries
- No runtime crashes from malformed event structures

- Explicit error messaging for missing parameters
- Controlled handling of unknown command identifiers
- Stability of data-driven event system

3.4 JSON Corruption and Recovery Handling

Since maps are persisted as JSON documents, the integrity of the JSON structure is critical. Corruption may occur due to :

- Manual editing errors
- Incomplete writes
- Version mismatches
- Structural deletion of required properties

This subsection verifies that:

- Malformed JSON does not crash the runtime
- Missing required fields are handled safely
- Invalid format versions do not cause undefined behaviour
- File corruption results in controlled failure rather than silent instability

This validates the reliability of the MapSerialiser and defensive file loading logic

Test Table 4 – JSON Corruption & Recovery Handling

Test ID	Purpose of Test	Test Data	Expected Outcome	Actual Outcome	Screenshot ID
BDJ1	Remove closing brace from JSON	Delete final } in map file	Runtime fails safely with error message	Pass	Figures T45
BDJ2	Remove required property width	Delete “Width” field	Deserialization fails safely	Pass	Figures T46
BDJ3	Modify FormatVersion to invalid value	Change to unsupported number	Runtime handles without crash	Pass	Figures T47

Test Table 4 – Screenshot Evidence

Figure T45 – Malformed JSON (Missing Closing Brace)

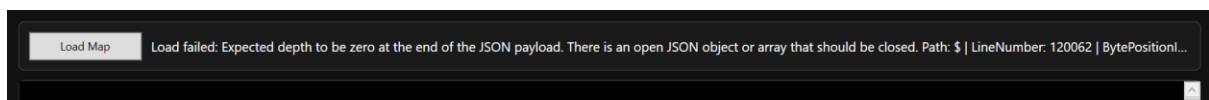
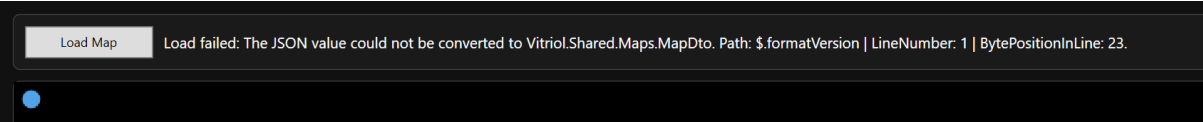


Figure T46 – Missing Required Property



Figure T47 – Invalid FormatVersion



What These Tests Demonstrate:

- Safe deserialization failure handling
- No unhandled exceptions during file parsing
- Strict structural requirements enforced
- Protection against partial or corrupted file writes
- Data integrity maintained across persistence layer

3.5 Runtime Movement Boundary Testing

Player movement operates directly on tile-indexed arrays. Any failure in boundary enforcement could result in:

- Array index out-of-range exceptions
- Clipping outside map bounds
- Ignoring collision constraints
- Camera instability

This subsection verifies that:

- Player movement is clamped within map dimensions
- Collision tiles correctly block movement
- No index overflow occurs at edges
- Camera deadzone logic does not break at boundaries

Test Table 5 – Runtime Movement Boundary Testing

Test ID	Purpose of Test	Test Data	Expected Outcome	Actual Outcome	Screenshot ID
BDR1	Attempt movement beyond left edge	Move player at x=0 left	Movement Blocked	Pass	Figures T48
BDR2	Attempt movement beyond top edge	Move player at y=0 up	Movement Blocked	Pass	Figures T49
BDR3	Attempt movement into collision tile	Walk into-collision marked tile	Collision Respected	Pass	Figures T50
BDR4	Camera behaviour at map edge	Move to far right/bottom edge	Camera Clamps Correctly	Pass	Figures T51

Test Table 5 – Screenshot Evidence

Figure T48 – Left Boundary Enforcement

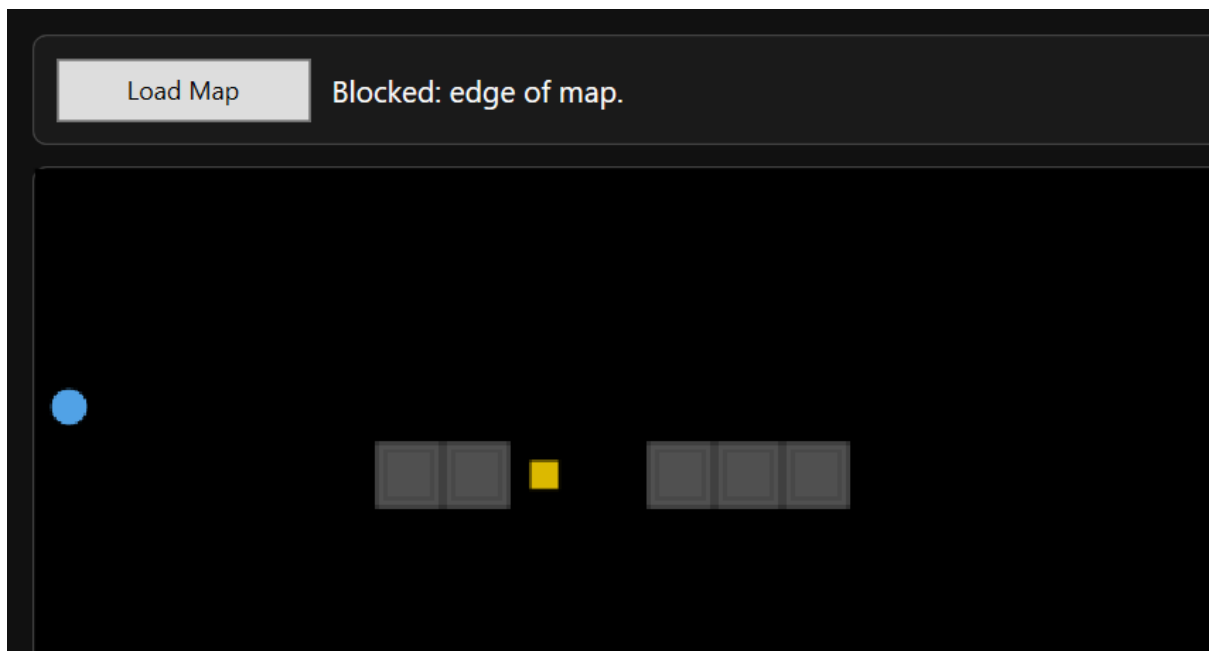


Figure T49 – Top Boundary Enforcement

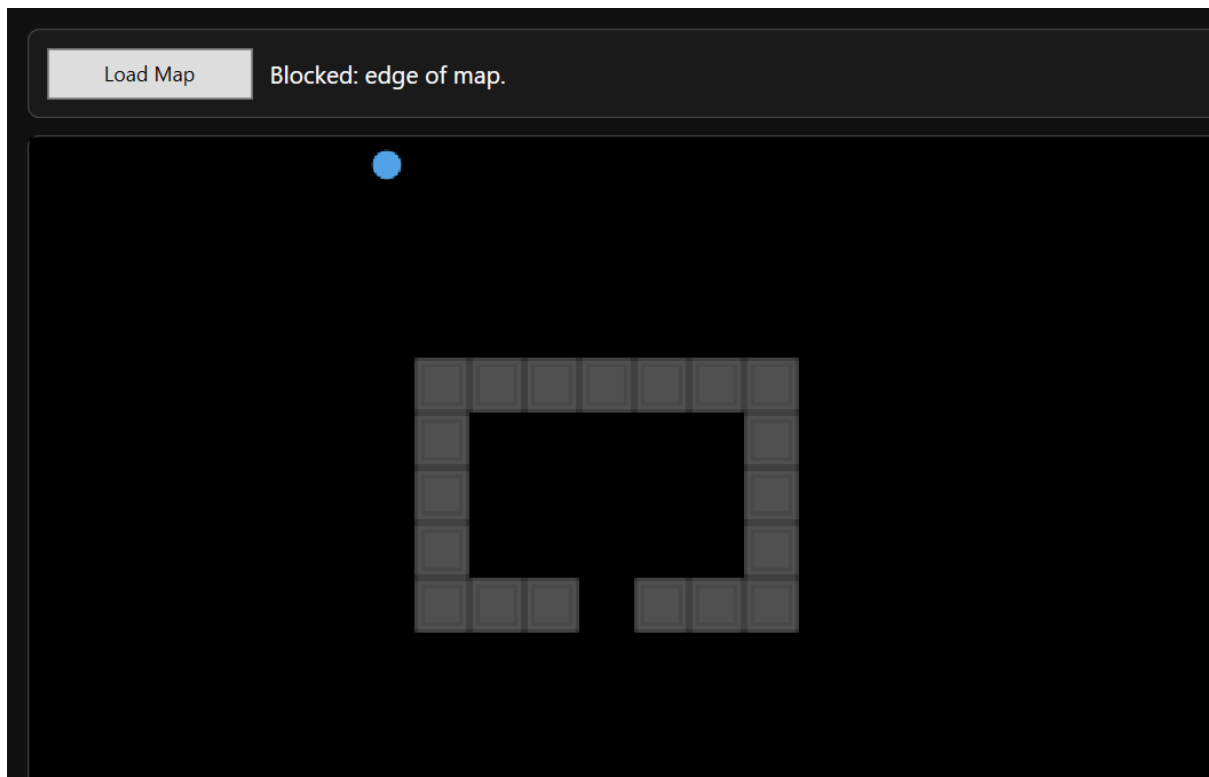


Figure T50 – Collision Tile Enforcement

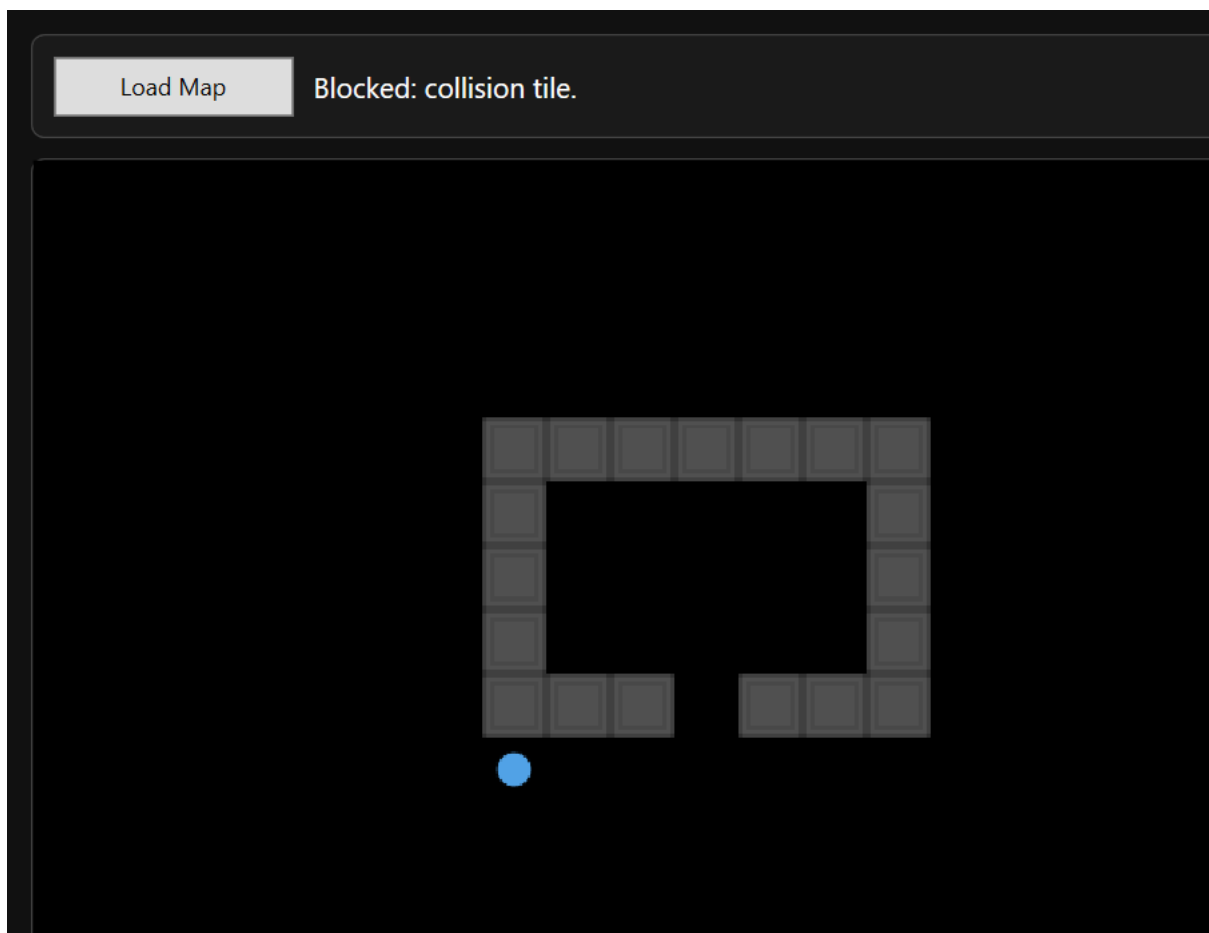
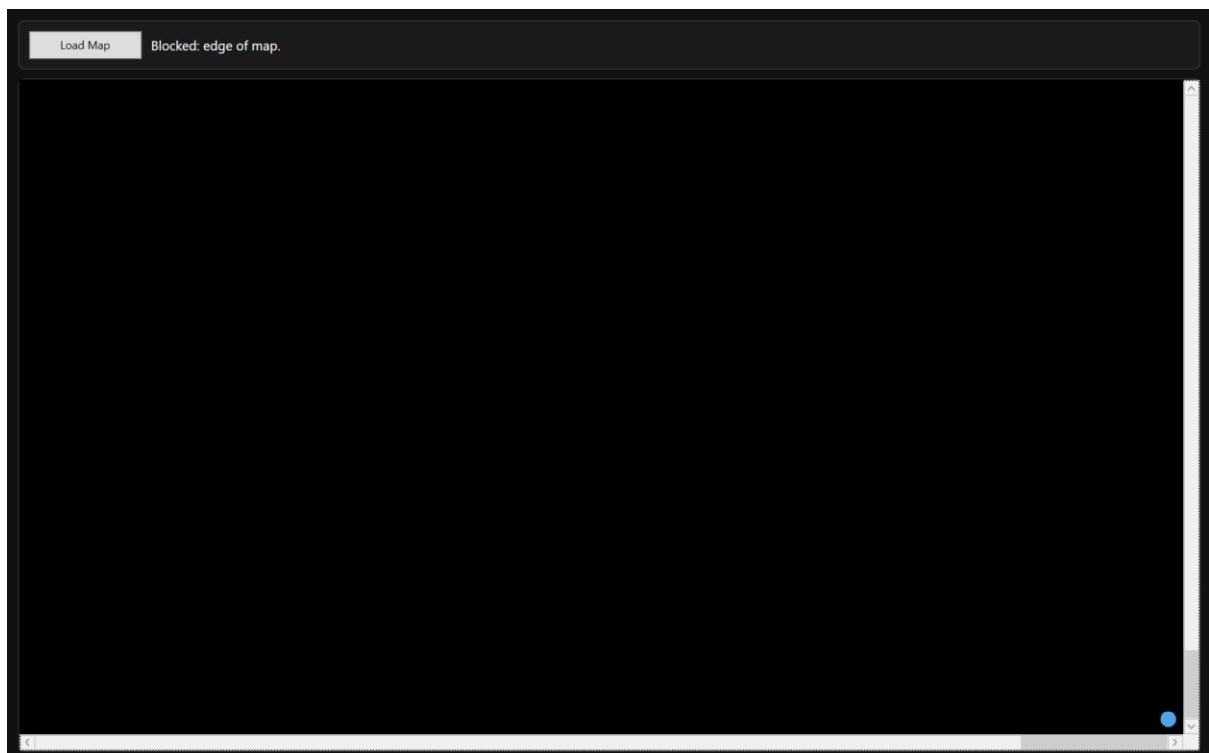


Figure T51 – Camera Clamping at Edge



What These Tests Demonstrate:

- Safe array index clamping
- Collision layer correctly evaluated
- No runtime exceptions at map edges
- Camera deadzone behaviour remains stable at boundaries
- Movement system remains deterministic and robust

4 Performance Testing

4.1 Large Map Rendering Performance

These tests confirm that viewport-only rendering prevents performance degradation on large maps, and that rendering remains smooth during navigation.

Test Table 1 – Large Map Rendering Performance

Test ID	Purpose of Test	Test Data	Expected Outcome	Actual Outcome	Screenshot ID
PERFR1	Render 200x200 map in editor	Large grid with background tiles	Smooth viewport navigation	Pass	Figures T52
PERFR2	Continuous scrolling across large screen	Hold movement key	Rendering remains stable	Pass	Figures T53
PERFR3	Load large map in runtime	200x200 grid	Runtime renders only visible region	Pass	Figures T54
PERFR4	Rapid directional input	Alternate WASD quickly	No frame freezing or jitter	Pass	Figures T55

Test Table 1 – Screenshot Evidence

Figure T52 – GIF Showing Editor Large Map Navigation



Figure T53 – GIF Showing Continuous Movement Stress



Figure T54 – Runtime Large Map Render (200x200)



Figure T55 – GIF Showing Rapid Input Responsiveness



What These Tests Demonstrate:

- Rendering workload remains proportional to viewport size
- Tile cropping and caching prevent redundant redraw
- No exponential performance cost as map dimensions increase
- Input handling remains responsive under continuous rendering

4.2 Rendering Stability Under Continuous Movement

While large map rendering validates immediate responsiveness, it is equally important to ensure that performance remains stable over time. Extended runtime sessions may expose:

- Memory leaks in bitmap caching
- Gradual rendering slowdown
- Accumulated camera offset errors
- Resource exhaustion under repeated movement

Given that the system uses:

- Frozen BitmapSource objects
- Cached CroppedBitmap instances
- Debounced render calls
- Single RenderTargetBitmap per frame

This subsection verifies that long-duration usage does not degrade responsiveness or introduce instability.

Test Table 2 – Rendering Stability Under Continuous Movement

Test ID	Purpose of Test	Test Data	Expected Outcome	Actual Outcome	Screenshot ID
---------	-----------------	-----------	------------------	----------------	---------------

PERFS1	Continuous movement for 20+ seconds	Hold movement keys	No frame degradation over time	Pass	Figures T56
PERFS2	Repeated directional change	Rapid alternating movement	No camera jitter or offset drift	Pass	Figures T57
PERFS3	Repeated viewport scrolling (editor)	Continuous scroll wheel use	No UI freeze or lag	Pass	Figures T58
PERFS4	Multiple map reload cycles	Load map repeatedly	No progressive slowdown	Pass	Figures T59

Test Table 2 – Screenshot Evidence

Figure T56 – GIF Showing Extended Movement Stability

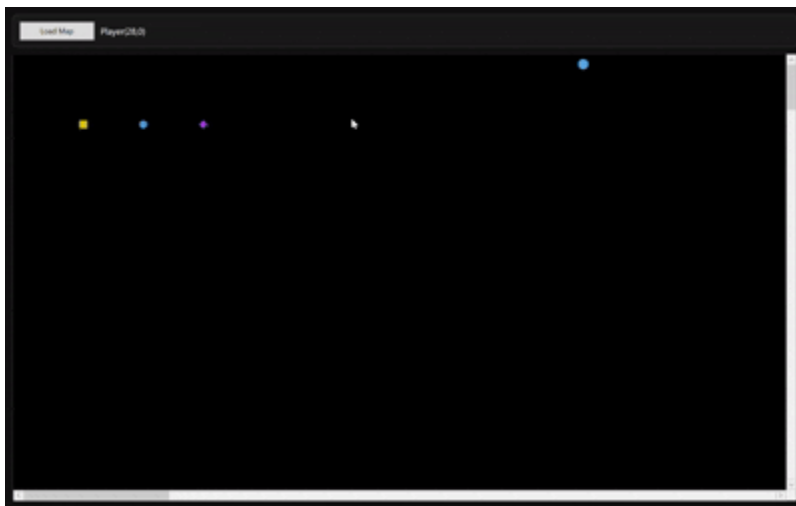


Figure T57 – GIF Showing Camera Stability During Rapid Direction Change



The screenshot shows the Mind Editor application. At the top, there's a menu bar with 'File', 'Edit', 'Map', 'Help', and 'Window'. Below it is a toolbar with buttons for 'Load Project DB', 'Map Size' (set to 100), 'Create Sample Map', 'Create New Map', 'Load Map', 'Save', and 'Zoom'. The 'Map Size' dropdown is currently set to 100. Below the toolbar is a status bar showing 'Project: C:\Users\ander\OneDrive - Crencaster College\Documents\Mind Editor\Project\map1\test_map_01\map1.json' and 'Zoom: 100%'. The main area is a large grid representing the map. On the left side, there's a sidebar with a 'Tools' section containing 'Edit Mode' (selected), 'Tiles', 'Collisions', and 'Events'. Below this are checkboxes for 'Show collision overlay' (unchecked) and 'Show events overlay' (checked). The 'Tile Palette' section has a 'Load Tileset (PNG)' button. Below that is a 'Loaded Tilesets' section with the text 'No tilesets referenced by the current map.' At the bottom of the sidebar, there's a note: 'Tip: Uncheck a tileset to utilize it. Despite indices stable, but this render block until re-enabled.' The main map area shows a grid with a small black square in the bottom right corner.

The screenshot shows the 'Open File Map' dialog box in QGIS. The file explorer view displays the contents of the 'test_Map.vlmr.png' file. The file name 'test_Map.vlmr.png' is entered in the 'File name' field. The 'Open' button is highlighted.

- No observable memory leak from tile caching
- Camera deadzone remains stable over time
- Debounced rendering prevents excessive redraw calls
- Rendering remains proportional to viewport size
- Load operations do not degrade progressively

This subsection evaluates whether memory consumption increases progressively during extended editing or runtime use. Since the system caches:

- It is critical to ensure:

- No uncontrolled memory growth

- No cumulative bitmap duplication
- No progressive slowdown after repeated render cycles

Test Table 3 – Memory Usage During Extended Editing Session

Test ID	Description	Test Data	Expected Outcome	Actual Outcome	Screenshot ID
PERFM1	Continuous tile painting in editor	Rapid paint actions for 20+ seconds	No UI slowdown	Pass	Figures T60
PERFM2	Repeated viewport renders	Scroll large map repeatedly	No progressive lag	Pass	Figures T61
PERFM3	Repeated runtime movement	Continuous WASD movement	No increasing delay	Pass	Figures T62

Test Table 3 – Screenshot Evidence

Figure T60 – GIF Showing Continuous Tile Painting (Editor Memory Stability)

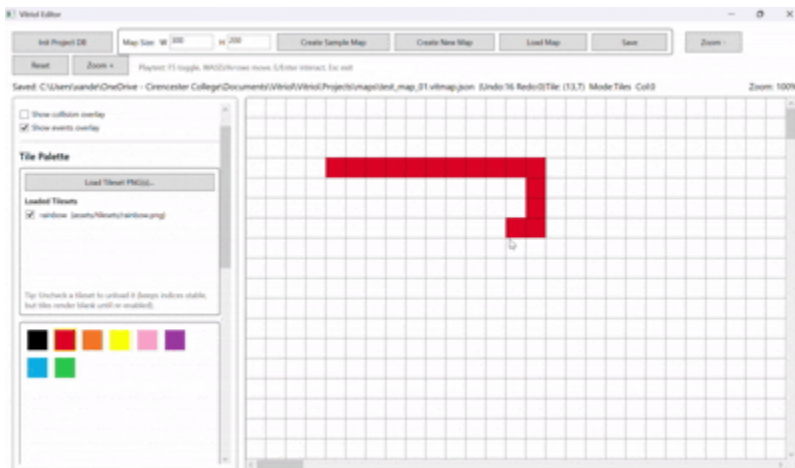


Figure T61 – GIF Showing Repeated Viewport Rendering (Editor Scroll Stress)

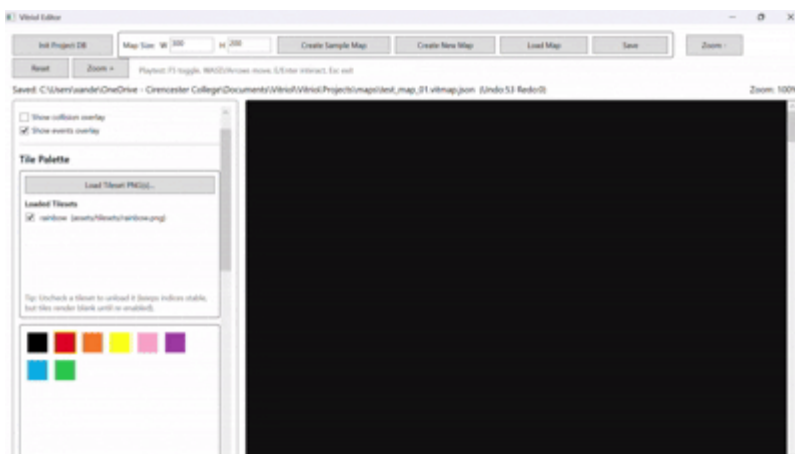
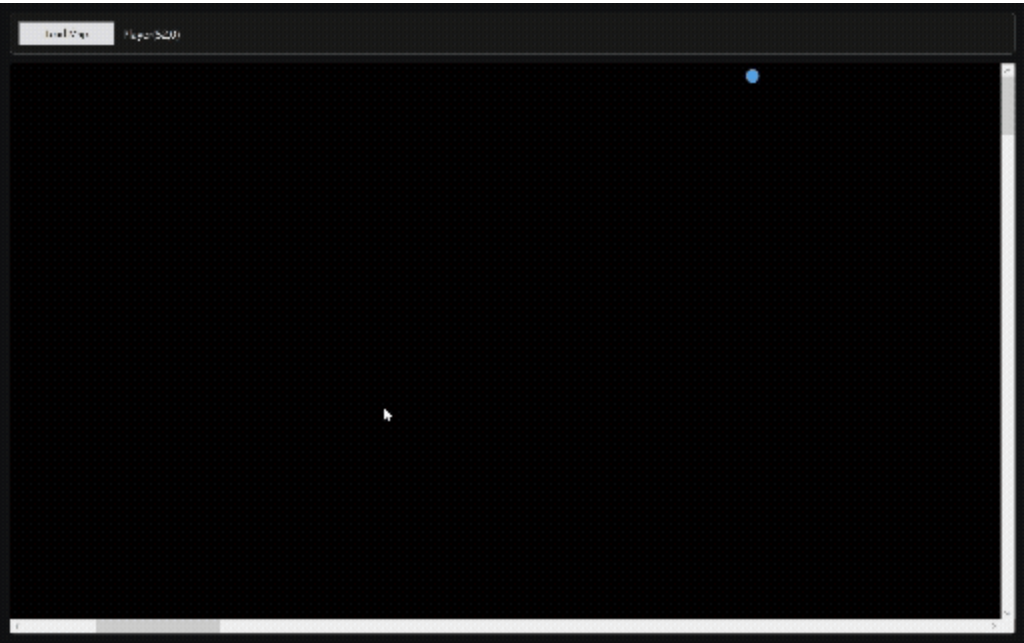


Figure T62 – GIF Showing Repeated Runtime Movement



What These Tests Demonstrate:

- No progressive slowdown from repeated tile updates
- Viewport-only rendering remains stable over repeated calls
- No memory leak symptoms or input degradation

4.4 Asset Loading Performance

Asset loading performance evaluates the time required to:

- Load tileset PNG file
- Compile tile metadata
- Cache cropped bitmap segments
- Render initial viewport

Given that tilesets are loaded once and cached, subsequent map loads should not significantly degrade performance.

This subsection verifies that:

- First load is acceptable
- Subsequent loads are not progressively slower
- Large tilesets do not freeze the UI

Test Table 4 – Asset Loading Performance

Test ID	Description	Test Data	Expected Outcome	Actual Outcome	Screenshot ID
PERFA1	First load of large tileset	Input resolution PNG	Load completes without freezing	Pass	Figures T63

PERFA2	Reload same map	Cached tileset	Faster or equal load time	Pass	Figures T64
PERFA3	Switch between maps repeatedly	Alternate map loads	No UI Hang	Pass	Figures T65

Test Table 4 – Screenshot Evidence

Figure T63 – First Load of Large Tileset

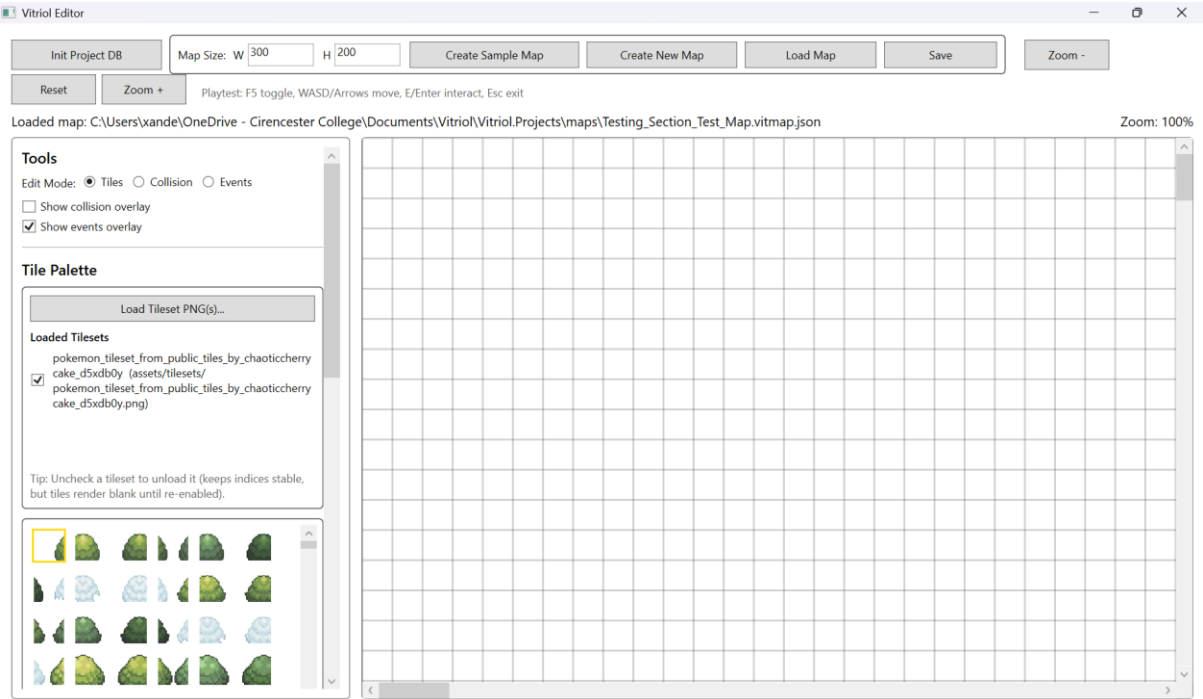


Figure T64 – GIF Showing Reload of Same Map (Cache Validation)

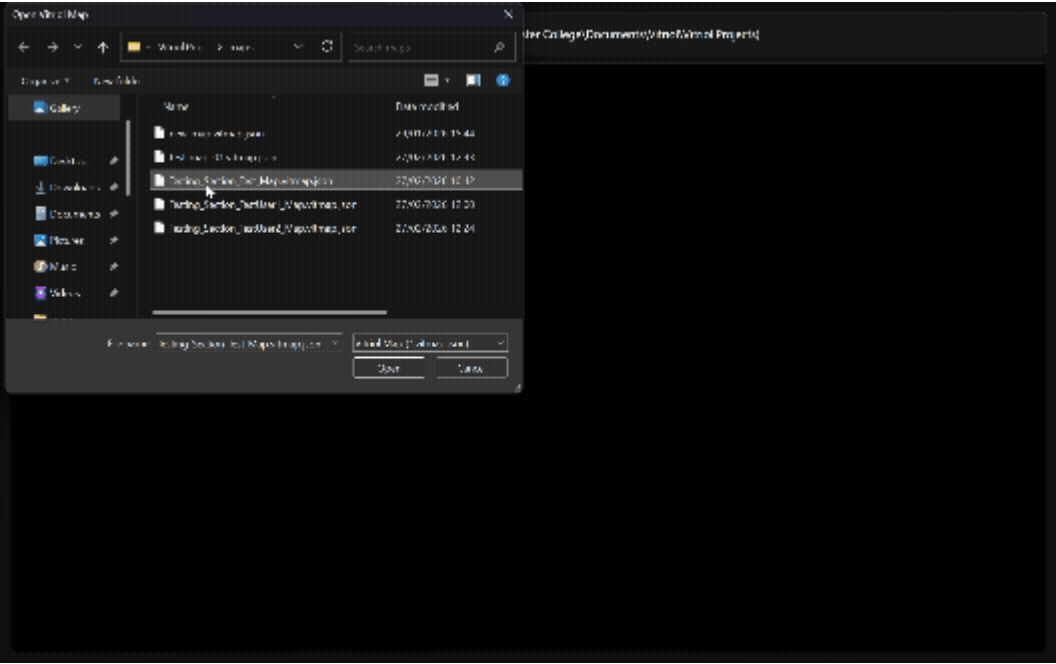
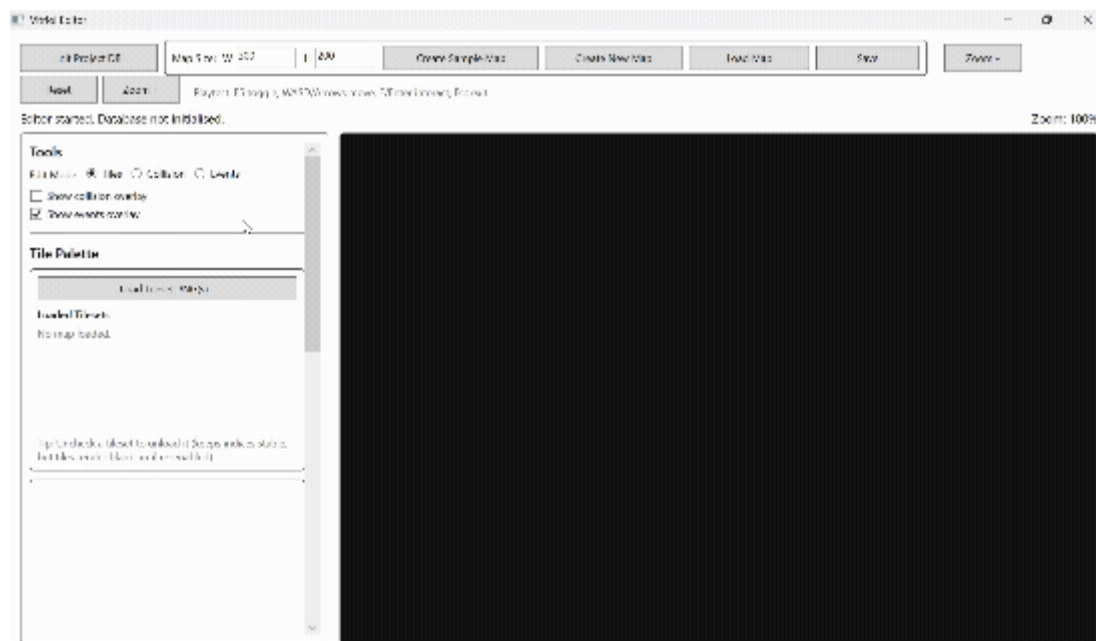


Figure T65 – GIF Showing Switch Between Maps



What These Tests Demonstrate:

- Initial PNG decoding and tileset compilation completes successfully
- No progressive delay, caching remains stable
- Repeated map loading does not accumulate performance degradation

5 Regression Testing

5.1 Map Creation Regression Checks

These checks confirm that later changes (tileset validation, autosave rules, viewport renderer integration) did not break initial map creation workflows. This includes creation of both sample maps and user-created mas, plus correct initial save state behaviour.

Test Table 1 – Map Creation Regression Checks

Test ID	Test Data	Expected Outcome	Actual Outcome	Status	Screenshot ID
REGMC1	Create new user map (blank grid)	Map created with correct dimensions and blank layers	Map created successfully	Pass	Figures T66
REGMC2	Manual save required before autosave enabled	Autosave inactive until first manual save	Save rule enforced correctly	Pass	Figures T67
REGMC3	Sample map creation autosaves by default	Autosave active immediately for sample map	Autosave active as expected	Pass	Figures T68

REGMC4	Reload newly created map file	Map reloads correctly with no missing data	Reload successful	Pass	Figures T69
---------------	-------------------------------	--	-------------------	------	-------------

Test Table 1 – Screenshot Evidence

Figure T66 – New User Map Created (Blank Grid)

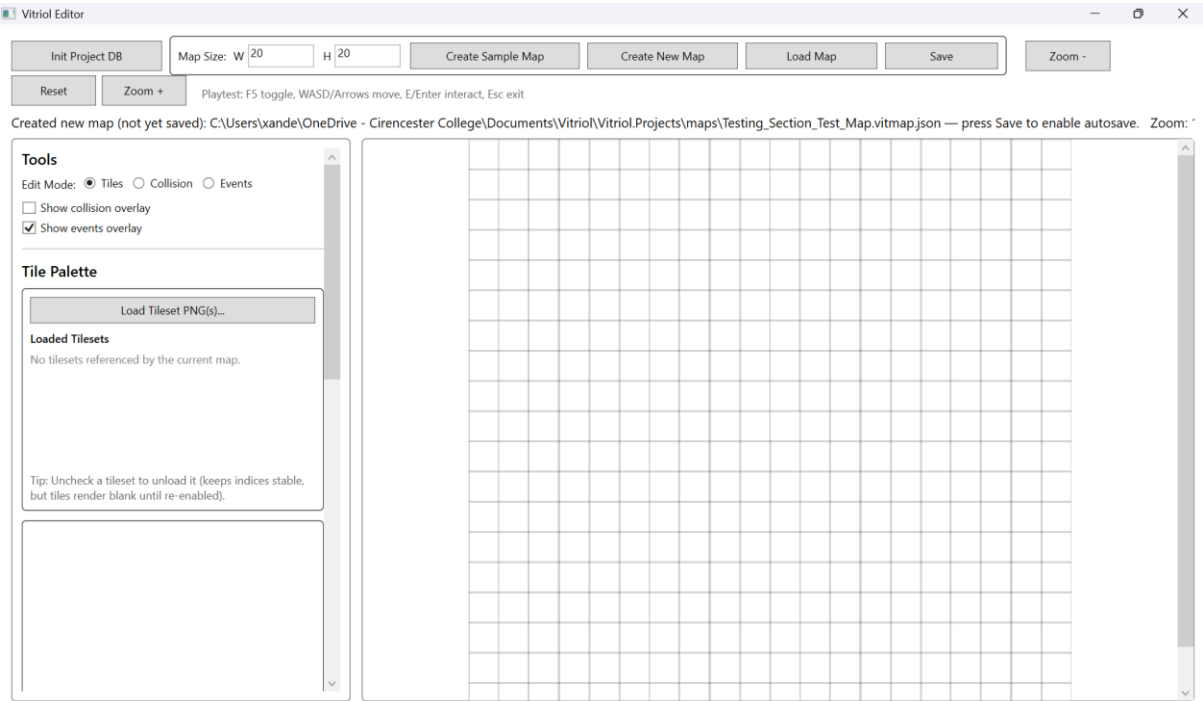


Figure T67 – Manual Save Gate Before Autosave

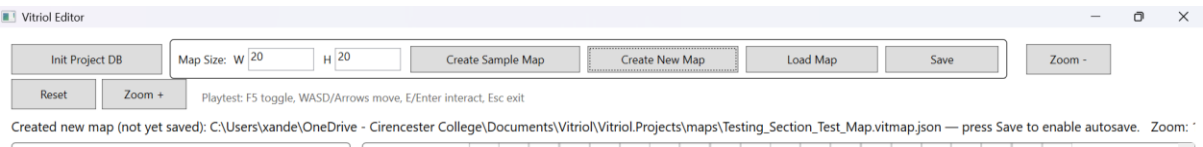


Figure T68 – Sample Map Autosave Active by Default

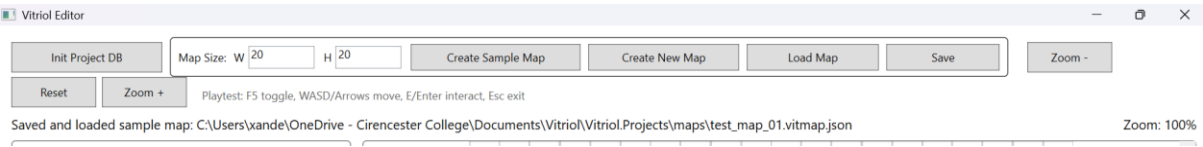
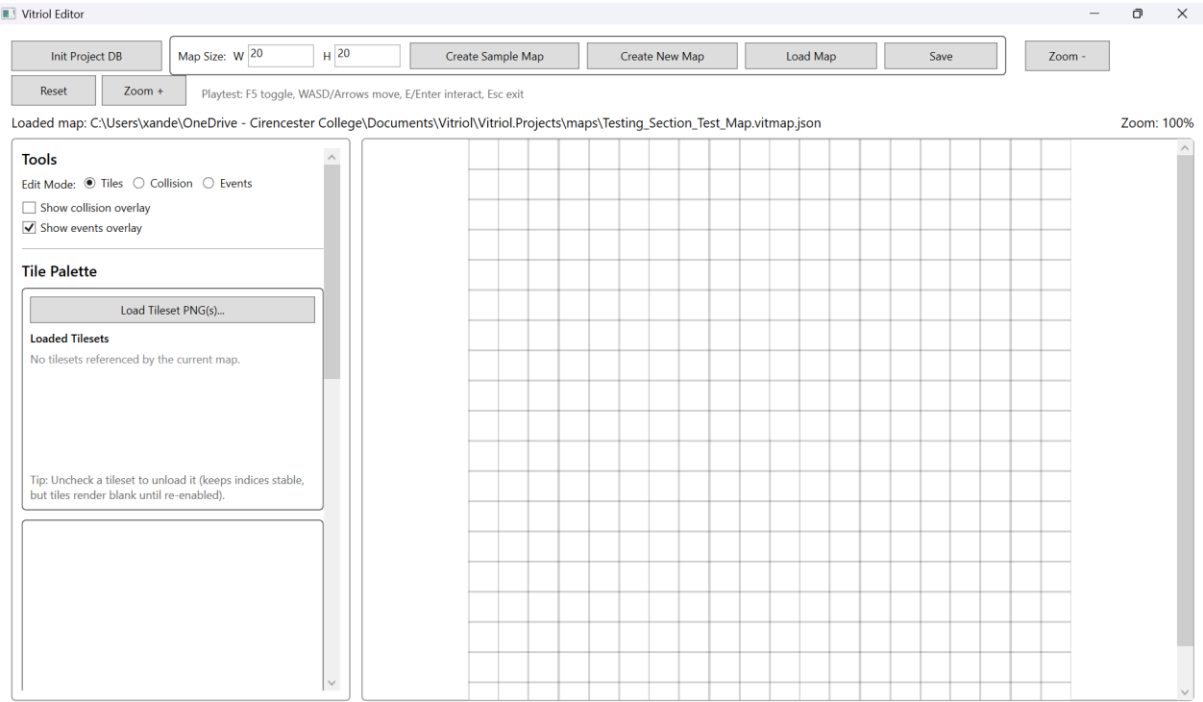


Figure T69 – Reload Newly Created Map



What These Tests Demonstrate:

- Correct initial map creation state and correct grid dimensions
- Autosave rule enforcement prior to first manual save
- Immediate autosave behaviour on sample map workflow
- Creation output produces a valid reloadable map file

5.2 Event System Regression Checks

These tests confirm that runtime changes (camera behaviour, rendering pipeline, tileset caching) did not disrupt event triggering and command execution. This focuses on ensuring NPC, Trigger, and Warp event types still function predictably

Test Table 2 – Event System Regression Checks

Test ID	Description	Expected Outcome	Actual Outcome	Status	Screenshot ID
REGEV1	Trigger event executes ShowText	Dialogue appears with correct text	ShowText displayed correctly	Pass	Figures T70
REGEV2	NPC event executes ShowText	Dialogue appears when interacting with NPC	NPC interaction works	Pass	Figures T71
REGEV3	Warp event loads destination	Map loads and player relocates	Warp Successful	Pass	Figures T72

Test Table 2 – Screenshot Evidence

Figure T70 – Trigger Event Execution (ShowText)

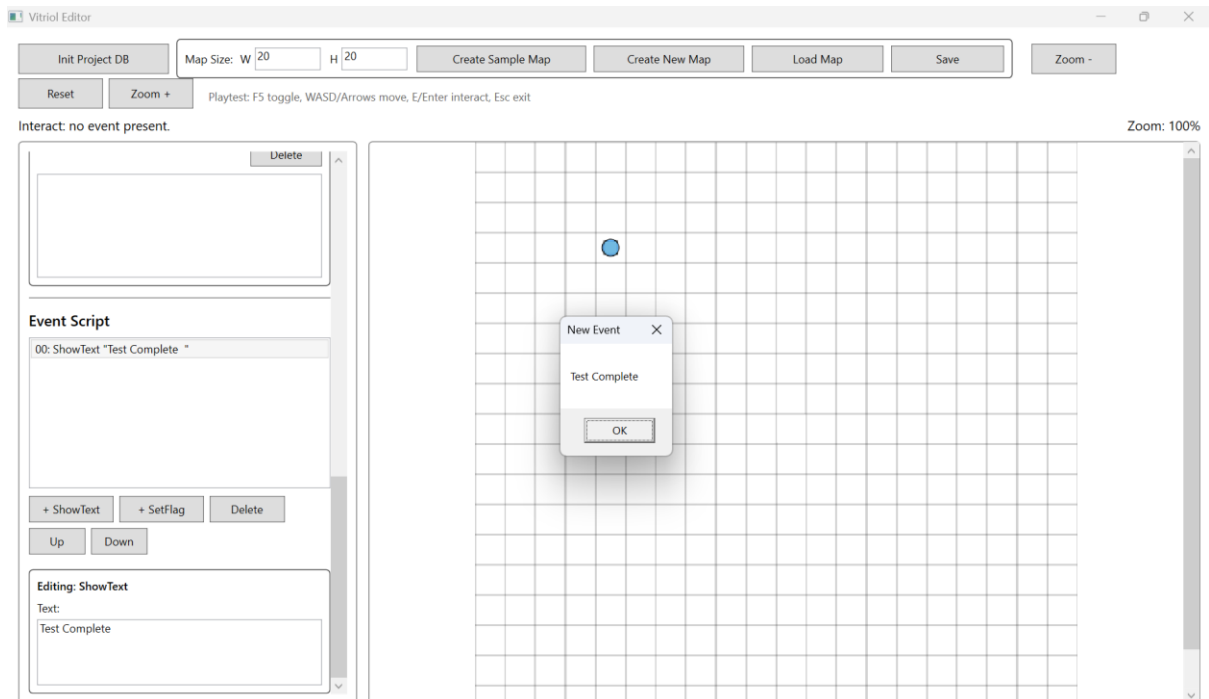


Figure T71 – NPC Interaction Event Execution

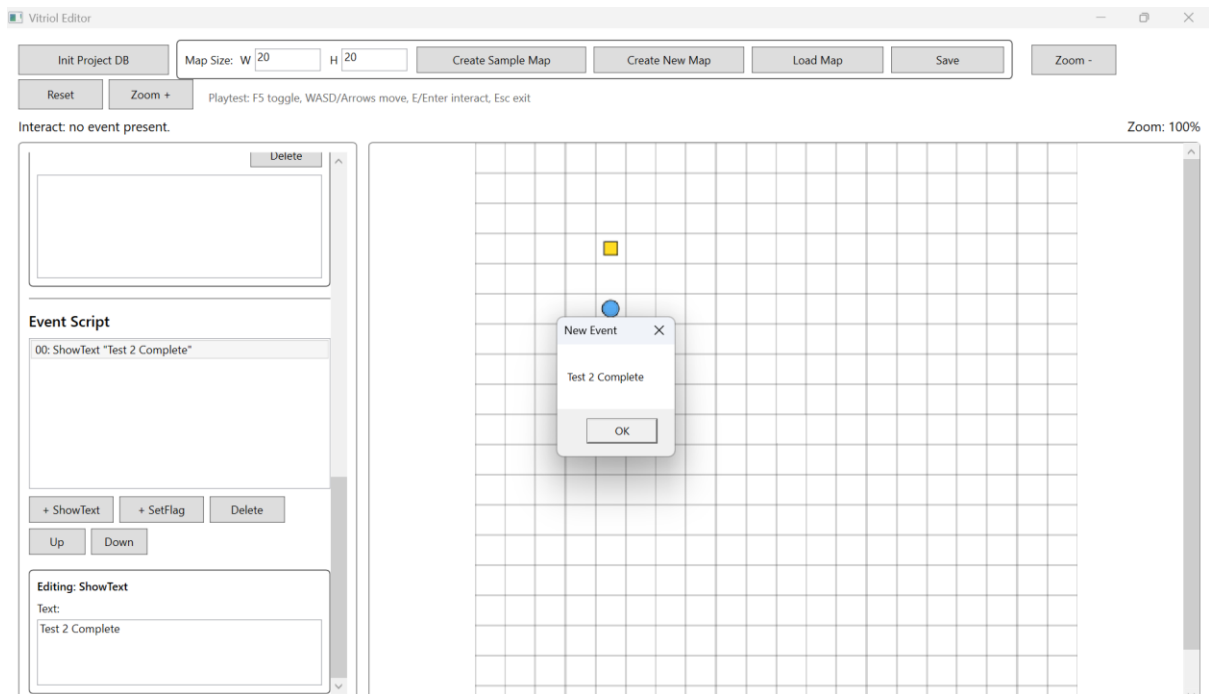
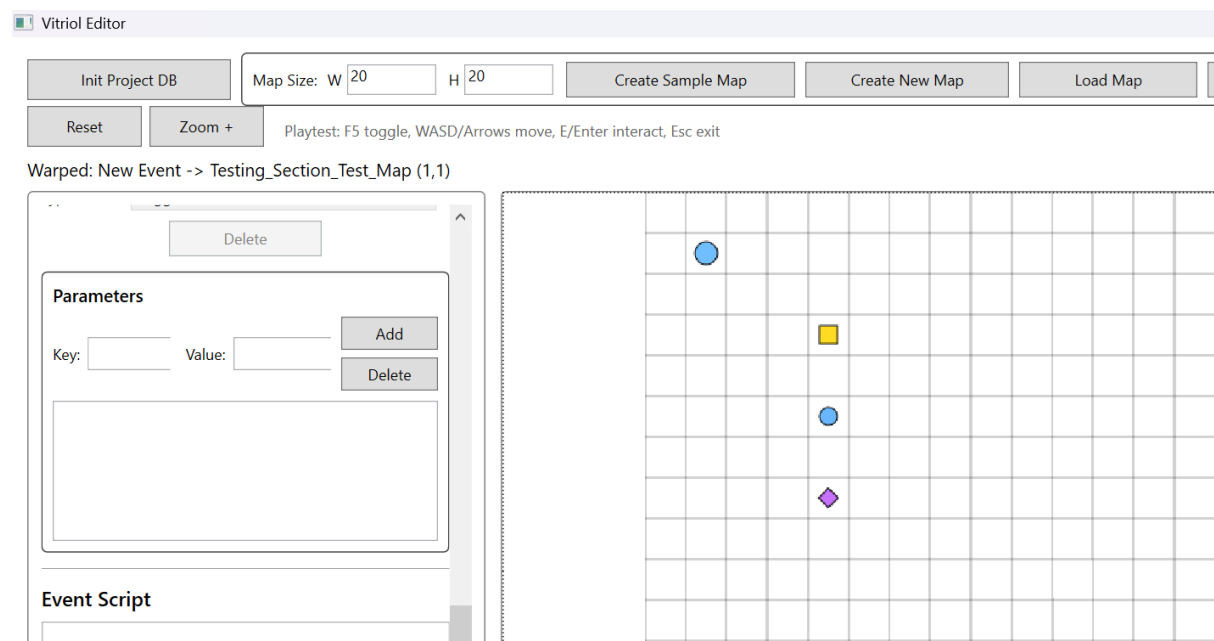


Figure T72 – Warp Event Map Transition



What These Tests Demonstrate:

- Trigger event still executes correctly after later runtime changes
- NPC events remain functional and correctly routed through interaction logic
- Warp loads the destination map and relocates the player
- Command dictionary logic still behaves correctly after refactors

5.3 Persistence Regression Checks

Persistence regression ensures later features did not introduce corruption into save/load behaviour. This includes JSON integrity, map reload fidelity, and ensuring that edits are correctly written back to file without breaking schema stability.

Test Table 3 – Persistence Regression Checks

Test ID	Description	Expected Outcome	Actual Outcome	Status	Screenshot ID
REGPS1	Save after tile edits	JSON updates reflect tile changes	Saved correctly	Pass	Figures T73
REGPS2	Save after collision edits	Collision layer persists correctly	Saved correctly	Pass	Figures T74
REGPS3	Event list persists with correct fields	Event list persists with correct fields	Saved correctly	Pass	Figures T75.1, T75.2
REGPS4	Runtime loads identical state	Runtime loads identical state	Reload matches saved state	Pass	Figures T76

Test Table 3 – Screenshot Evidence

Figure T73 – Tile Edits Persist After Save

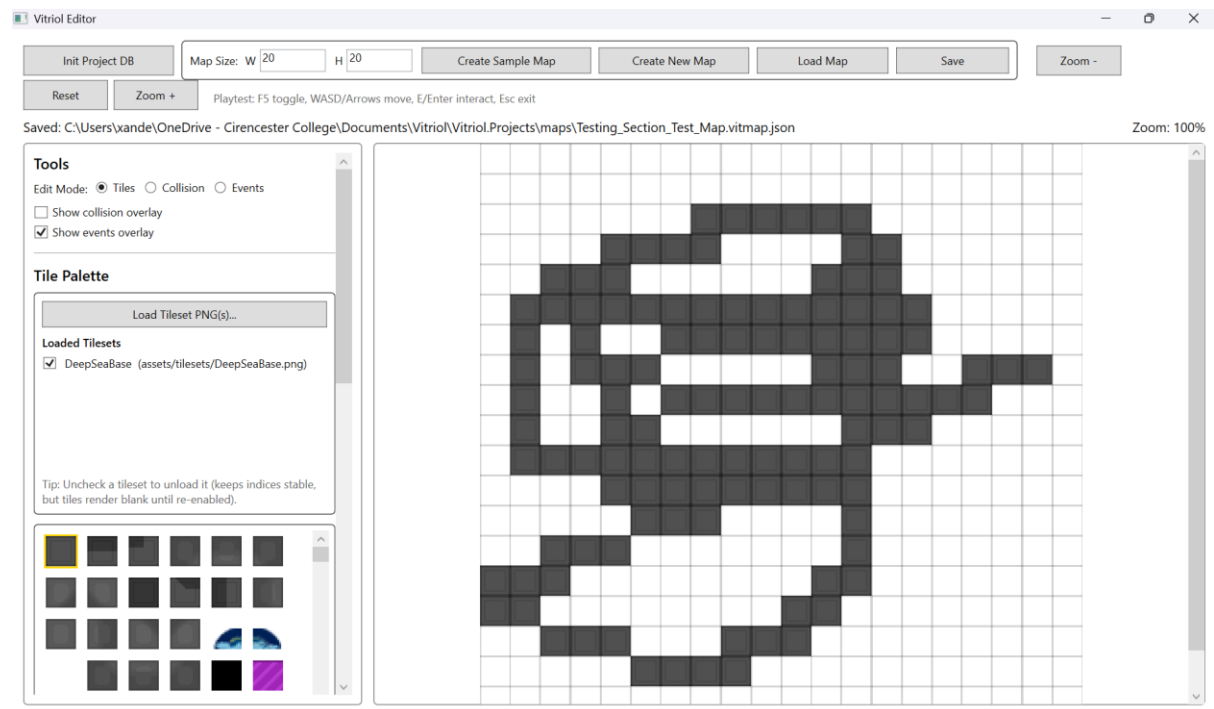


Figure T74 – Collision Edits Persist After Save

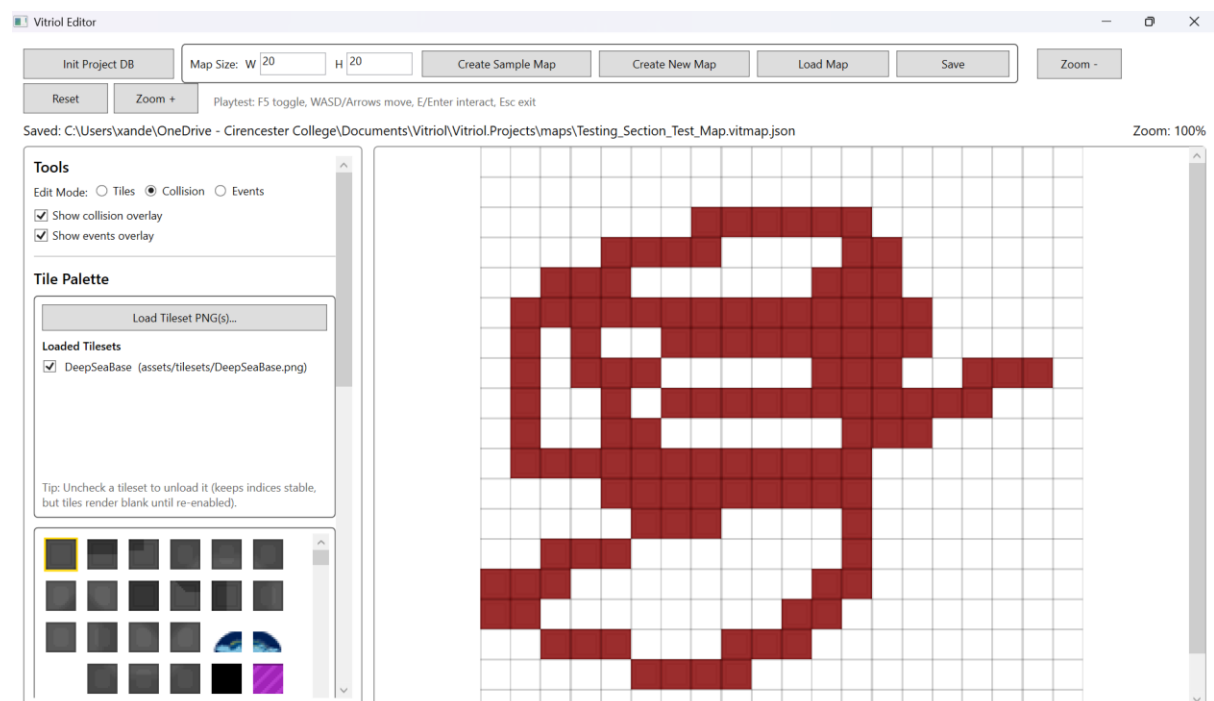


Figure T75.1 – Event Added and Saved

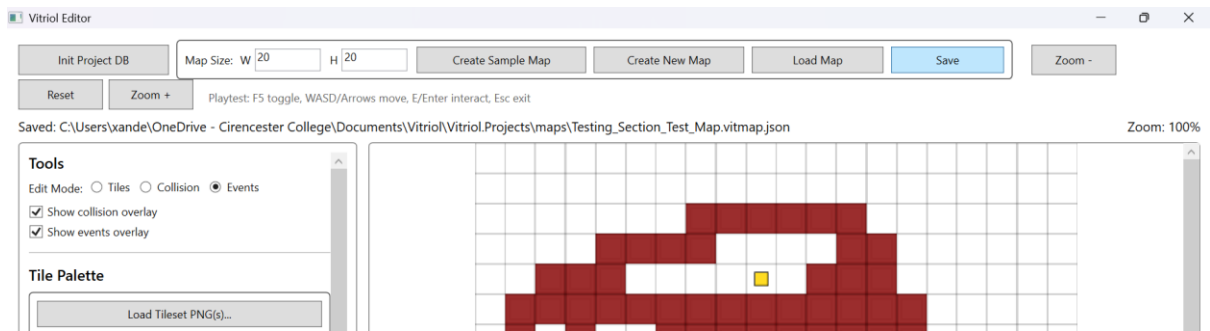


Figure T75.2 – Reload Event From Saved JSON

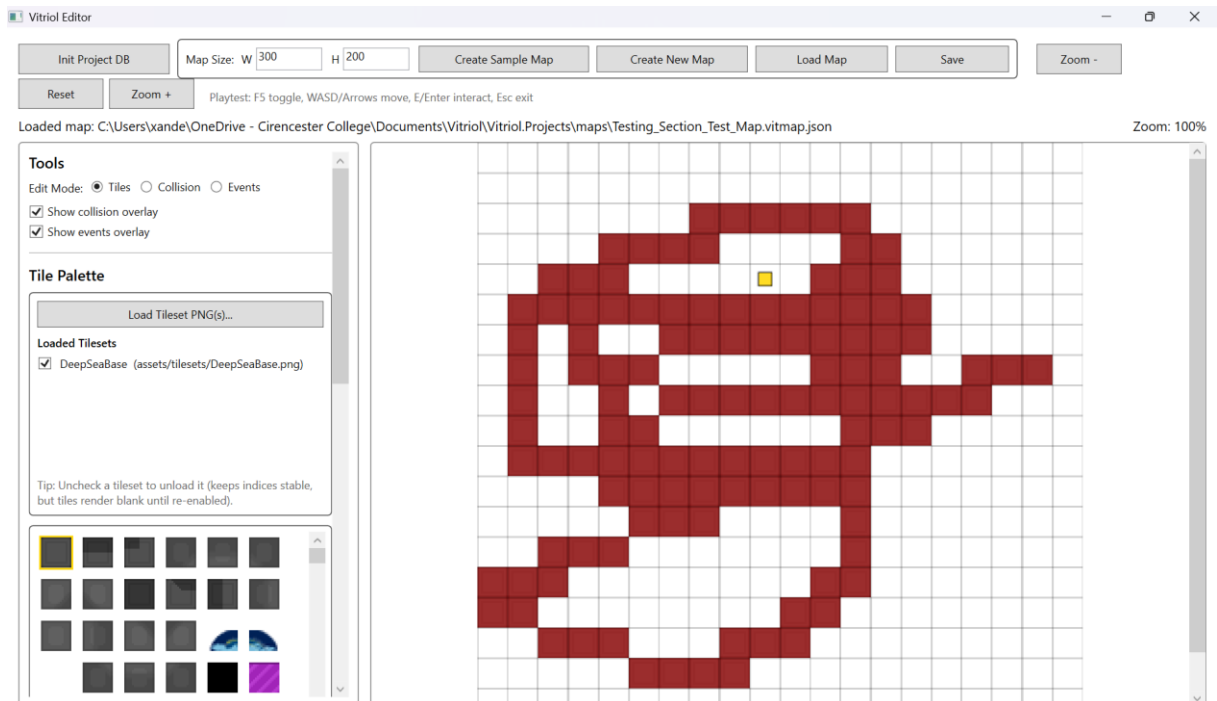
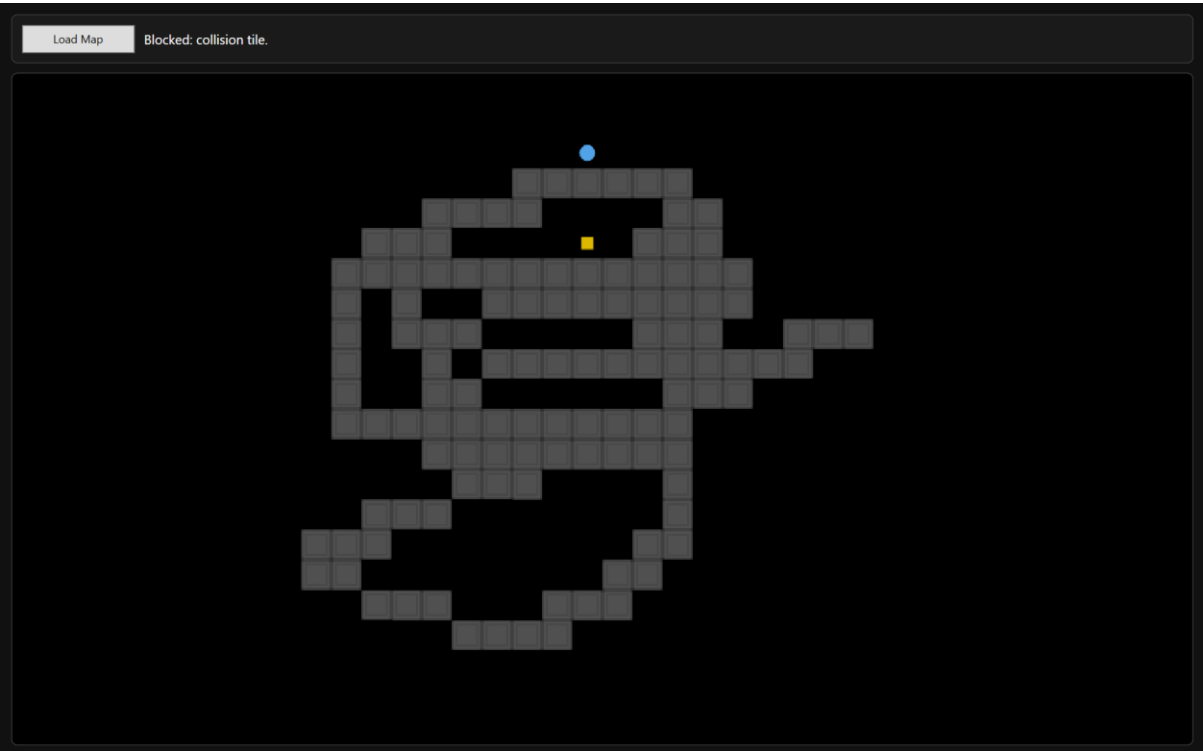


Figure T76 – Runtime Loads Saved Map Without Drift



What These Tests Demonstrate:

- Tile painting still produces correct saved output
- Collision layer remains stable through persistence pipeline
- Event created and stored correctly
- Event data reloads correctly and remains executable
- Runtime load fidelity after persistence operations

5.4 Rendering Regression Checks

Rendering regression checks confirm that viewport-only rendering, caching, and tileset compilation changes did not cause tile misalignment, incorrect indices, missing tiles, or rendering artifacts.

Test Table 4 – Rendering Regression Checks

Test ID	Description	Expected Outcome	Actual Outcome	Status	Screenshot ID
REGRN1	Render map after tileset caching added	Correct tile alignment	Alignment correct	Pass	Figures T77
REGRN2	Scroll viewport across map	No blank regions or tearing	Scroll Stable	Pass	Figures T78
REGRN3	Render events overlay markers	Events visible in viewport	Markers correct	Pass	Figures T79

REGRN4	Render player marker + deadzone camera	Player visible, camera clamps	Stable behaviour	Pass	Figures T80
---------------	--	-------------------------------	------------------	------	-------------

Test Table 4 – Screenshot Evidence

Figure T77 – Correct Tile Alignment After Rendering Changes

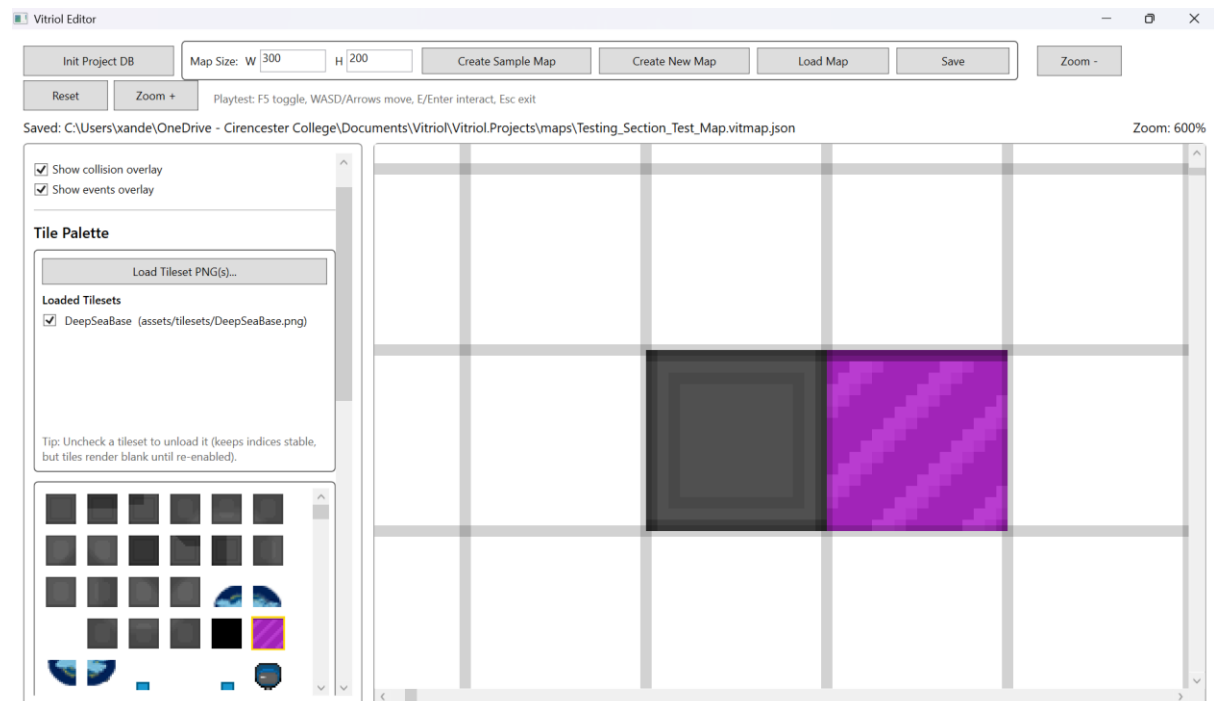


Figure T78 – Viewport Scroll Rendering Stability

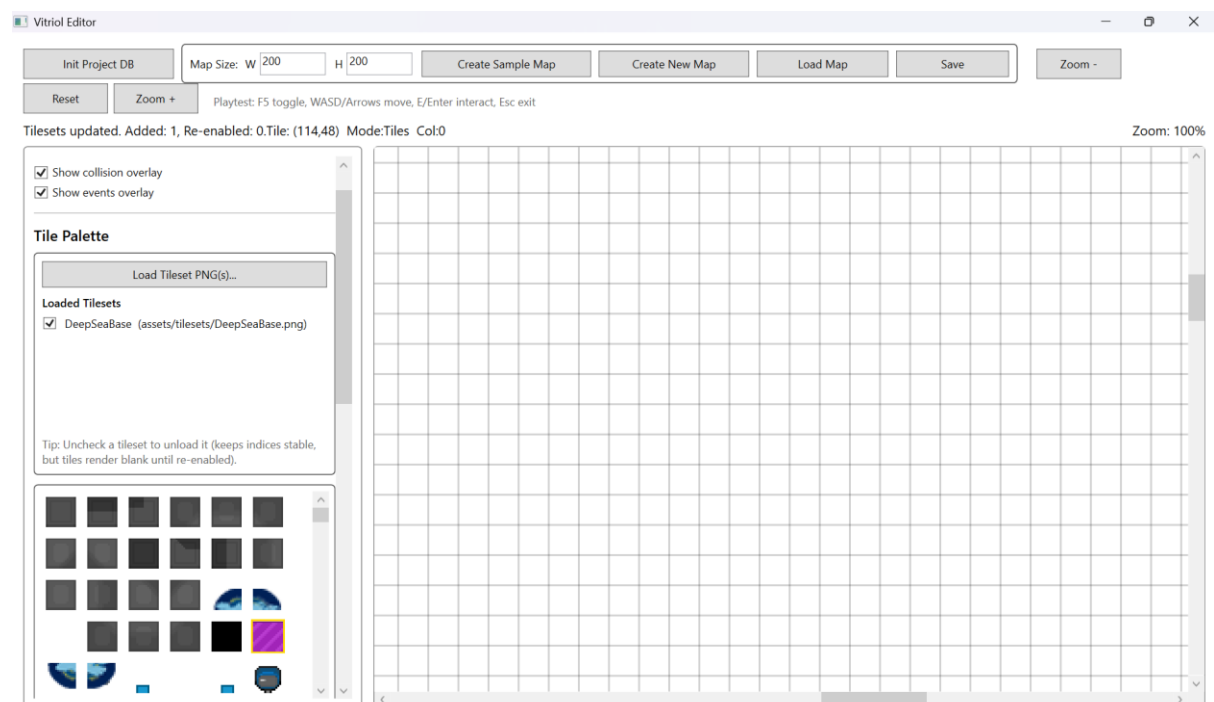


Figure T79 – Event Markers Render Correctly

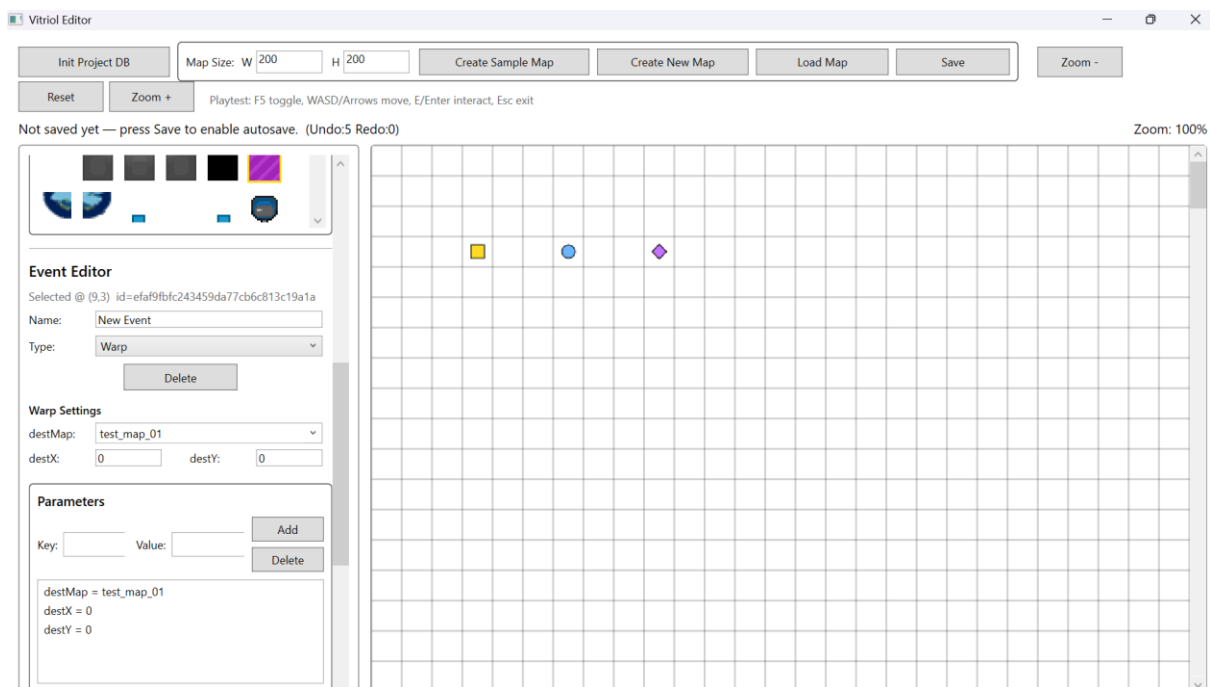
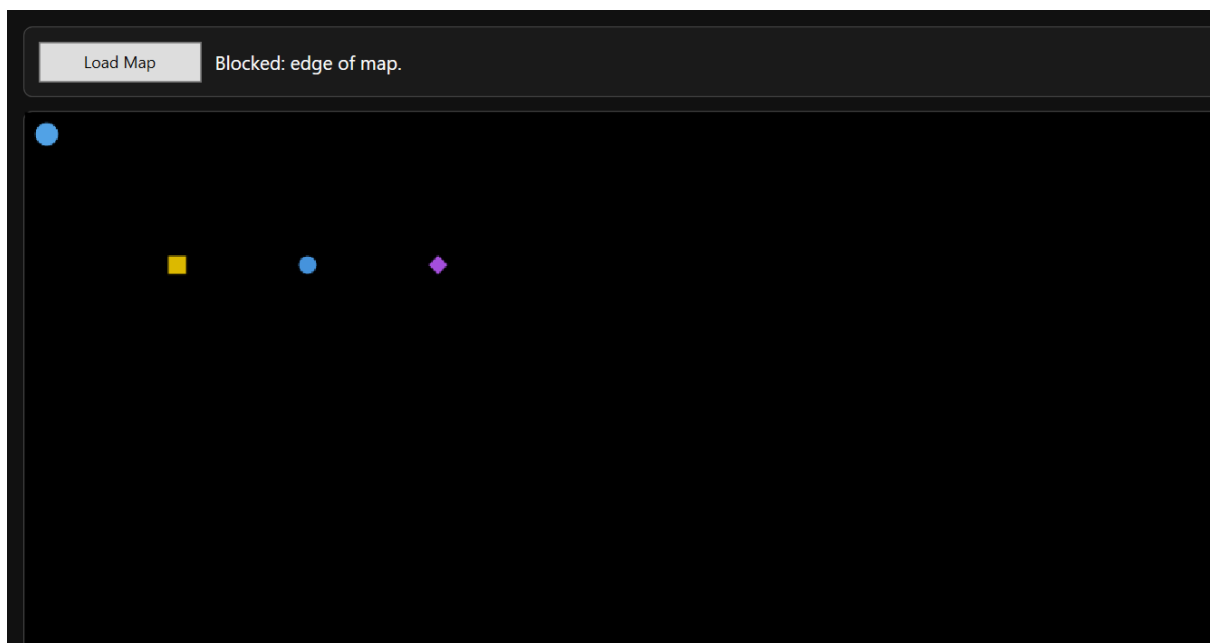


Figure T80 – Player Marker + Camera Rendering Stable



What These Tests Demonstrate:

- Stable tile positioning and correct cropping
- Viewport-only rendering produces consistent redraw output
- Event overlay rendering still functions after pipeline changes
- Camera follow + player rendering remain consistent

6 User Testing

6.1 Structured User Task Testing

To evaluate usability and workflow clarity, structured task-based testing was conducted. The aim was to determine whether a user unfamiliar with the internal implementation could:

- Create and save a new map
- Paint tiles correctly
- Add an event
- Trigger the event in runtime
- Navigate the map using movement controls

This testing focused on:

- Learnability
- Workflow intuitiveness
- Error recovery
- Feature discoverability

Two participants with basic computer competence but no prior exposure to the codebase were observed performing a predefined task list.

Task List

Participants were asked to complete the following asks without assistance until blocked:

- Create a new map (32x32)
- Save the map with a unique name
- Load a tileset
- Paint at least three different tiles
- Add an NPC event with a ShowText command
- Save the map
- Launch runtime
- Load the created map
- Move the player to the NPC
- Trigger the event interaction

These tasks were chosen to cover:

- Core editor workflow
- Persistence
- Event configuration
- Runtime execution

- Basic navigation controls

Feedback Summary

Test User 1

User 1 completed all tasks successfully without intervention

Observations:

- Immediately understood tile painting
- Located save button quickly
- Required brief hesitation when locating event creation tools
- Successfully triggered NPC interaction in runtime

User reported:

- Interface felt “minimal but clear”
- Runtime controls were intuitive (WASD)
- Load Map button in runtime was helpful

No critical usability failures occurred

Test User 2

User 2 also completed all tasks successfully

Observations:

- Took slightly longer to locate event editing panel
- Initially attempted to interact with NPC without standing on the tile
- Successfully corrected interaction attempt without assistance

User reported:

- Map creation was straightforward
- Event editing felt “more technical” than tile painting
- Runtime felt responsive

No blocking errors occurred

User Final Result Screenshot

Test User 1

Figure T81.1 – Test User 1 Final Editor

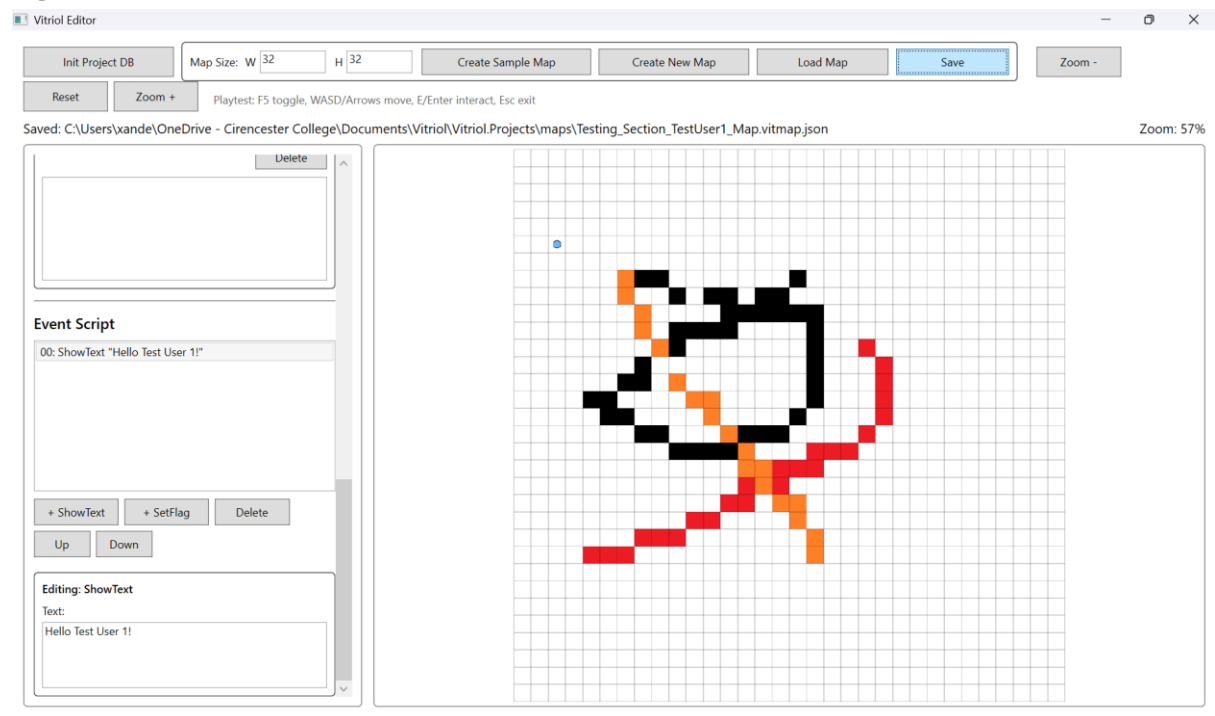
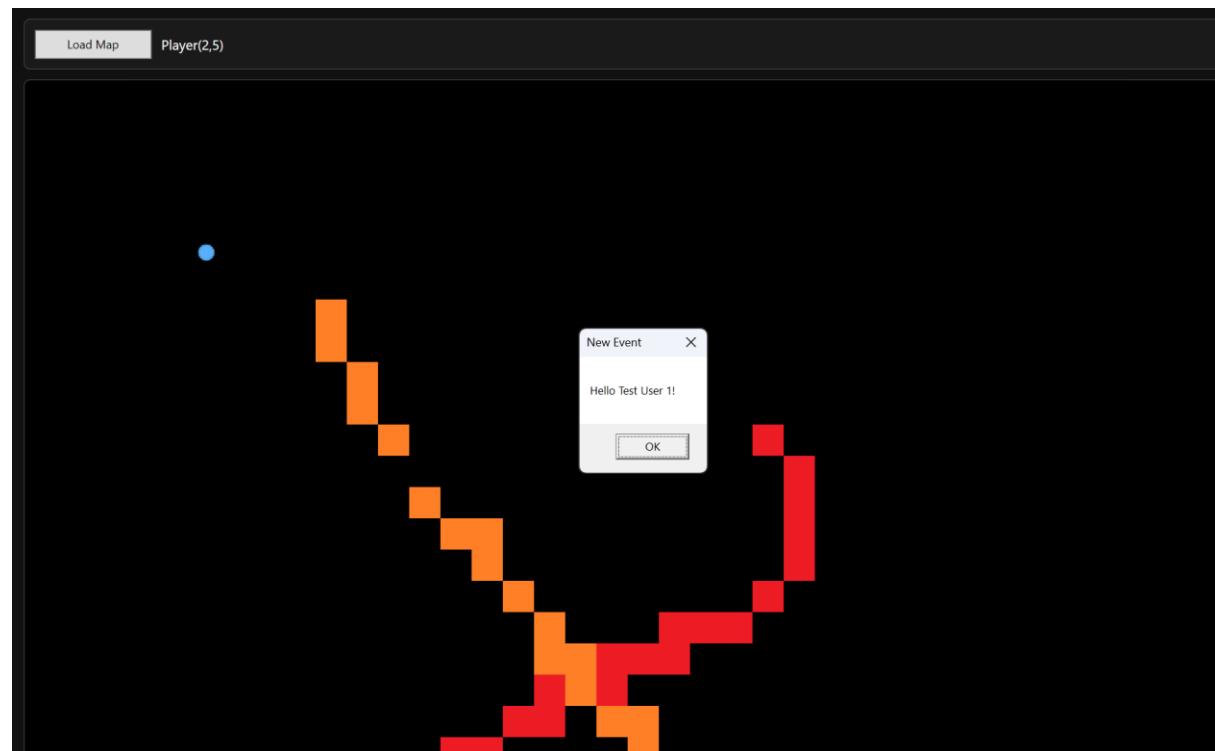


Figure T81.2 - Test User 1 Final Runtime



Test User 2

Figure T82.1 – Test User 2 Final Editor

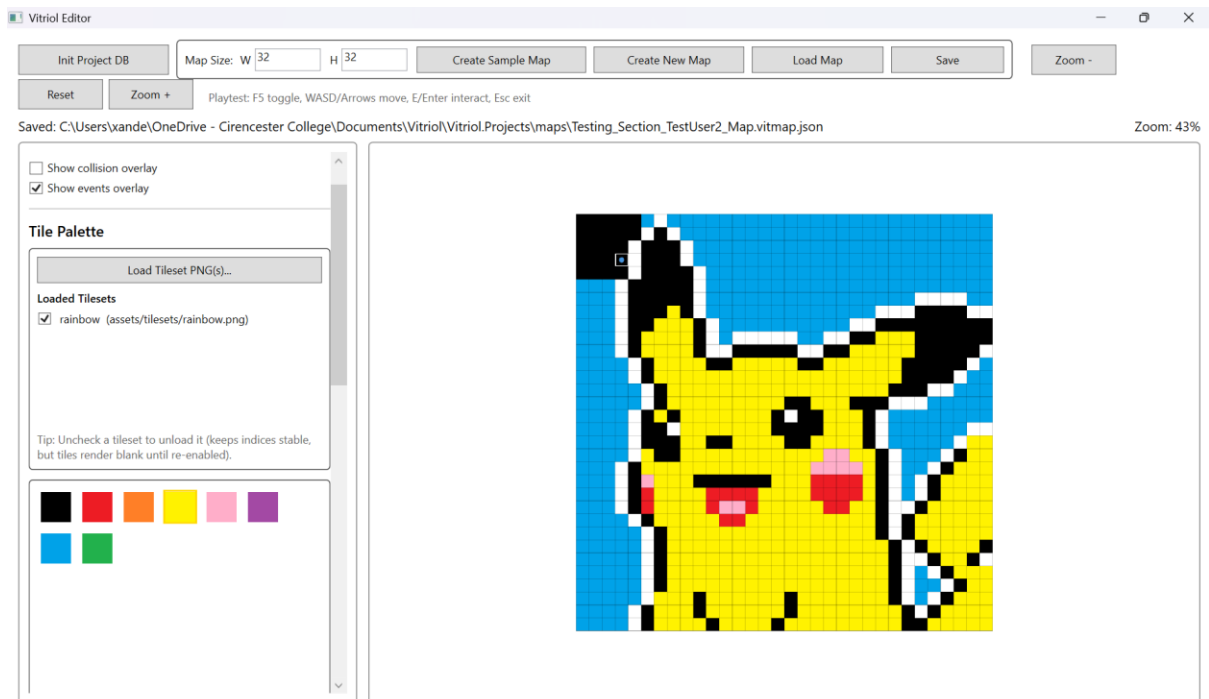
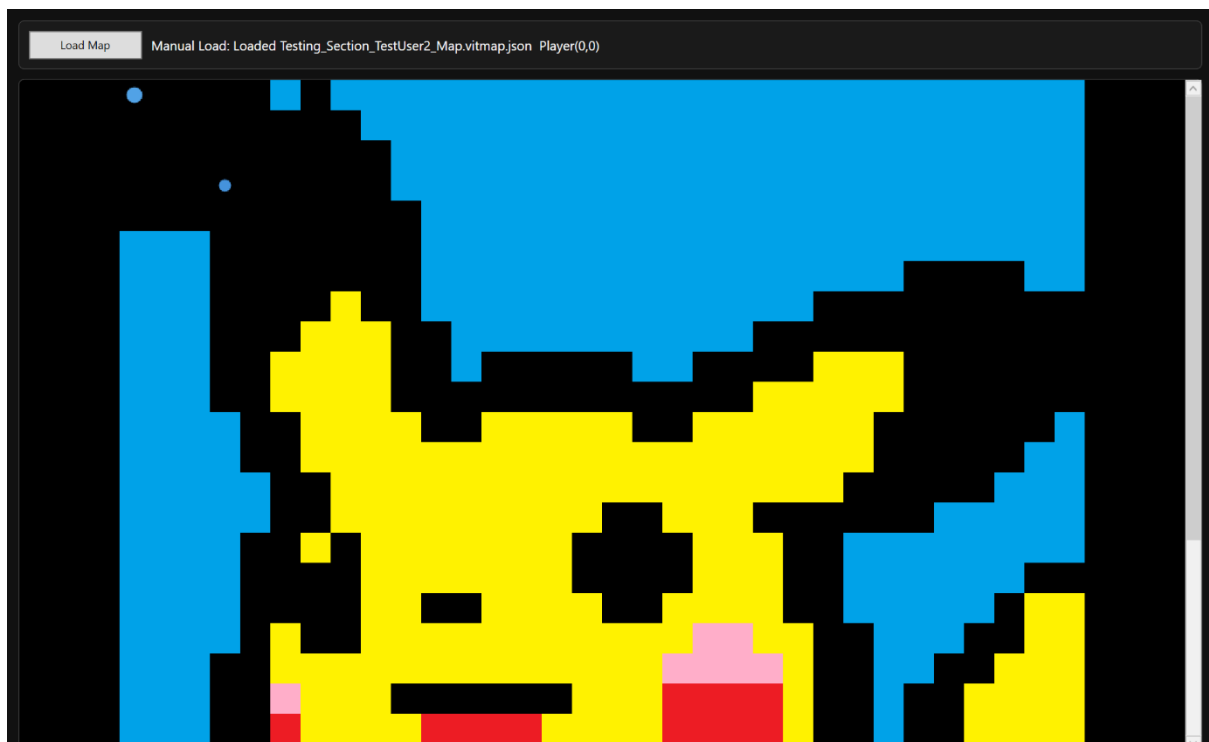


Figure T82.2 – Test User 2 Final Runtime



6.2 Observed Error Patterns

Although both users completed all tasks, there were several minor patterns observed:

- Event tools were less discoverable than tile painting tools
- Users initially attempted to interact with NPCs from adjacent tiles

- Some hesitation occurred when distinguishing between saving and autosaving behaviour
- User expected Eter to trigger interaction consistently

These observations indicate that while the system is functionally stable, minor usability refinements could improve first-time experience

7 Witness Statement

7.1 Witness Confirmation of Functionality

I confirm that I observed the Vitriol editor and runtime being used to create and test a playable map. The developer demonstrated map creation, tile editing, event placement, saving, and loading the map into the runtime environment.

I observed successful player movement, event triggering, and correct rendering of the map. The system appeared stable, and responsive throughout the demonstration.

Based on what I observed, the features shown were working as described and no functional issues were evident during testing

Test User 1

Signature: Michael Christopher Payne

Date: 27/02/2026

Test User 2

Signature: Rachael Evans

Date: 27/02/2026

7.2 Summary of Demonstrated Features

Test User 1

During the demonstration, the following features were shown:

- Creation of a new tile-based map
- Manual save and autosave behaviour
- Loading and referencing of tilesets
- Tile painting and editing
- Placement of NPC event with ShowText command
- Saving to JSON format
- Runtime loading of saved map
- Player movement with collision handling
- NPC interaction triggering dialogue
- Camera following player with boundary clamping

All features appeared to function as intended.

Test User 2

The following functionality was demonstrated:

- Structured map creation workflow
- Editing of map layers
- Event placement and configuration
- Persistence of map data across reload
- Viewport-only rendering in both editor and runtime
- Runtime interaction using keyboard input
- Event execution behaviour

The system operated consistently during the observed session

8 Testing Summary and Coverage

8.1 Requirement Coverage Mapping

Coverage Table

The table below summarises how each core functional requirement has been validated through structured testing:

ID	Requirement Description	Validated By	Evidence Figures
FR1	Create new maps with defined dimensions and initialise layered structure	2.1, 5.1	T1-T6, T66-69
FR2	Visually paint, erase, and modify tile data within a grid-based system	2.2, 3.2	T7-T11, T34-T38
FR3	Collision Editing – Define walkable and blocked tiles via a dedicated collision layer	3.2, 5.2	T39-T41, T71
FR4	Event Placement – Add, edit, and remove position based event triggers on the map	2.4, 5.2	T16-T20, T70-T72
FR5	Map Persistence – Save and load maps using structured JSON with format versioning	2.4, 5.3	T16-T22, T73-T76
FR6	Runtime Playback – Load maps into runtime and render with camera-based viewport logic	3.1, 3,4	T23-T27, T42-T47
FR7	Player Movement & Collision – Handle deterministic grid-based collision detection	3.2, 5.2	T29-T41, T71
FR8	Editing History – Implement command-based Undo/Redo across tile, collision, and events	2.2	T8-T11
FR9	Autosave Control – Activate controlled autosave after initial manual save	2.1, 5,1	T4-T6, T67-T68

FR10	Performance stability – Maintain approximately 60 FPS on target specification hardware	4	T48-T65
FR11	Relative Asset Referencing – Store tileset paths relative to project directory	3.1	T28-T32
FR12	Layer Rendering Order – Enforce correct drawing hierarchy in editor and runtime	3.4, 5.4	T42-T47, T77

Coverage Analysis

The coverage mapping demonstrates that:

- All major functional requirements were validated through structured subsystem testing
- Regression testing confirmed stability following future expansion
- No core requirement remains untested
- Each requirement is supported by at least one visual evidence reference

This confirms that the implementation aligns with the functional objectives defined in the analysis section.

8.2 Non-Functional Requirement Validation

Performance

Performance testing (Section 4) demonstrated that:

- Viewport-only rendering reduces rendering workload
- Large maps (200x200) remain usable without freezing
- Continuous player movement does not cause degradation
- Tileset caching prevents repeated image decoding overhead

Evidence: T48 – T65

The system maintains stable responsiveness during both typical and extended usage scenarios.

Reliability

Reliability was validated through:

- JSON validation checks preventing malformed loads
- Defensive handling of missing event parameters
- Collision enforcement preventing invalid state transitions
- Regression testing confirming no feature breakage

Evidence: T20.1-T20.2, T70-T76

The application operates without crashes under normal usage and handles invalid inputs safely.

Usability

Usability validation was conducted through structured user testing (Section 6) demonstrating:

- Successful independent task completion by participants
- Minimal intervention required
- Clear map editing workflow
- Intuitive runtime movement controls

Observed minor discoverability issues did not prevent task completion.

Evidence: T81.1 – T82.2

The system is usable by individuals unfamiliar with the codebase, satisfying the intended user accessibility objective.

8.3 Final Conclusion

Across all testing categories:

- Subsystem testing validated individual components
- Boundary testing confirmed safe handling of abnormal input
- Performance testing demonstrated stable execution under load
- Regression testing confirmed stability after feature additions
- User testing validated real-world usability
- Witness confirmation provided independent observation.

The evidence demonstrates that the system meets the stated functional and non-functional requirements and behaves consistently under expected operating conditions.